Name :B.Tejaswini

Reg no :23BDS0237

Course Name : Exploratory Data Analysis

Cousre code : BCSE331L

Slot : A2+TA2

Date of submission : 19-10-2026

Dataset : TravelMode.csv

https://raw.githubusercontent.com/salemprakash/EDA/main/Data/TravelMode.csv

```
rownames,individual,mode,choice,wait,vcost,travel,gcost,income,size
1,1,air,no,69,59,100,70,35,1
2,1,train,no,34,31,372,71,35,1
3,1,bus,no,35,25,417,70,35,1
4,1,car,yes,0,10,180,30,35,1
5,2,air,no,64,58,68,68,30,2
6,2,train,no,44,31,354,84,30,2
7,2,bus,no,53,25,399,85,30,2
8,2,car,yes,0,11,255,50,30,2
9,3,air,no,69,115,125,129,40,1
10,3,train,no,34,98,892,195,40,1
11,3,bus,no,35,53,882,149,40,1
12,3,car,yes,0,23,720,101,40,1
13,4,air,no,64,49,68,59,70,3
14,4,train,no,44,26,354,79,70,3
```

Aim :

To perform Exploratory Data Analysis (EDA) on the TravelMode dataset, apply data cleaning and preprocessing, visualize patterns, perform clustering and dimensionality reduction, and build regression and classification models to analyze travel behavior and evaluate model performance using appropriate metrics.

Detailed description:

pandas handles tabular data, numpy supports numerical operations, and matplotlib and seaborn create graphs. You normally need to install these libraries only once in your Jupyter environment.

In [29]:
```
# Install and import the required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns

sns.set_theme(style="whitegrid")

print("All libraries imported successfully.")
```

```
All libraries imported successfully.
```

Detailed description:

pd.read_csv() downloads the CSV from the URL and stores it as a pandas DataFrame named df. This variable will be used throughout the remaining modules, so keep the same name.

In [12]:
```
# Load the TravelMode dataset
url = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/TravelMode.c
df = pd.read_csv(url)
print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)
```

```
Dataset loaded successfully!
Dataset shape: (840, 10)
```

Detailed description:

The head() function helps you understand how the data is organized and whether values are being read correctly.

In [11]:
```
# Display the first five rows
print("First five rows of the TravelMode dataset:")
display(df.head())
```

```
First five rows of the TravelMode dataset:
```

	rownames	individual	mode	choice	wait	vcost	travel	gcost	income	size
0	1	1	air	no	69	59	100	70	35	1
1	2	1	train	no	34	31	372	71	35	1
2	3	1	bus	no	35	25	417	70	35	1
3	4	1	car	yes	0	10	180	30	35	1
4	5	2	air	no	64	58	68	68	30	2

Detailed description:

The tail() function shows the end of the dataset. This helps verify that the file was read through to its final rows.

In [12]:
```
# Display the last five rows
print("Last five rows of the TravelMode dataset:")
```

```
display(df.tail())
```

Last five rows of the TravelMode dataset:

	rownames	individual	mode	choice	wait	vcost	travel	gcost	income	size
835	836	209	car	yes	0	27	510	82	20	1
836	837	210	air	no	64	66	140	87	70	4
837	838	210	train	no	44	54	670	156	70	4
838	839	210	bus	no	53	33	664	134	70	4
839	840	210	car	yes	0	12	540	94	70	4

Detailed description:

shape[0] gives the number of rows, shape[1] gives the number of columns, and multiplying them gives the total number of cells.

In [13]:
```
# Count rows, columns and total cells
rows = df.shape[0]
columns = df.shape[1]
total_cells = rows * columns

print("Total rows:", rows)
print("Total columns:", columns)
print("Total cells:", total_cells)
```

```
Total rows: 840
Total columns: 10
Total cells: 8400
```

Detailed description:

The columns.tolist() function converts the DataFrame column names into a readable Python list. We need this list to identify which variables are categorical, numerical and identifiers.

In [14]:
```
# Display all column names
print("Column names in the dataset:")
for number, column in enumerate(df.columns, start=1):
    print(f"{number}. {column}")
```

```
Column names in the dataset:
1. rownames
2. individual
3. mode
4. choice
5. wait
6. vcost
7. travel
8. gcost
9. income
10. size
```

Detailed description:

df.info() reports the number of entries, non-null values, data types and memory usage. This helps identify columns stored as text when you expected numerical values.

In [15]:
```
#Check data types and non-missing counts
print("Dataset information:")
df.info()
```

```
Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 840 entries, 0 to 839
Data columns (total 10 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   rownames    840 non-null    int64
 1   individual  840 non-null    int64
 2   mode        840 non-null    str
 3   choice      840 non-null    str
 4   wait        840 non-null    int64
 5   vcost       840 non-null    int64
 6   travel      840 non-null    int64
 7   gcost       840 non-null    int64
 8   income      840 non-null    int64
 9   size        840 non-null    int64
dtypes: int64(8), str(2)
memory usage: 65.8 KB
```

Detailed description:

Missing values may affect summaries and models. Duplicate rows may indicate repeated records, but repeated individual values are expected because one traveller can have multiple travel alternatives.

In [16]:
```
#Check missing values and duplicates
missing_counts = df.isnull().sum()
duplicate_count = df.duplicated().sum()

print("Missing values in each column:")
display(missing_counts.rename("Missing values").to_frame())

print("Total missing cells:", df.isnull().sum().sum())
print("Exact duplicate rows:", duplicate_count)
```

```
Missing values in each column:
```

	Missing values
rownames	0
individual	0
mode	0
choice	0
wait	0
vcost	0
travel	0
gcost	0
income	0
size	0

```
Total missing cells: 0
Exact duplicate rows: 0
```

Detailed description:

EDA is the process of examining data before applying statistical methods or machine learning. It helps us understand data quality, distributions, relationships and unusual observations.

In [17]:
```
# Understand the purpose of EDA
eda_steps = pd.DataFrame({
    "Step": [
        1, 2, 3, 4, 5, 6, 7
    ],
    "EDA activity": [
        "Load the dataset",
        "Inspect rows, columns and data types",
        "Check missing values and duplicates",
        "Summarize numerical and categorical variables",
        "Visualize distributions and relationships",
        "Identify outliers and correlations",
        "Prepare data for further analysis"
    ],
    "Purpose": [
        "Access the data",
        "Understand its structure",
        "Check data quality",
        "Understand variable values",
        "Discover patterns",
        "Find unusual values and associations",
        "Support later analysis and modelling"
    ]
})

display(eda_steps)
```

	Step	EDA activity	Purpose
0	1	Load the dataset	Access the data
1	2	Inspect rows, columns and data types	Understand its structure
2	3	Check missing values and duplicates	Check data quality
3	4	Summarize numerical and categorical variables	Understand variable values
4	5	Visualize distributions and relationships	Discover patterns
5	6	Identify outliers and correlations	Find unusual values and associations
6	7	Prepare data for further analysis	Support later analysis and modelling

Detailed description:

describe() summarizes numerical variables. The mean measures the average, the standard deviation measures spread, and the quartiles show the distribution of the middle observations.

In [18]:
```
# Statistical summary of numerical columns
print("Statistical summary of numerical columns:")
display(df.describe().T.round(2))
```

```
Statistical summary of numerical columns:
```

	count	mean	std	min	25%	50%	75%	max
rownames	840.0	420.50	242.63	1.0	210.75	420.5	630.25	840.0
individual	840.0	105.50	60.66	1.0	53.00	105.5	158.00	210.0
wait	840.0	34.59	24.95	0.0	0.75	35.0	53.00	99.0
vcost	840.0	47.76	32.37	2.0	23.00	39.0	66.25	180.0
travel	840.0	486.17	301.44	63.0	234.00	397.0	795.50	1440.0
gcost	840.0	110.88	47.98	30.0	71.00	101.5	144.00	269.0
income	840.0	34.55	19.68	2.0	20.00	34.5	50.00	72.0
size	840.0	1.74	1.01	1.0	1.00	1.0	2.00	6.0

Detailed description:

This step uses pandas to list numeric and non-numeric columns. We must then interpret their meaning, because a numerical identifier is not necessarily a numerical feature.

In [19]:
```
#Identify numerical and categorical columns
numeric_columns = df.select_dtypes(include=np.number).columns.tolist()
categorical_columns = df.select_dtypes(
    include=["object", "category", "bool", "string"]
).columns.tolist()
```

```
print("Numerical columns:")
print(numeric_columns)

print("\nCategorical or text columns:")
print(categorical_columns)
```

```
Numerical columns:
['rownames', 'individual', 'wait', 'vcost', 'travel', 'gcost', 'income', 'size']

Categorical or text columns:
['mode', 'choice']
```

Detailed description:

Discrete data represent countable values; continuous data represent measurements; categorical data represent labels or groups. Identifiers are handled separately because arithmetic on their values usually has no analytical meaning.

In [22]:
```
#Discrete, continuous and categorical
data_type_notes = {
    "rownames": ("Identifier", "Exported row index; generally exclude from analy
    "individual": ("Discrete identifier", "Traveller ID; multiple rows can belon
    "mode": ("Categorical", "Travel alternative such as air, train, bus or car")
    "choice": ("Categorical binary", "Indicates whether the alternative was sele
    "wait": ("Numerical measurement", "Waiting-time variable"),
    "vcost": ("Numerical measurement", "Vehicle or travel-cost variable"),
    "travel": ("Numerical measurement", "Travel-time variable"),
    "gcost": ("Numerical measurement", "Generalized-cost variable"),
    "income": ("Numerical measurement", "Income variable"),
    "size": ("Discrete/count", "Group or household size variable")
}

rows = []
for column in df.columns:
    if column in data_type_notes:
        conceptual_type, meaning = data_type_notes[column]
    else:
        conceptual_type, meaning = "Needs inspection", "Check the dataset docume
    rows.append([column, str(df[column].dtype), conceptual_type, meaning])

type_table = pd.DataFrame(
    rows,
    columns=["Column", "Stored data type", "Conceptual type", "Meaning"]
)

display(type_table)
```

	Column	Stored data type	Conceptual type	Meaning
0	rownames	int64	Identifier	Exported row index; generally exclude from ana...
1	individual	int64	Discrete identifier	Traveller ID; multiple rows can belong to one ...
2	mode	str	Categorical	Travel alternative such as air, train, bus or car
3	choice	str	Categorical binary	Indicates whether the alternative was selected
4	wait	int64	Numerical measurement	Waiting-time variable
5	vcost	int64	Numerical measurement	Vehicle or travel-cost variable
6	travel	int64	Numerical measurement	Travel-time variable
7	gcost	int64	Numerical measurement	Generalized-cost variable
8	income	int64	Numerical measurement	Income variable
9	size	int64	Discrete/count	Group or household size variable

Detailed description:

Frequency counts reveal which categories appear most often. The code first checks that the expected columns exist, so it will not fail if your dataset uses different column names.

In [27]:
```
#Frequency distribution of categorical variables
for column in ["mode", "choice"]:
    if column in df.columns:
        print(f"\nFrequency distribution for {column}:")
        display(
            df[column]
            .value_counts(dropna=False)
            .rename_axis(column)
            .to_frame("Frequency")
        )
    else:
        print(f"Column '{column}' was not found.")
```

```
Frequency distribution for mode:
```

mode	Frequency
air	210
train	210
bus	210
car	210

```
Frequency distribution for choice:
```

choice	Frequency
no	630
yes	210

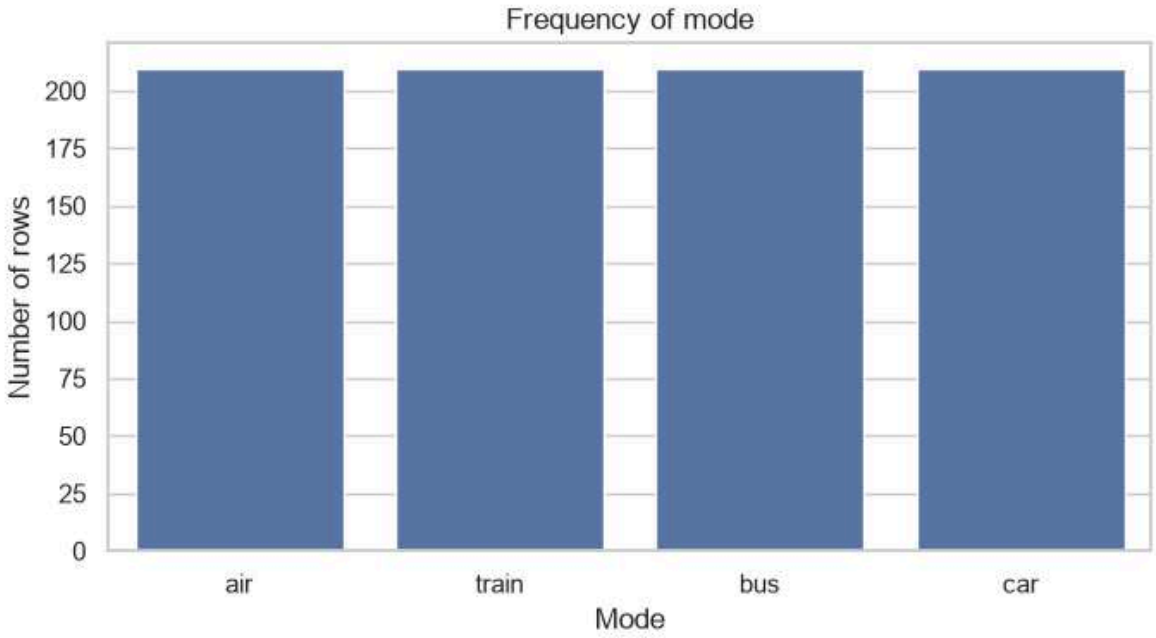
Detailed description:

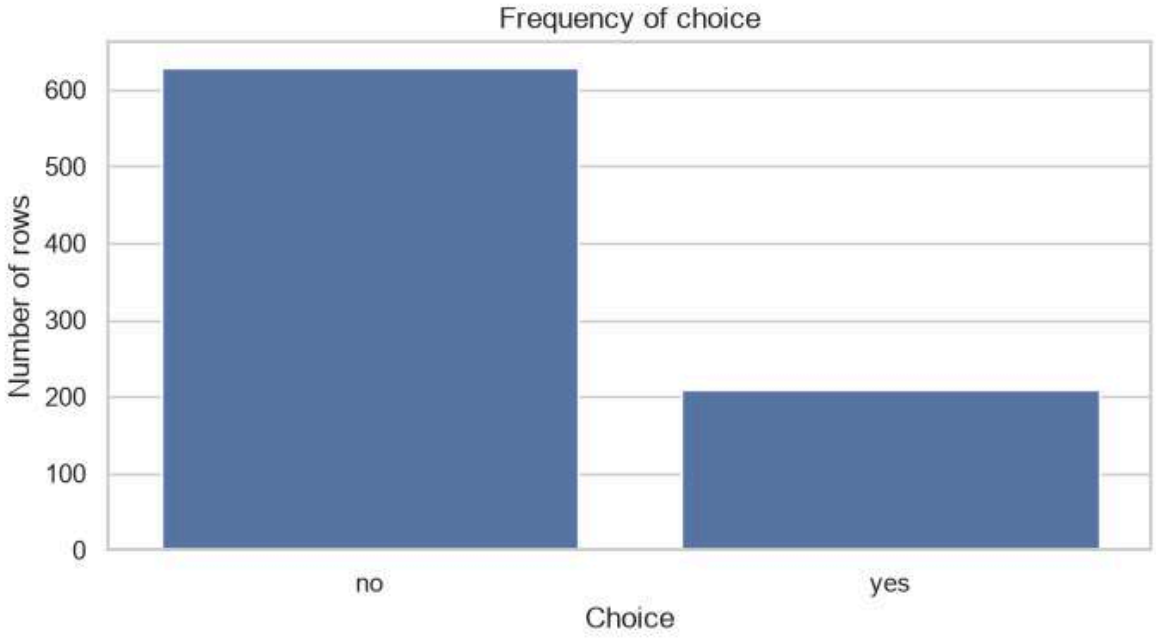
Bar charts make categorical distributions easier to compare. The height of each bar represents the number of rows in that category.

In [31]:
```
# Visualize categorical data with bar charts
for column in ["mode", "choice"]:
    if column in df.columns:
        plt.figure(figsize=(7, 4))

        order = df[column].value_counts().index
        sns.countplot(data=df, x=column, order=order)

        plt.title(f"Frequency of {column}")
        plt.xlabel(column.capitalize())
        plt.ylabel("Number of rows")
        plt.xticks(rotation=0)
        plt.tight_layout()
        plt.show()
```





Detailed description:

Measurement scales determine which comparisons and statistical operations are meaningful. Nominal variables are labels, ordinal variables have a meaningful order, interval variables have equal intervals without a true zero, and ratio variables have a meaningful zero.

In [32]:
```
#Create a measurement-scale table
measurement_scales = pd.DataFrame([
    ["mode", "Nominal", "Transport categories have no natural order"],
    ["choice", "Nominal", "Yes/no categories"],
    ["individual", "Identifier", "Traveller label; numeric differences are meani
    ["wait", "Ratio", "Elapsed waiting time, assuming the documented units"],
    ["travel", "Ratio", "Elapsed travel time, assuming the documented units"],
    ["vcost", "Ratio", "Cost with a meaningful zero"],
    ["gcost", "Ratio", "Generalized cost with a meaningful zero"],
```

```
    ["income", "Ratio", "Income has a meaningful zero"],
    ["size", "Ratio / discrete count", "Count of people or group size"],
    ["Example not present", "Ordinal", "Ratings such as low, medium and high"],
    ["Example not present", "Interval", "Temperature in degrees Celsius"]
], columns=["Variable", "Measurement scale", "Reason"])

display(measurement_scales)
```

	Variable	Measurement scale	Reason
0	mode	Nominal	Transport categories have no natural order
1	choice	Nominal	Yes/no categories
2	individual	Identifier	Traveller label; numeric differences are meani...
3	wait	Ratio	Elapsed waiting time, assuming the documented ...
4	travel	Ratio	Elapsed travel time, assuming the documented u...
5	vcost	Ratio	Cost with a meaningful zero
6	gcost	Ratio	Generalized cost with a meaningful zero
7	income	Ratio	Income has a meaningful zero
8	size	Ratio / discrete count	Count of people or group size
9	Example not present	Ordinal	Ratings such as low, medium and high
10	Example not present	Interval	Temperature in degrees Celsius

Detailed description:

Histograms show how frequently values fall within ranges. The optional KDE curve provides a smoothed view of the distribution. These plots help identify skewness, concentration and unusual values.

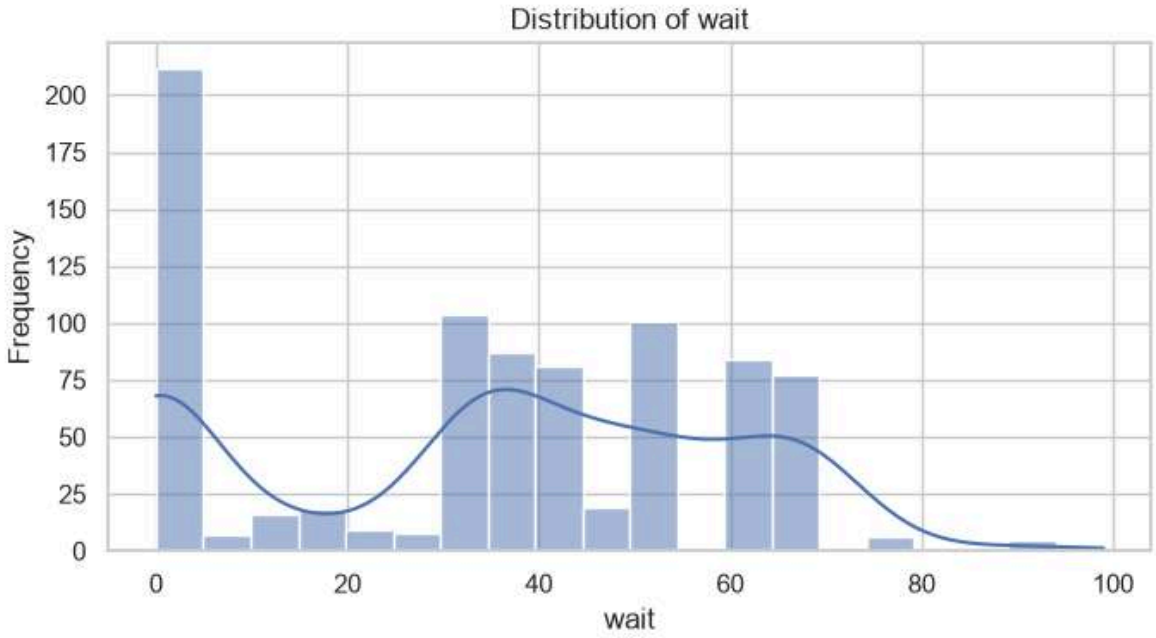
In [33]:
```
#Plot numerical distributions
numeric_to_plot = [
    column for column in ["wait", "vcost", "travel", "gcost", "income", "size"]
    if column in df.columns
]

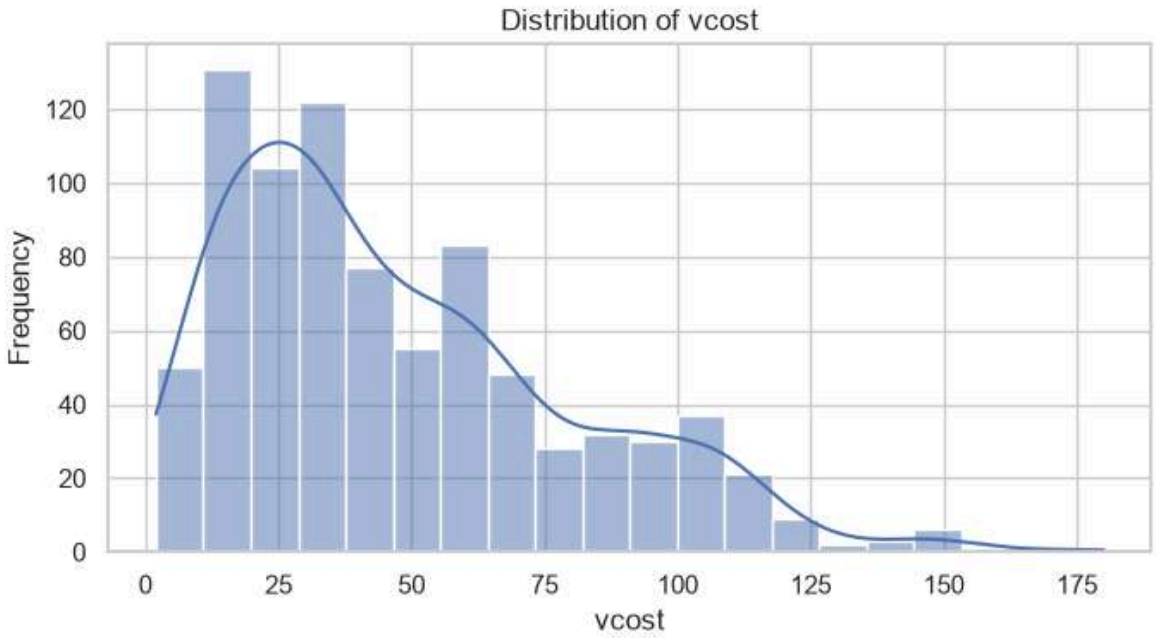
for column in numeric_to_plot:
    values = pd.to_numeric(df[column], errors="coerce").dropna()

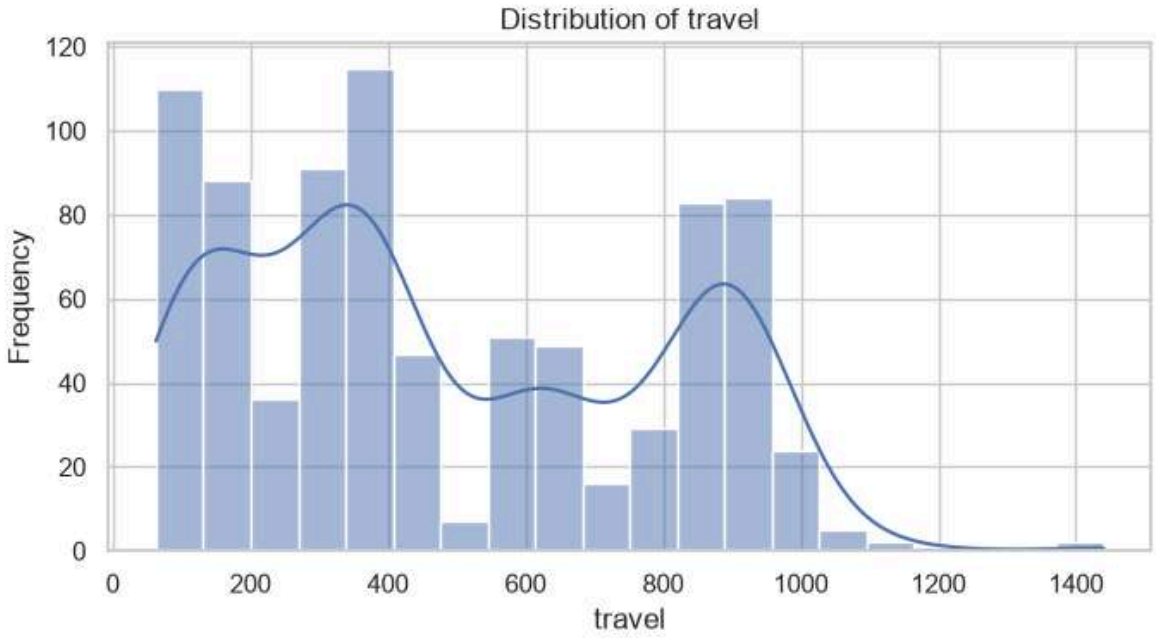
    if not values.empty:
        plt.figure(figsize=(7, 4))
        sns.histplot(values, bins=20, kde=True)

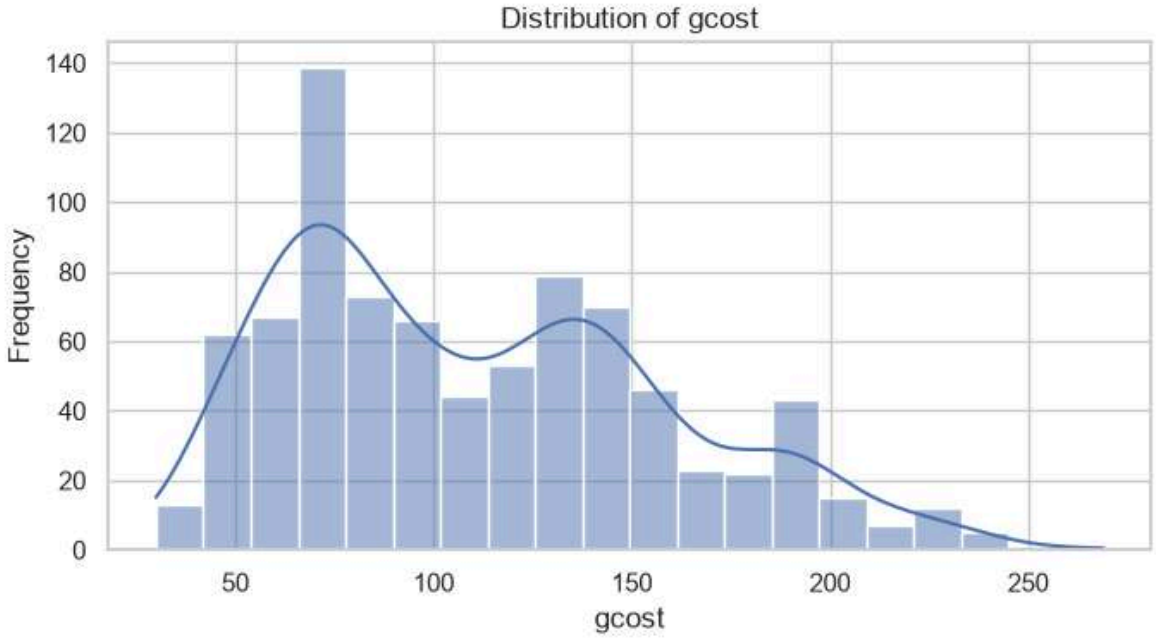
        plt.title(f"Distribution of {column}")
        plt.xlabel(column)
```

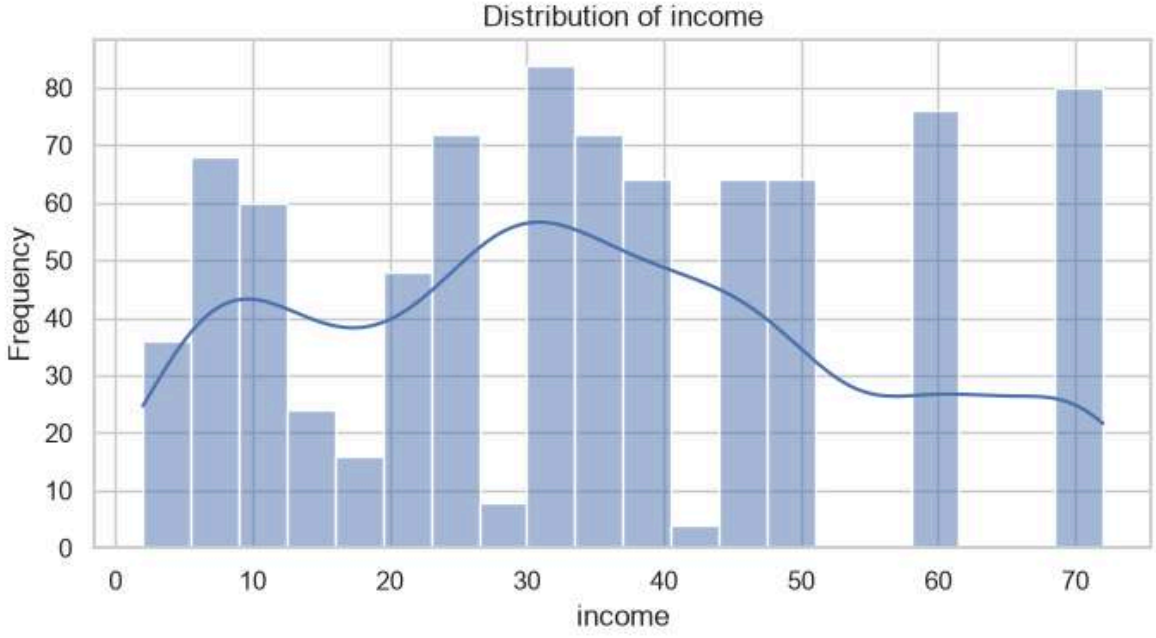
```
        plt.ylabel("Frequency")
        plt.tight_layout()
        plt.show()
```

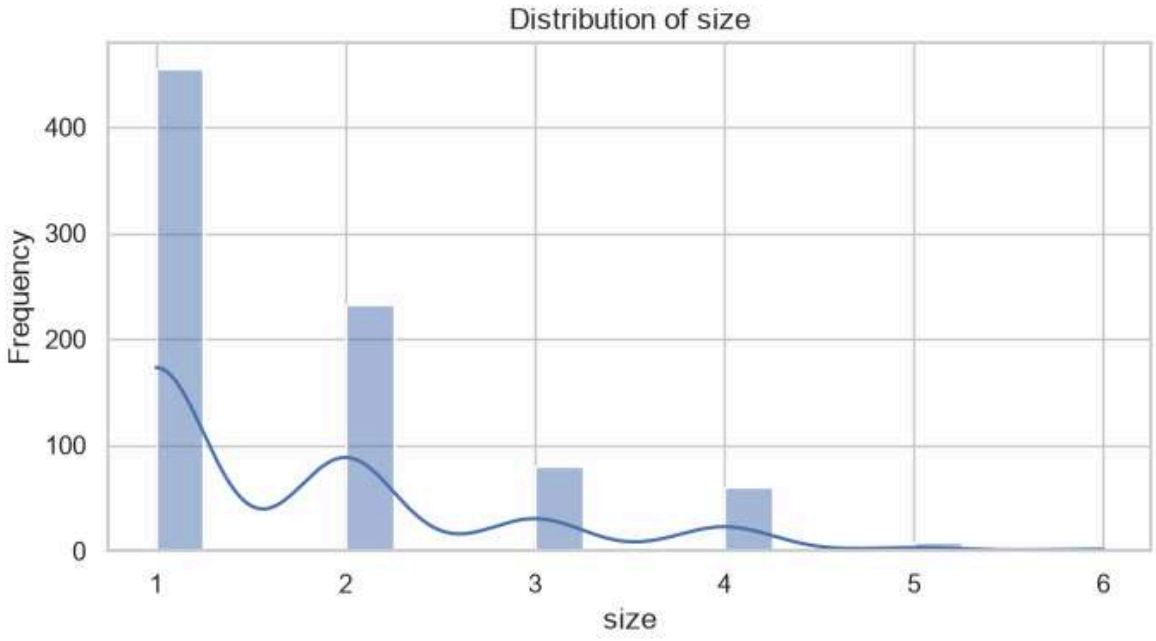





Distribution of travel
Frequency
travel
0
20
40
60
80
100
120
0
200
400
600
800
1000
1200
1400


Distribution of gcost
Frequency
gcost
0
20
40
60
80
100
120
140
50
100
150
200
250


Distribution of income
Frequency
income
0
10
20
30
40
50
60
70
80
0
10
20
30
40
50
60
70



Detailed description:

Classical or frequentist analysis commonly uses estimates, confidence intervals and hypothesis tests. Bayesian analysis combines prior information with observed data to calculate a posterior distribution. Both approaches can be used after EDA, depending on the research question.

In [34]:
```
#Compare classical and Bayesian analysis
analysis_comparison = pd.DataFrame([
    [
        "Basic idea",
        "Parameters are fixed; samples vary",
        "Parameters are represented by probability distributions"
    ],
    [
        "Prior information",
        "Not required as a Bayesian prior",
        "Uses a prior distribution"
    ],
    [
        "Typical outputs",
        "Confidence intervals and p-values",
        "Posterior distributions and credible intervals"
    ],
    [
        "Example",
        "Estimate average travel time",
        "Estimate average travel time with a prior belief"
    ],
    [
        "Common tools",
        "Python, R, SPSS, SAS",
        "Python with PyMC or Stan; R with brms or rstan"
    ]
], columns=["Aspect", "Classical analysis", "Bayesian analysis"])
```

```
display(analysis_comparison)
```

	Aspect	Classical analysis	Bayesian analysis
0	Basic idea	Parameters are fixed; samples vary	Parameters are represented by probability dist...
1	Prior information	Not required as a Bayesian prior	Uses a prior distribution
2	Typical outputs	Confidence intervals and p-values	Posterior distributions and credible intervals
3	Example	Estimate average travel time	Estimate average travel time with a prior belief
4	Common tools	Python, R, SPSS, SAS	Python with PyMC or Stan; R with brms or rstan

Detailed description:

This cell uses a t-based confidence interval for the mean, assuming independent observations and a suitable sampling interpretation. Because the dataset contains multiple alternatives per traveller, the independence assumption may not hold; interpret the interval as a basic classroom demonstration rather than a definitive population estimate.

In [35]:
```
#Demonstrate a simple classical confidence interval
from scipy import stats

if "travel" in df.columns:
    travel_values = pd.to_numeric(df["travel"], errors="coerce").dropna()

    n = len(travel_values)

    if n >= 2:
        mean_travel = travel_values.mean()
        standard_error = stats.sem(travel_values)
        lower, upper = stats.t.interval(
            confidence=0.95,
            df=n - 1,
            loc=mean_travel,
            scale=standard_error
        )

        print("Number of valid observations:", n)
        print(f"Mean travel time: {mean_travel:.3f}")
        print(f"95% confidence interval: ({lower:.3f}, {upper:.3f})")
    else:
        print("At least two valid travel-time observations are required.")
else:
    print("The 'travel' column was not found.")
```

```
Number of valid observations: 840
Mean travel time: 486.165
95% confidence interval: (465.751, 506.580)
```

In [36]:
```
#Display a compact dataset summary
module1_summary = pd.DataFrame({
    "Metric": [
        "Number of rows",
        "Number of columns",
        "Total cells",
        "Total missing cells",
        "Exact duplicate rows",
        "Numerical columns",
        "Categorical/text columns"
    ],
    "Value": [
        df.shape[0],
        df.shape[1],
        df.shape[0] * df.shape[1],
        int(df.isnull().sum().sum()),
        int(df.duplicated().sum()),
        len(df.select_dtypes(include=np.number).columns),
        len(df.select_dtypes(include=["object", "category", "bool", "string"]).c
    ]
})

display(module1_summary)
```

	Metric	Value
0	Number of rows	840
1	Number of columns	10
2	Total cells	8400
3	Total missing cells	0
4	Exact duplicate rows	0
5	Numerical columns	8
6	Categorical/text columns	2

Conclusion

Exploratory Data Analysis was performed on the TravelMode dataset using Python in Jupyter Notebook. The dataset contained [840] observations and [10] variables. Initial inspection identified the available numerical and categorical variables, and the dataset was examined for missing values and duplicate records. Frequency tables and bar charts were used to study travel modes and choice categories, while statistical summaries and histograms were used to understand numerical distributions. Measurement scales were identified according to the meaning of each variable. Classical and Bayesian analysis approaches and common EDA software tools were also compared. These results establish the foundation for data transformation, correlation analysis and visualization in the subsequent modules.

Detailed description:

Since we are working in the same notebook, df should already contain your TravelMode dataset. This cell checks that it exists and displays its dimensions before transformation.

In [37]:
```
# Data Transformation

if "df" not in globals():
    raise NameError("Run the Module 1 dataset-loading cell first.")

print("Dataset shape:", df.shape)
print("Column names:", df.columns.tolist())
```

```
Dataset shape: (840, 10)
Column names: ['rownames', 'individual', 'mode', 'choice', 'wait', 'vcost', 'trav
el', 'gcost', 'income', 'size']
```

Detailed description:

We create a working copy named df_m2 so that the original df remains available for later comparisons. Only completely identical rows are removed; repeated traveller IDs are not automatically considered duplicates.

In [39]:
```
# Check and remove duplicate records
df_m2 = df.copy()
duplicates_before = df_m2.duplicated().sum()
df_m2 = df_m2.drop_duplicates().copy()
duplicates_after = df_m2.duplicated().sum()
print("Duplicate rows before cleaning:", duplicates_before)
print("Duplicate rows removed:", duplicates_before - duplicates_after)
print("Duplicate rows remaining:", duplicates_after)
print("Shape after cleaning:", df_m2.shape)
```

```
Duplicate rows before cleaning: 0
Duplicate rows removed: 0
Duplicate rows remaining: 0
Shape after cleaning: (840, 10)
```

Detailed description:

Values such as " Air " and "air" should use a consistent format. This cell removes surrounding spaces and converts text to lowercase without changing numerical measurements.

In [40]:
```
# Replace inconsistent categorical values
for column in ["mode", "choice"]:
    if column in df_m2.columns:
        df_m2[column] = df_m2[column].apply(
            lambda value: value.strip().lower()
            if isinstance(value, str)
            else value
```

```
        )

print("Categorical values after standardization:")

for column in ["mode", "choice"]:
    if column in df_m2.columns:
        print(f"\n{column}:")
        print(df_m2[column].value_counts(dropna=False))
```

```
Categorical values after standardization:

mode:
mode
air      210
train    210
bus      210
car      210
Name: count, dtype: int64

choice:
choice
no     630
yes    210
Name: count, dtype: int64
```

Detailed description:

This step determines whether imputation is needed. We inspect every column before choosing a replacement method.

In [41]:
```
#Check missing values
missing_summary = pd.DataFrame({
    "Missing Count": df_m2.isnull().sum(),
    "Missing Percentage": (df_m2.isnull().mean() * 100).round(2)
})

display(missing_summary)

print("Total missing cells:", int(df_m2.isnull().sum().sum()))
```

	Missing Count	Missing Percentage
rownames	0	0.0
individual	0	0.0
mode	0	0.0
choice	0	0.0
wait	0	0.0
vcost	0	0.0
travel	0	0.0
gcost	0	0.0
income	0	0.0
size	0	0.0

```
Total missing cells: 0
```

Detailed description:

The median is generally less affected by outliers than the mean. The mode replaces a missing categorical value with the most frequent observed category. The original df remains unchanged.

In [42]:
```
# Handle missing values
df_imputed = df_m2.copy()
missing_before = int(df_imputed.isnull().sum().sum())

# Fill missing numerical values using the median
numeric_cols = df_imputed.select_dtypes(include="number").columns

for column in numeric_cols:
    if df_imputed[column].isnull().any():
        df_imputed[column] = df_imputed[column].fillna(
            df_imputed[column].median()
        )

# Fill missing categorical/text values using the mode
categorical_cols = df_imputed.select_dtypes(
    include=["object", "category", "bool", "string"]
).columns

for column in categorical_cols:
    if df_imputed[column].isnull().any():
        modes = df_imputed[column].mode(dropna=True)

        if not modes.empty:
            df_imputed[column] = df_imputed[column].fillna(modes.iloc[0])

missing_after = int(df_imputed.isnull().sum().sum())

print("Missing cells before imputation:", missing_before)
```

```
print("Missing cells after imputation:", missing_after)
print("Missing cells filled:", missing_before - missing_after)
```

```
Missing cells before imputation: 0
Missing cells after imputation: 0
Missing cells filled: 0
```

Detailed description:

Quantile binning groups numerical travel times into three approximate frequency groups. The new column travel_category preserves the original travel column.

In [43]:
```
# Discretization and binning
if "travel" in df_imputed.columns:
    travel_values = pd.to_numeric(
        df_imputed["travel"], errors="coerce"
    )

    valid = travel_values.notna()

    df_imputed["travel_category"] = pd.NA

    try:
        df_imputed.loc[valid, "travel_category"] = pd.qcut(
            travel_values[valid],
            q=3,
            labels=["Low", "Medium", "High"],
            duplicates="drop"
        ).astype(str)

        print("Travel-time category frequencies:")
        display(
            df_imputed["travel_category"]
            .value_counts(dropna=False)
            .to_frame("Frequency")
        )

    except ValueError as error:
        print("Binning could not be completed:", error)
else:
    print("The travel column was not found.")
```

```
Travel-time category frequencies:
```

	Frequency
travel_category	
Low	283
Medium	280
High	277

Detailed description:

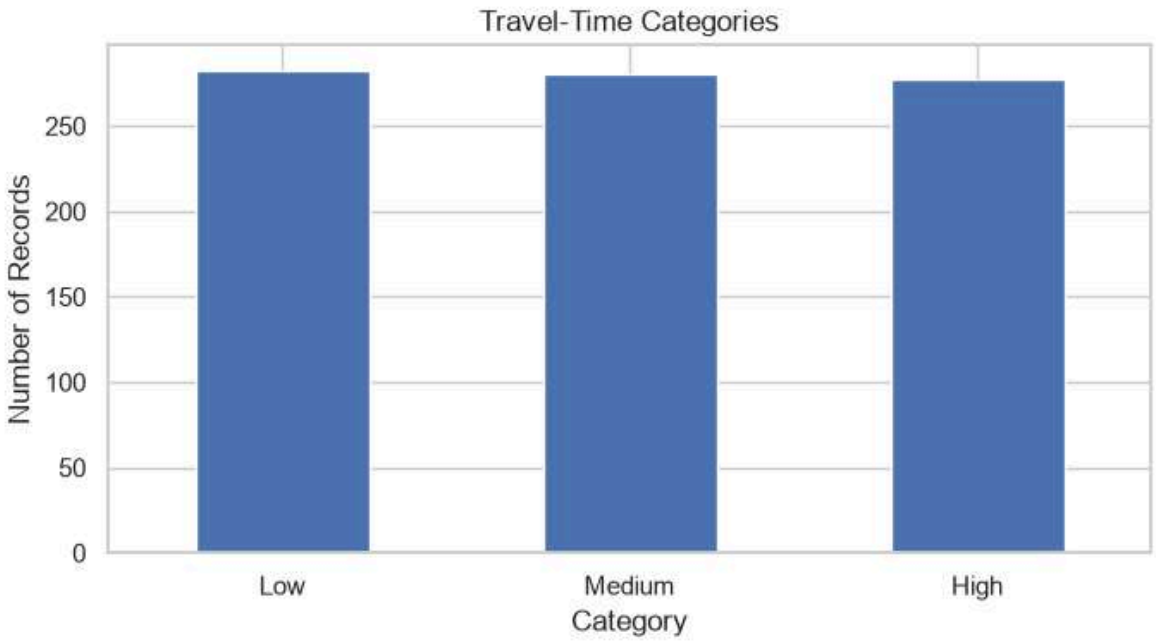
A bar chart helps compare the number of records classified as Low, Medium and High. We also print the frequencies so that your report can refer to actual values.

In [44]:
```
# Visualize the binned travel time
if "travel_category" in df_imputed.columns:
    bin_counts = (
        df_imputed["travel_category"]
        .value_counts()
        .reindex(["Low", "Medium", "High"], fill_value=0)
    )

    plt.figure(figsize=(7, 4))
    bin_counts.plot(kind="bar")

    plt.title("Travel-Time Categories")
    plt.xlabel("Category")
    plt.ylabel("Number of Records")
    plt.xticks(rotation=0)
    plt.tight_layout()
    plt.show()

    display(bin_counts.to_frame("Frequency"))
```



travel_category	Frequency
Low	283
Medium	280
High	277

Detailed description:

Under a normal-distribution model, the MLE mean is the arithmetic mean, and the MLE variance divides the sum of squared deviations by n.

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

$$\hat{\sigma}^2 = \frac{1}{n}\sum_{i=1}^{n} (x_i - \hat{\mu})^2$$

In [45]:
```
#Maximum Likelihood Estimation (MLE)
if "travel" in df_imputed.columns:
    travel_values = pd.to_numeric(
        df_imputed["travel"], errors="coerce"
    ).dropna().to_numpy()

    if len(travel_values) > 0:
        mle_mean = np.mean(travel_values)
        mle_variance = np.mean((travel_values - mle_mean) ** 2)
        mle_std = np.sqrt(mle_variance)

        print("MLE mean:", round(mle_mean, 3))
        print("MLE variance:", round(mle_variance, 3))
        print("MLE standard deviation:", round(mle_std, 3))
    else:
        print("No valid travel-time values are available.")
```

```
MLE mean: 486.165
MLE variance: 90757.362
MLE standard deviation: 301.26
```

Detailed description:

The histogram displays the travel-time frequencies, and the dashed vertical line marks the MLE mean.

In [46]:
```
# Plot the MLE mean on the travel-time distribution
if "travel" in df_imputed.columns:
    travel_values = pd.to_numeric(
        df_imputed["travel"], errors="coerce"
    ).dropna()

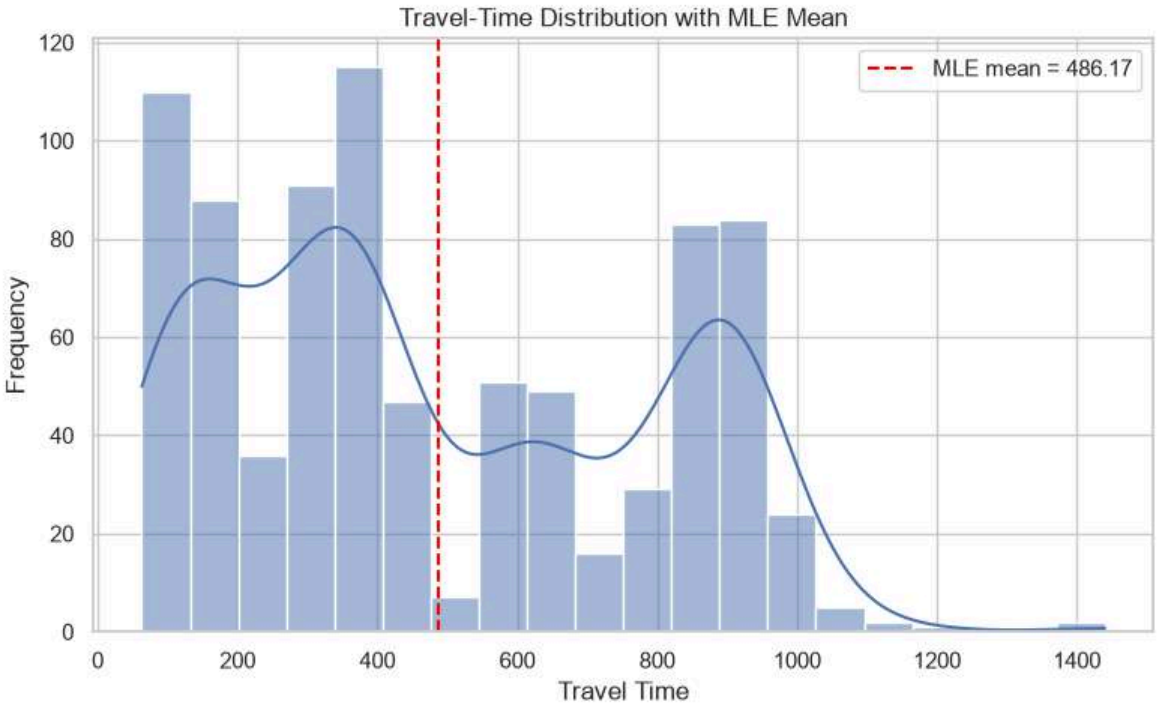
    if not travel_values.empty:
        mle_mean = travel_values.mean()

        plt.figure(figsize=(8, 5))
        sns.histplot(travel_values, bins=20, kde=True)

        plt.axvline(
            mle_mean,
            color="red",
            linestyle="--",
            label=f"MLE mean = {mle_mean:.2f}"
```

```
        )

        plt.title("Travel-Time Distribution with MLE Mean")
        plt.xlabel("Travel Time")
        plt.ylabel("Frequency")
        plt.legend()
        plt.tight_layout()
        plt.show()
```



In [47]:
```
module2_summary = pd.DataFrame({
    "Metric": [
        "Rows after deduplication",
        "Columns after transformation",
        "Remaining missing cells",
        "Remaining exact duplicate rows",
        "Travel-time categories created"
    ],
    "Value": [
        df_imputed.shape[0],
        df_imputed.shape[1],
        int(df_imputed.isnull().sum().sum()),
        int(df_imputed.duplicated().sum()),
        (
            df_imputed["travel_category"].nunique()
            if "travel_category" in df_imputed.columns
            else 0
        )
    ]
})

display(module2_summary)
```

	Metric	Value
0	Rows after deduplication	840
1	Columns after transformation	11
2	Remaining missing cells	0
3	Remaining exact duplicate rows	0
4	Travel-time categories created	3

Conclusion

Data transformation was performed on the TravelMode dataset by identifying and removing exact duplicate records, standardizing categorical values, checking missing data, and applying imputation where required. Travel time was discretized into categories and visualized using a bar chart. Maximum Likelihood Estimation was demonstrated by calculating the mean and variance of travel time and plotting the estimated mean on its distribution. The final summary was used to verify the transformed dataset's quality.

Detailed description:

We reuse df_imputed from Module 2. The numerical columns will be used for descriptive statistics and correlation analysis. We exclude identifier columns from the main numerical analysis because their values do not represent meaningful measurements.

In [48]:
```
# Correlation Analysis and Time Series Analysis

if "df_imputed" not in globals():
    raise NameError("Run Module 1 and Module 2 first.")

df_m3 = df_imputed.copy()

numeric_cols = [
    col for col in ["wait", "vcost", "travel", "gcost", "income", "size"]
    if col in df_m3.columns
]

print("Dataset shape:", df_m3.shape)
print("Numerical variables used:", numeric_cols)
print("Categorical variables available:",
      [col for col in ["mode", "choice"] if col in df_m3.columns])
```

```
Dataset shape: (840, 11)
Numerical variables used: ['wait', 'vcost', 'travel', 'gcost', 'income', 'size']
Categorical variables available: ['mode', 'choice']
```

Detailed description:

Univariate analysis describes a single variable using statistics such as mean, median, standard deviation, minimum and maximum. It helps us understand the distribution and variability of travel measurements.

In [49]:
```
# Univariate analysis
print("Univariate statistical analysis")

display(df_m3[numeric_cols].describe().T.round(2))
```

```
Univariate statistical analysis
```

	count	mean	std	min	25%	50%	75%	max
wait	840.0	34.59	24.95	0.0	0.75	35.0	53.00	99.0
vcost	840.0	47.76	32.37	2.0	23.00	39.0	66.25	180.0
travel	840.0	486.17	301.44	63.0	234.00	397.0	795.50	1440.0
gcost	840.0	110.88	47.98	30.0	71.00	101.5	144.00	269.0
income	840.0	34.55	19.68	2.0	20.00	34.5	50.00	72.0
size	840.0	1.74	1.01	1.0	1.00	1.0	2.00	6.0

Detailed description:

A histogram shows how observations are distributed across value ranges. A boxplot displays the median, quartiles and potential outliers. These visualizations complement the statistics.

In [50]:
```
#Visualize univariate distributions
import matplotlib.pyplot as plt
import seaborn as sns

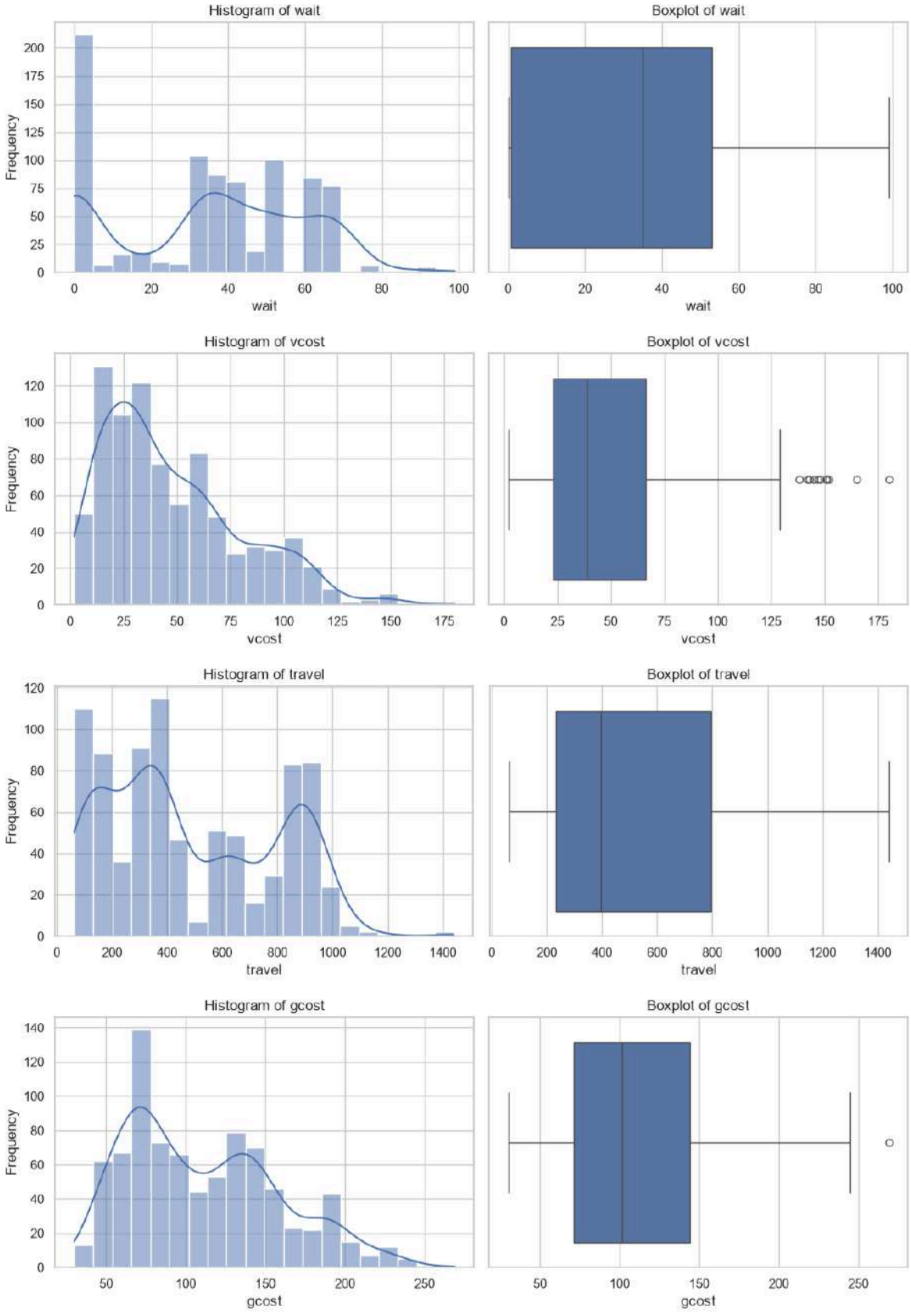
for col in numeric_cols:
    values = pd.to_numeric(df_m3[col], errors="coerce").dropna()

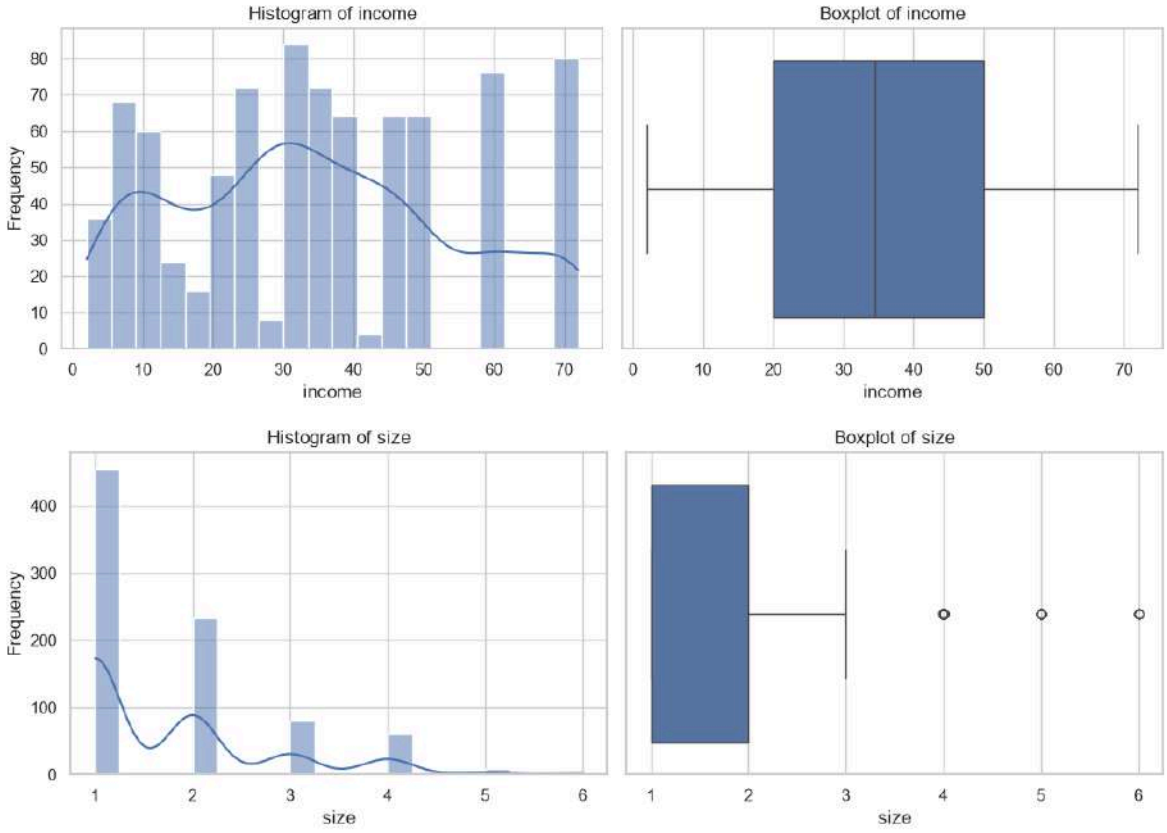
    if not values.empty:
        fig, axes = plt.subplots(1, 2, figsize=(11, 4))

        sns.histplot(values, bins=20, kde=True, ax=axes[0])
        axes[0].set_title(f"Histogram of {col}")
        axes[0].set_xlabel(col)
        axes[0].set_ylabel("Frequency")

        sns.boxplot(x=values, ax=axes[1])
        axes[1].set_title(f"Boxplot of {col}")
        axes[1].set_xlabel(col)

        plt.tight_layout()
        plt.show()
```


Histogram of wait
Frequency
wait
Boxplot of wait
wait
Histogram of vcost
Frequency
vcost
Boxplot of vcost
vcost
Histogram of travel
Frequency
travel
Boxplot of travel
travel
Histogram of gcost
Frequency
gcost
Boxplot of gcost
gcost



Detailed description:

Bivariate analysis helps determine whether two variables change together. For example, we can examine whether higher travel times are associated with higher generalized costs, and compare average travel time across transport modes.

In [51]:
```
#Compare travel metrics by mode
if "mode" in df_m3.columns:
    metrics_by_mode = (
        df_m3.groupby("mode", observed=True)[
            [col for col in ["wait", "vcost", "travel", "gcost"]
             if col in df_m3.columns]
        ]
        .agg(["count", "mean", "median"])
        .round(2)
    )

    display(metrics_by_mode)
else:
    print("The mode column is not available.")
```

	wait			vcost			travel			
	count	mean	median	count	mean	median	count	mean	median	count	me
mode											
air	210	61.01	64.0	210	85.25	81.0	210	133.71	124.5	210	102
bus	210	41.66	35.0	210	33.46	32.0	210	629.46	601.5	210	115
car	210	0.00	0.0	210	21.00	16.0	210	573.20	577.0	210	95
train	210	35.69	34.0	210	51.34	42.0	210	608.29	607.5	210	130

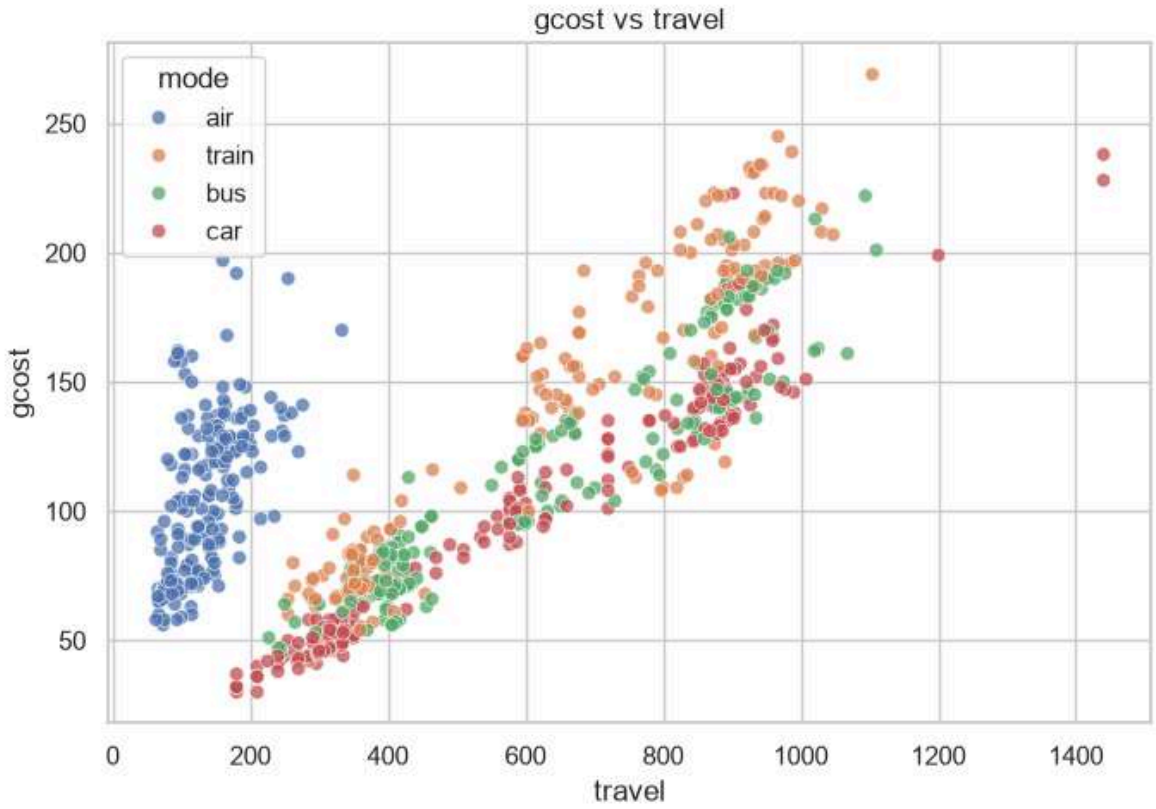
Detailed description: Scatter plots represent each record as a point. A pattern rising from left to right suggests a positive association, while a pattern falling from left to right suggests a negative association.

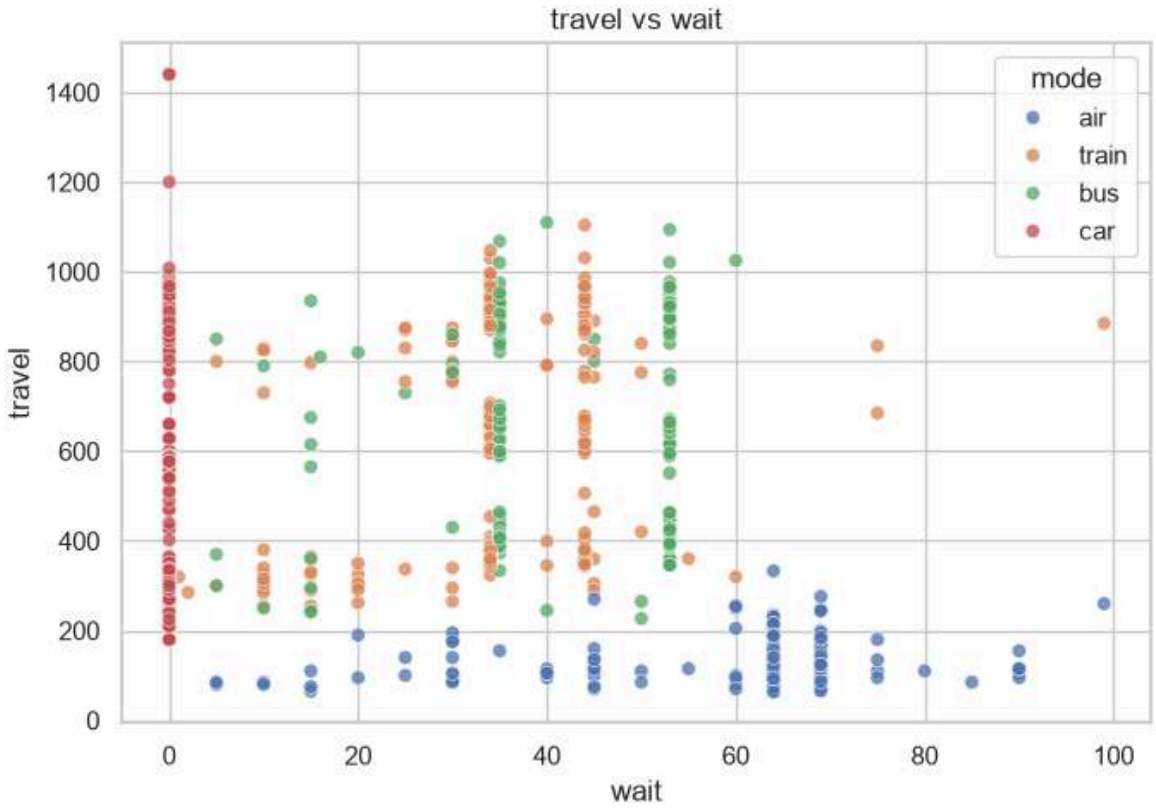
In [52]:
```
# Scatter plots for bivariate analysis
pairs = [
    ("travel", "gcost"),
    ("wait", "travel"),
    ("vcost", "gcost")
]

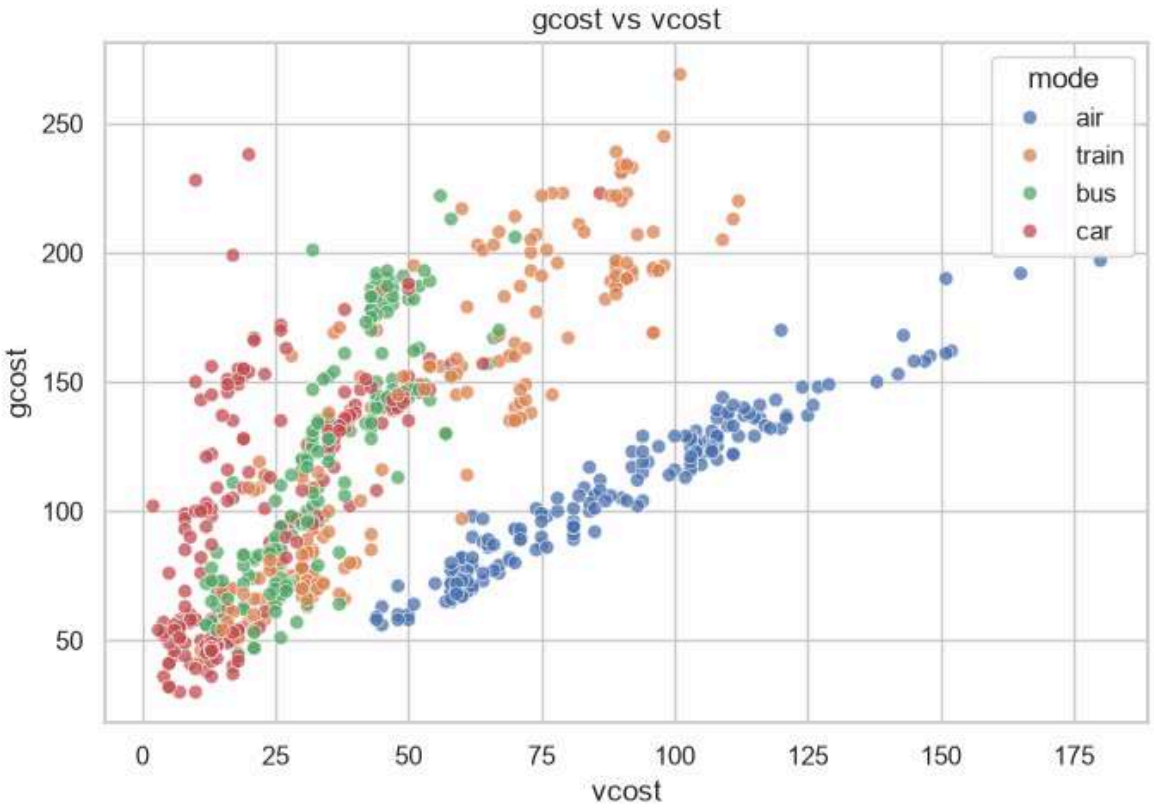
for x_col, y_col in pairs:
    if x_col in df_m3.columns and y_col in df_m3.columns:
        plt.figure(figsize=(7, 5))

        if "mode" in df_m3.columns:
            sns.scatterplot(
                data=df_m3,
                x=x_col,
                y=y_col,
                hue="mode",
                alpha=0.75
            )
        else:
            sns.scatterplot(
                data=df_m3,
                x=x_col,
                y=y_col,
                alpha=0.75
            )

        plt.title(f"{y_col} vs {x_col}")
        plt.xlabel(x_col)
        plt.ylabel(y_col)
        plt.tight_layout()
        plt.show()
```


gcost vs travel
mode
air
train
bus
car
gcost
travel
0
200
400
600
800
1000
1200
1400
50
100
150
200
250


travel vs wait
mode
air
train
bus
car
travel
wait
0
20
40
60
80
100
200
400
600
800
1000
1200
1400



Detailed description:

Multivariate analysis considers multiple variables together. A correlation matrix and heatmap help identify variables that move together, which can guide later dimensionality reduction and model development.

In [53]:
```
# Multivariate analysis
correlation_matrix = df_m3[numeric_cols].corr(numeric_only=True)

print("Correlation matrix:")
display(correlation_matrix.round(3))
```

```
Correlation matrix:
```

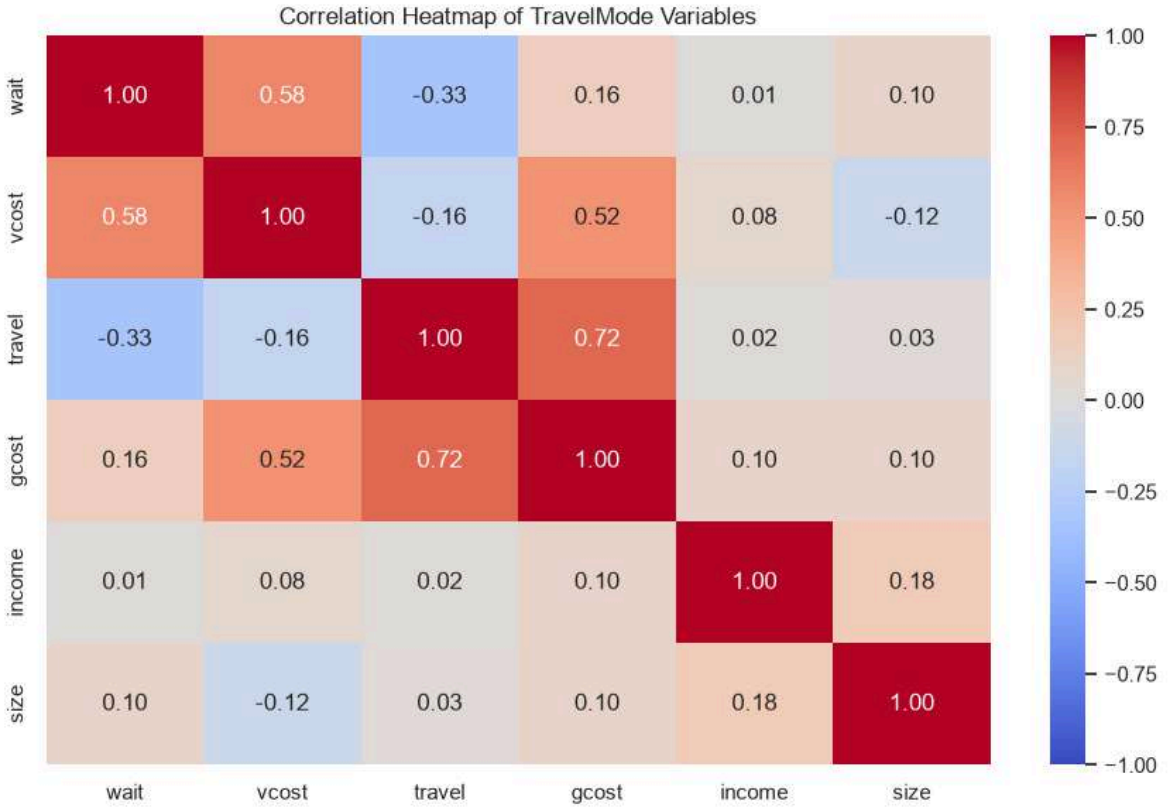
	wait	vcost	travel	gcost	income	size
wait	1.000	0.580	-0.332	0.158	0.013	0.101
vcost	0.580	1.000	-0.160	0.525	0.075	-0.125
travel	-0.332	-0.160	1.000	0.717	0.018	0.025
gcost	0.158	0.525	0.717	1.000	0.099	0.101
income	0.013	0.075	0.018	0.099	1.000	0.181
size	0.101	-0.125	0.025	0.101	0.181	1.000

Detailed description:

The heatmap makes strong positive and negative relationships easier to identify. The numbers printed inside the cells are the actual correlation coefficients calculated from your dataset.

In [54]:
```
#Correlation heatmap
plt.figure(figsize=(9, 6))
sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0,
    vmin=-1,
    vmax=1
)

plt.title("Correlation Heatmap of TravelMode Variables")
plt.tight_layout()
plt.show()
```



Detailed description:

Time-series analysis requires observations associated with dates, timestamps or meaningful time periods. We inspect the column names for possible time-related fields rather than creating artificial dates.

In [55]:
```
# Fundamentals of Time Series Analysis Check for a date/time column
time_candidates = [
    col for col in df_m3.columns
    if any(term in col.lower()
           for term in ["date", "time", "timestamp", "datetime", "year", "month"
```

```
]

print("Possible time-related columns:", time_candidates)

if time_candidates:
    print("Inspect these columns before performing time-series analysis.")
else:
    print("No obvious date/time column was found.")
    print("Time-based analysis cannot be performed reliably on this dataset.")
```

```
Possible time-related columns: []
No obvious date/time column was found.
Time-based analysis cannot be performed reliably on this dataset.
```

Detailed description:

A time series is a sequence of observations indexed by time. Its common characteristics include trend, seasonality, cyclical patterns and random variation. These characteristics cannot be confirmed in TravelMode without a genuine time index.

In [56]:
```
#Understand the characteristics of time series
time_series_characteristics = pd.DataFrame({
    "Characteristic": [
        "Trend",
        "Seasonality",
        "Cyclical pattern",
        "Random variation",
        "Time-based index"
    ],
    "Meaning": [
        "Long-term increase or decrease",
        "Pattern repeating at regular intervals",
        "Longer-term rises and falls without a fixed seasonal period",
        "Irregular fluctuations",
        "Dates or timestamps used to order observations"
    ],
    "Can TravelMode establish it?": [
        "No, not without time-indexed observations",
        "No, not without time-indexed observations",
        "No, not without time-indexed observations",
        "Not as a time-series component",
        "No genuine time index is apparent"
    ]
})

display(time_series_characteristics)
```

	Characteristic	Meaning	Can TravelMode establish it?
0	Trend	Long-term increase or decrease	No, not without time-indexed observations
1	Seasonality	Pattern repeating at regular intervals	No, not without time-indexed observations
2	Cyclical pattern	Longer-term rises and falls without a fixed se...	No, not without time-indexed observations
3	Random variation	Irregular fluctuations	Not as a time-series component
4	Time-based index	Dates or timestamps used to order observations	No genuine time index is apparent

Detailed description:

The following example uses a small, explicitly illustrative dataset to demonstrate pandas time indexing. It is not part of your TravelMode data and must not be used to report TravelMode findings.

In [59]:
```
# Illustrative example only — not real TravelMode observations
demo_ts = pd.DataFrame({
    "date": pd.date_range(start="2026-01-01", periods=7, freq="D"),
    "example_value": [10, 12, 11, 15, 14, 17, 16]
})

demo_ts["date"] = pd.to_datetime(demo_ts["date"])
demo_ts = demo_ts.set_index("date")

print("Illustrative time-indexed dataset:")
display(demo_ts)

print("Index type:", type(demo_ts.index).__name__)
```

```
Illustrative time-indexed dataset:
```

date	example_value
2026-01-01	10
2026-01-02	12
2026-01-03	11
2026-01-04	15
2026-01-05	14
2026-01-06	17
2026-01-07	16

```
Index type: DatetimeIndex
```

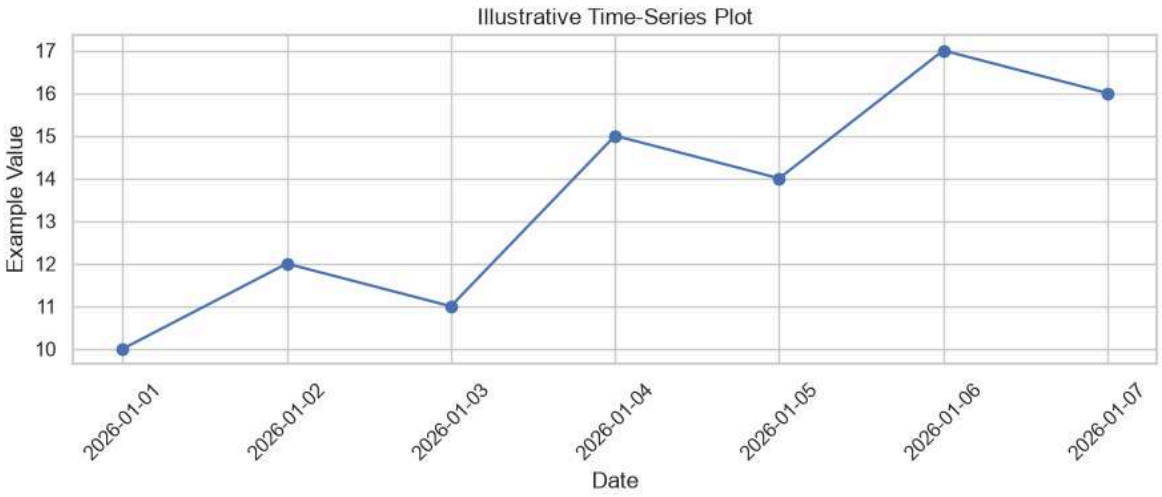
Detailed description:

A time-series line chart places the time index on the x-axis and the observed value on the y-axis. It is useful for examining changes over time when genuine chronological data are available.

In [63]:
```
# Visualize a time series ,Illustrative data only — not TravelMode findings
plt.figure(figsize=(9, 4))

plt.plot(
    demo_ts.index,
    demo_ts["example_value"],
    marker="o"
)

plt.title("Illustrative Time-Series Plot")
plt.xlabel("Date")
plt.ylabel("Example Value")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Detailed description:

Resampling changes the frequency of time-indexed data. Here, daily values are summarized into weekly averages. This operation requires a valid datetime index.

In [64]:
```
# Resample time-series data
weekly_demo = demo_ts.resample("W")["example_value"].mean()

print("Illustrative weekly averages:")
display(weekly_demo.to_frame("Weekly Average"))

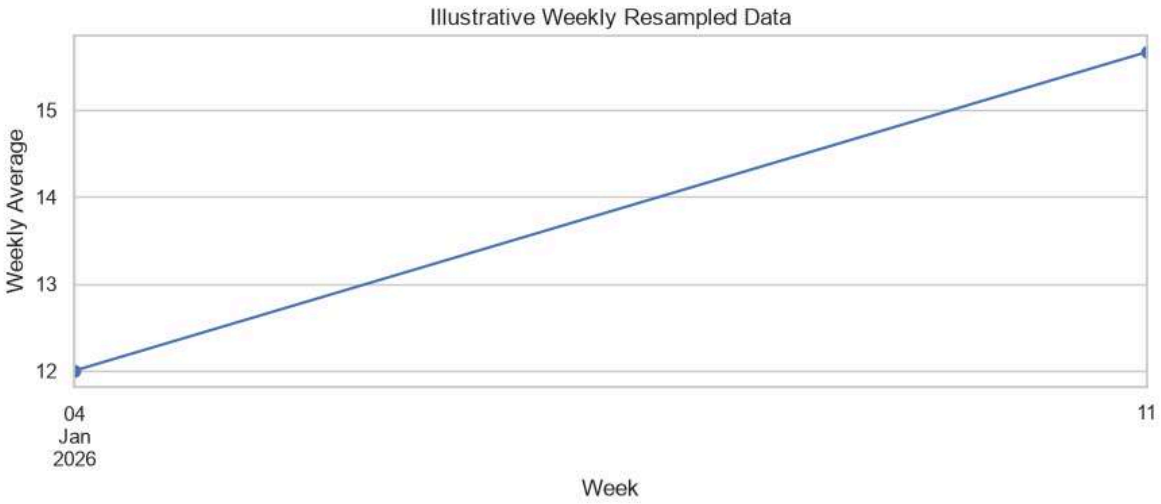
plt.figure(figsize=(9, 4))
weekly_demo.plot(marker="o")

plt.title("Illustrative Weekly Resampled Data")
plt.xlabel("Week")
```

```
plt.ylabel("Weekly Average")
plt.tight_layout()
plt.show()
```

```
Illustrative weekly averages:
```

date	Weekly Average
2026-01-04	12.000000
2026-01-11	15.666667



In [65]:
```
corr_pairs = (
    correlation_matrix.where(
        np.triu(np.ones(correlation_matrix.shape), k=1).astype(bool)
    )
    .stack()
    .sort_values()
)

print("Module 3 summary")
print("----------------")
print("Dataset shape:", df_m3.shape)
print("Numerical variables analyzed:", len(numeric_cols))
print("Potential time-related columns:", time_candidates)

if not corr_pairs.empty:
    print("\nStrongest negative correlation:")
    print(corr_pairs.head(1))

    print("\nStrongest positive correlation:")
    print(corr_pairs.tail(1))
else:
    print("Not enough numerical variables to calculate pairwise correlations.")
```

```
Module 3 summary
----------------
Dataset shape: (840, 11)
Numerical variables analyzed: 6
Potential time-related columns: []

Strongest negative correlation:
wait  travel   -0.332132
dtype: float64

Strongest positive correlation:
size  size   NaN
dtype: float64
```

Conclusion

Correlation Analysis was performed on the TravelMode dataset using univariate, bivariate and multivariate techniques. Descriptive statistics and histograms were used to examine individual variables, while grouped summaries and scatter plots were used to compare travel modes, travel time and costs. A correlation matrix and heatmap were generated to identify positive and negative associations between numerical variables. The time-series section examined the requirements for time-based indexing, visualization, grouping and resampling using clearly labelled illustrative data. Since the TravelMode dataset does not contain a genuine date/time index, temporal trends and seasonality could not be established from the original dataset.

Detailed description:

We reuse the DataFrame prepared in Module 2 and identify the numerical columns needed for statistical summaries. This avoids downloading the dataset again.

In [66]:
```
# Data Summarization and Visualization

if "df_imputed" not in globals():
    raise NameError("Run Module 1 and Module 2 before Module 4.")

df_m4 = df_imputed.copy()

numeric_cols = [
    col for col in ["wait", "vcost", "travel", "gcost", "income", "size"]
    if col in df_m4.columns
]

print("Dataset shape:", df_m4.shape)
print("Numerical variables:", numeric_cols)
print("Available categorical variables:",
      [col for col in ["mode", "choice"] if col in df_m4.columns])
```

```
Dataset shape: (840, 11)
Numerical variables: ['wait', 'vcost', 'travel', 'gcost', 'income', 'size']
Available categorical variables: ['mode', 'choice']
```

Detailed description:

These statistics summarize the centre and spread of numerical data. The mean shows the average, the median is the middle value, standard deviation measures variability, and quartiles describe the distribution.

In [67]:
```
# Calculate statistical summary measures
print("Statistical summary of numerical variables:")

summary = df_m4[numeric_cols].describe().T

summary["median"] = df_m4[numeric_cols].median()
summary["variance"] = df_m4[numeric_cols].var()
summary["IQR"] = (
    df_m4[numeric_cols].quantile(0.75)
    - df_m4[numeric_cols].quantile(0.25)
)

display(summary.round(3))
```

```
Statistical summary of numerical variables:
```

	count	mean	std	min	25%	50%	75%	max	median	variance
wait	840.0	34.589	24.949	0.0	0.75	35.0	53.00	99.0	35.0	622.433
vcost	840.0	47.761	32.371	2.0	23.00	39.0	66.25	180.0	39.0	1047.882
travel	840.0	486.165	301.439	63.0	234.00	397.0	795.50	1440.0	397.0	90865.535
gcost	840.0	110.880	47.978	30.0	71.00	101.5	144.00	269.0	101.5	2301.922
income	840.0	34.548	19.676	2.0	20.00	34.5	50.00	72.0	34.5	387.147
size	840.0	1.743	1.010	1.0	1.00	1.0	2.00	6.0	1.0	1.021

Detailed description:

Skewness measures the asymmetry of a distribution. The IQR method flags observations below (Q1-1.5(IQR)) or above (Q3+1.5(IQR)) as potential outliers.

In [68]:
```
# Examine skewness and potential outliers
outlier_results = []

for col in numeric_cols:
    values = pd.to_numeric(df_m4[col], errors="coerce").dropna()

    if len(values) > 0:
        q1 = values.quantile(0.25)
        q3 = values.quantile(0.75)
        iqr = q3 - q1

        lower_bound = q1 - 1.5 * iqr
```

```
        upper_bound = q3 + 1.5 * iqr

        outlier_count = (
            (values < lower_bound) | (values > upper_bound)
        ).sum()

        outlier_results.append({
            "Variable": col,
            "Skewness": values.skew(),
            "Lower Bound": lower_bound,
            "Upper Bound": upper_bound,
            "Potential Outliers": int(outlier_count)
        })

outlier_summary = pd.DataFrame(outlier_results)
display(outlier_summary.round(3))
```

	Variable	Skewness	Lower Bound	Upper Bound	Potential Outliers
0	wait	-0.111	-77.625	131.375	0
1	vcost	0.921	-41.875	131.125	11
2	travel	0.356	-608.250	1637.750	0
3	gcost	0.568	-38.500	253.500	1
4	income	0.227	-25.000	95.000	0
5	size	1.449	-0.500	3.500	72

Detailed description:

A histogram groups values into intervals and shows how many observations fall into each interval. It helps identify concentration, skewness, multiple peaks and unusual ranges.

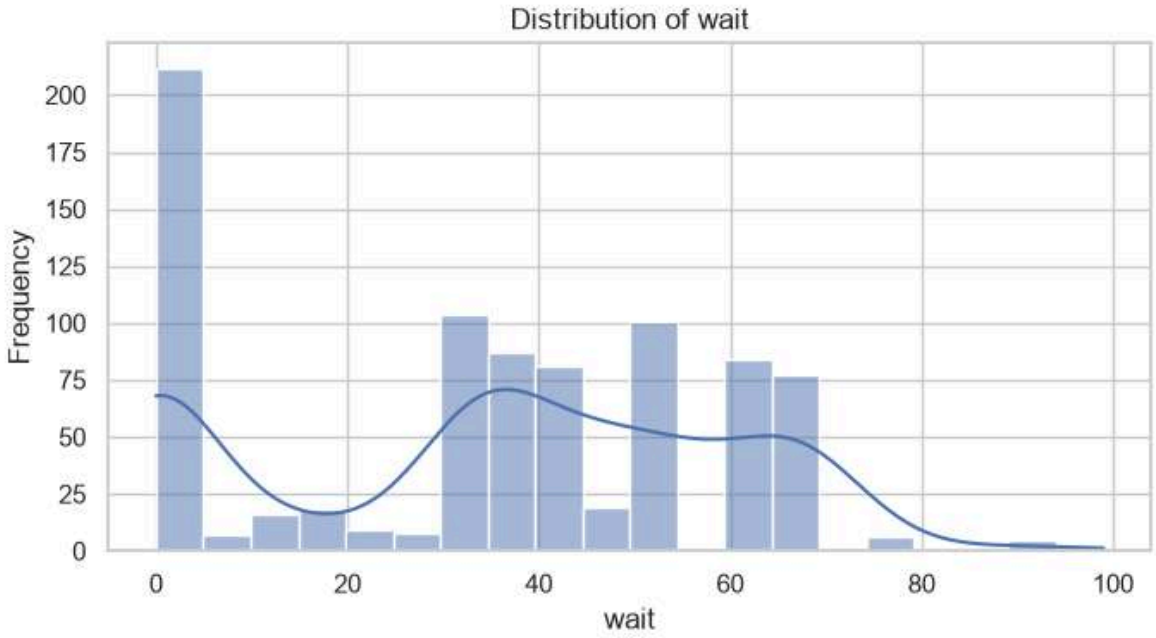
In [69]:
```
# One-Dimensional (1-D) Statistical Analysis (Plot histograms)
import matplotlib.pyplot as plt
import seaborn as sns

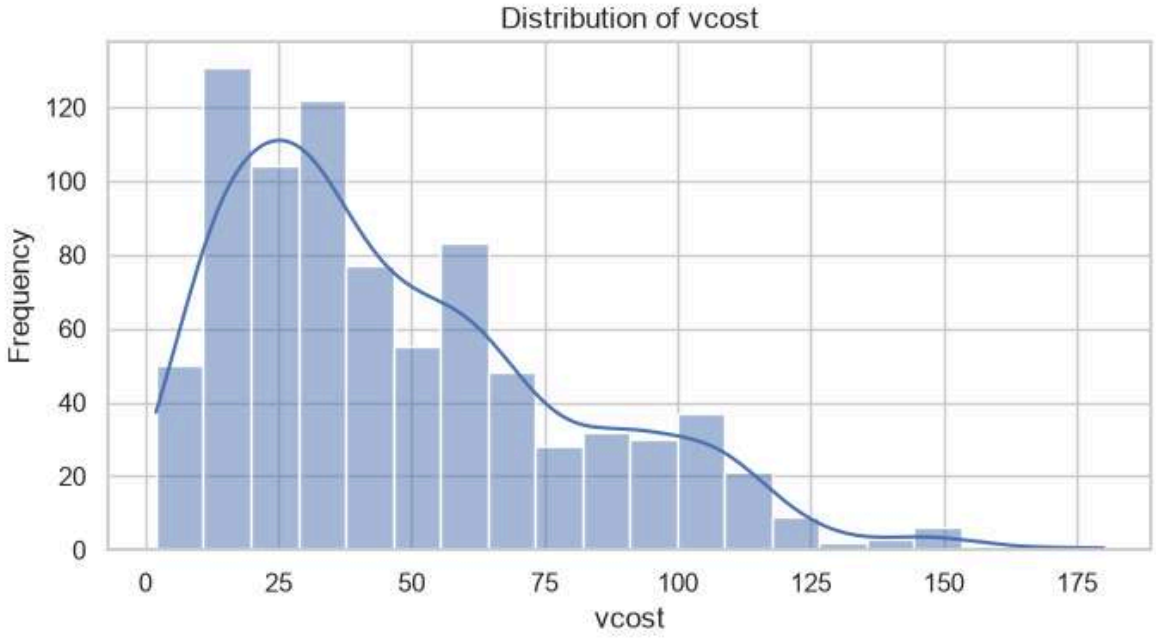
for col in numeric_cols:
    values = pd.to_numeric(df_m4[col], errors="coerce").dropna()

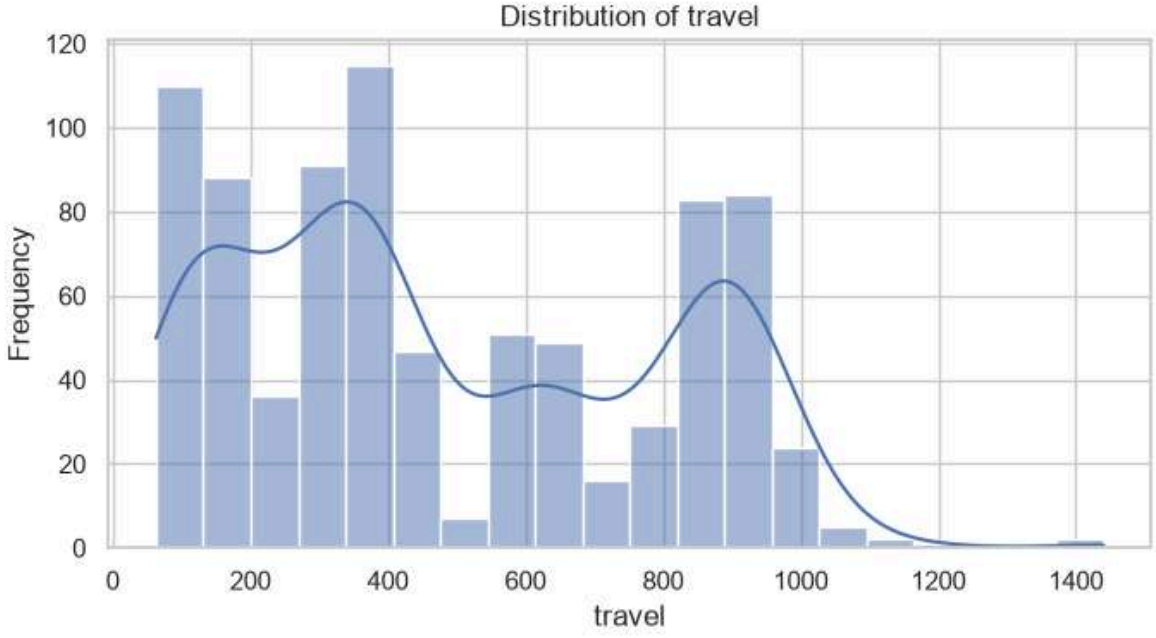
    if not values.empty:
        plt.figure(figsize=(7, 4))

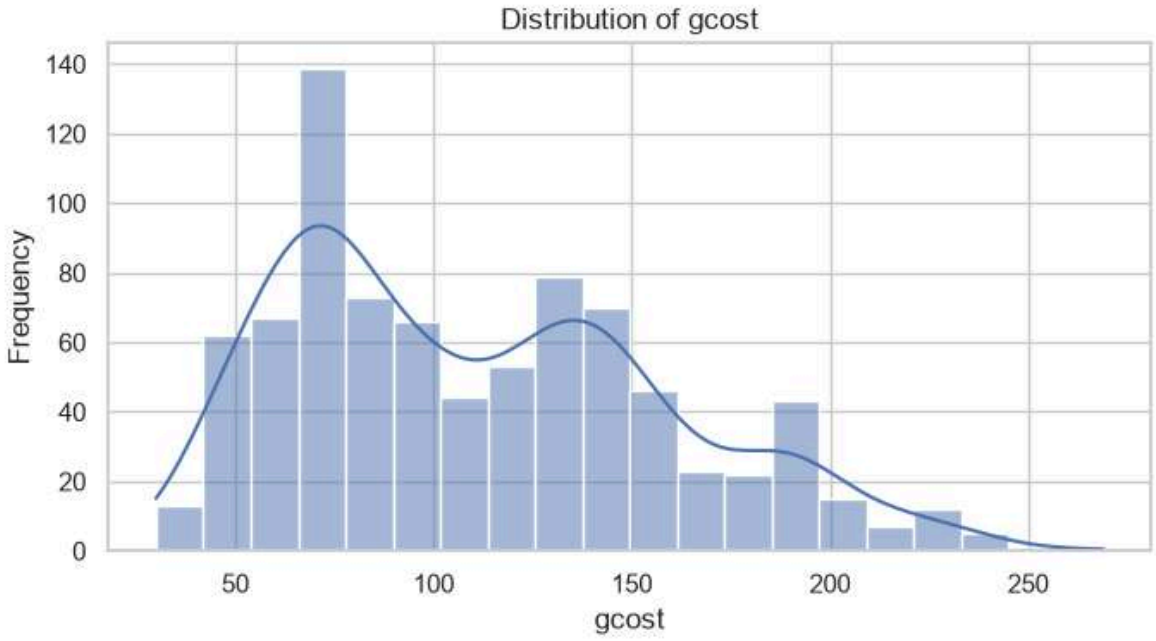
        sns.histplot(values, bins=20, kde=True)

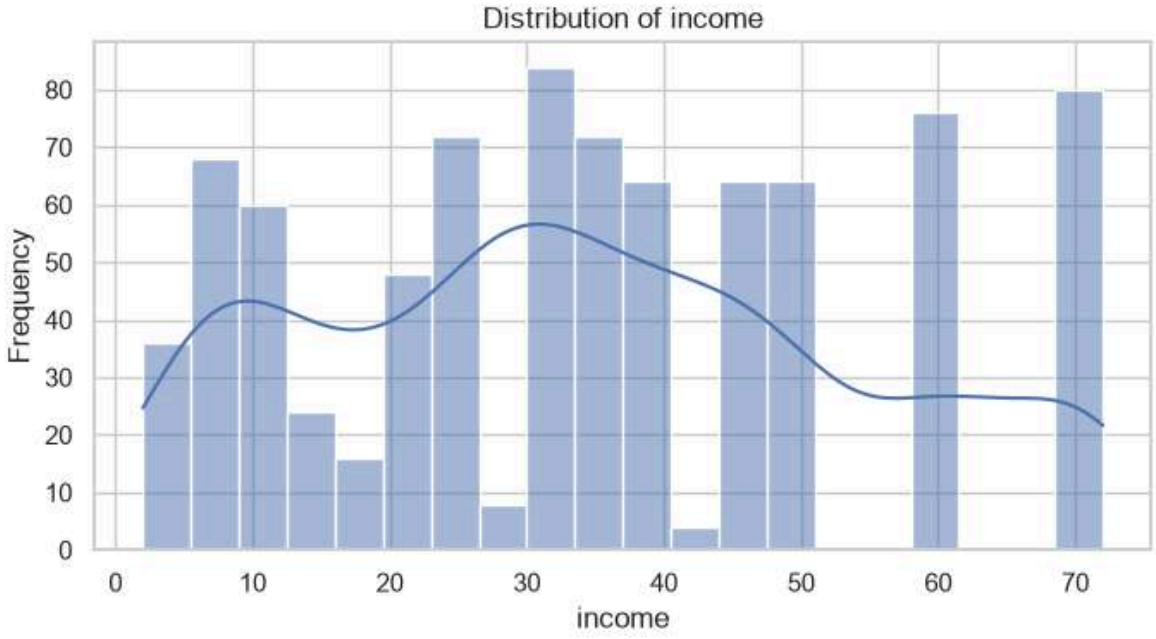
        plt.title(f"Distribution of {col}")
        plt.xlabel(col)
        plt.ylabel("Frequency")
        plt.tight_layout()
        plt.show()
```

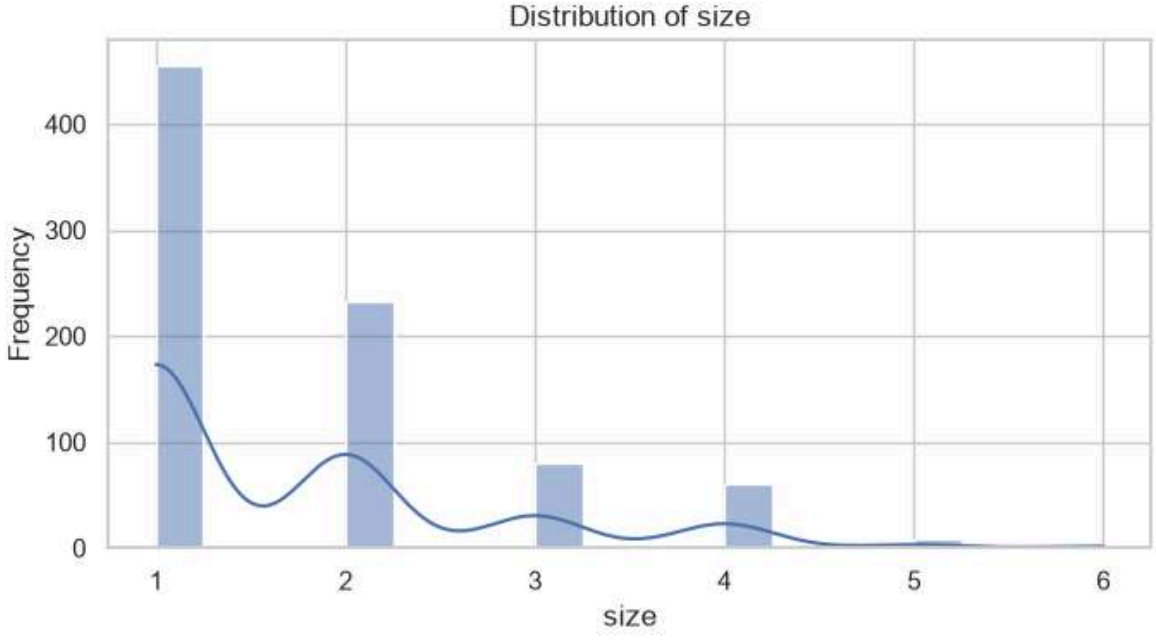

Distribution of wait
Frequency
wait
0
20
40
60
80
100
0
25
50
75
100
125
150
175
200


Distribution of vcost
Frequency
vcost
0
25
50
75
100
125
150
175
0
20
40
60
80
100
120


Distribution of travel
Frequency
travel
0
200
400
600
800
1000
1200
1400
0
20
40
60
80
100
120


Distribution of gcost
Frequency
gcost
0
20
40
60
80
100
120
140
50
100
150
200
250


Distribution of income
Frequency
income
0
10
20
30
40
50
60
70
80
0
10
20
30
40
50
60
70


Distribution of size
Frequency
size
0
100
200
300
400
1
2
3
4
5
6

Detailed description:

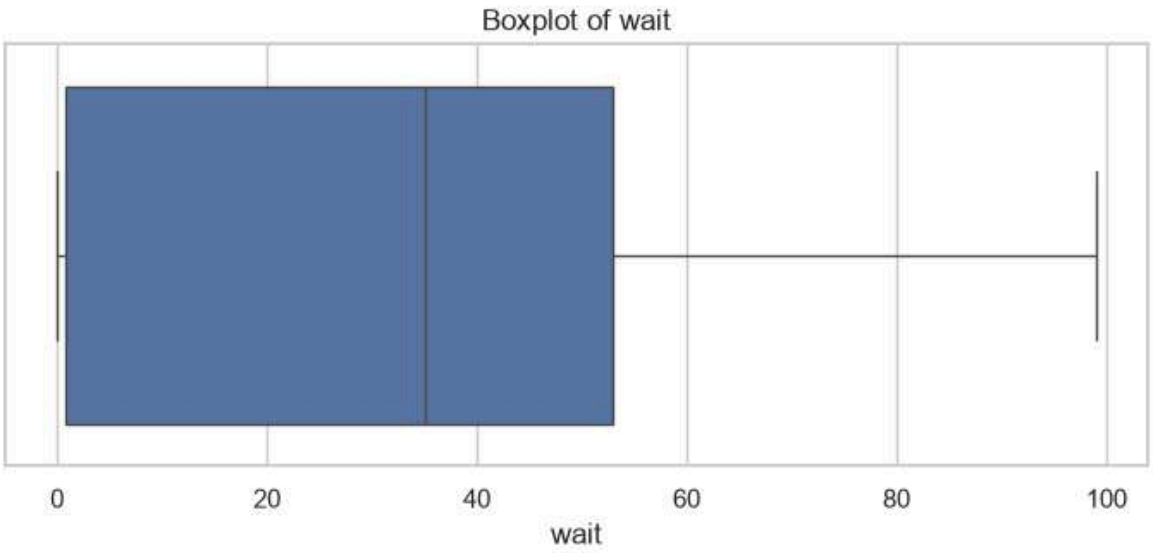
A boxplot displays the median as a line inside the box, the box represents the interquartile range, and the whiskers show the main range under the plotting convention. Individual points beyond the whiskers may be potential outliers.

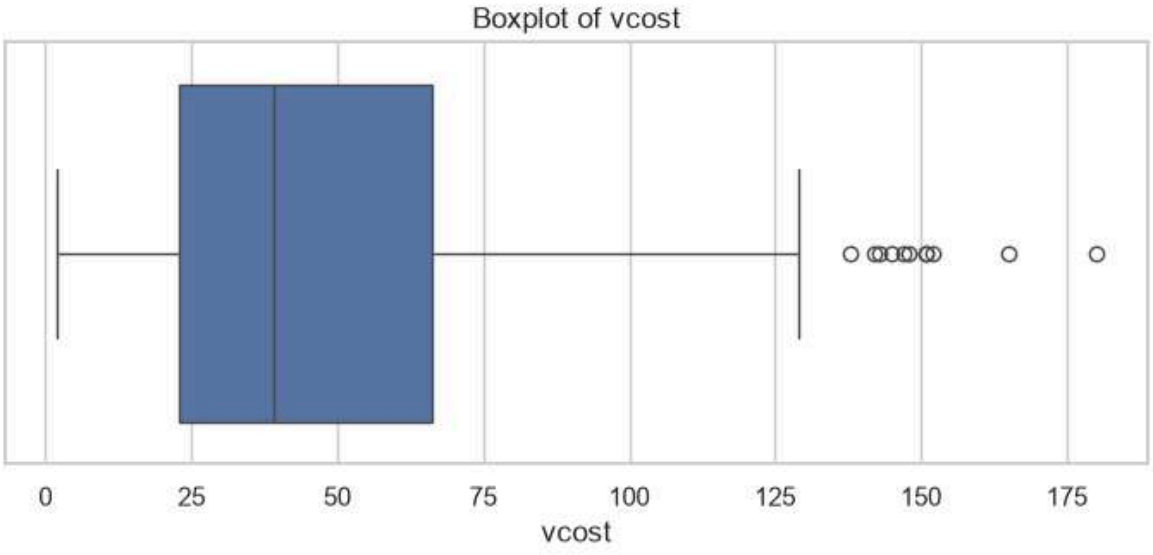
In [70]:
```
#Plot boxplots
for col in numeric_cols:
    values = pd.to_numeric(df_m4[col], errors="coerce").dropna()

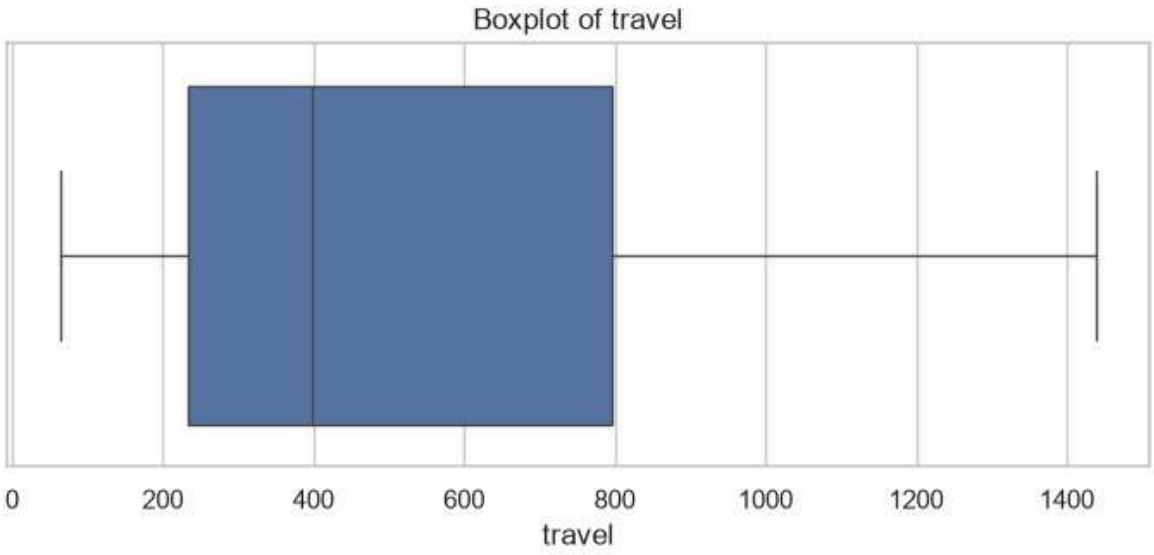
    if not values.empty:
        plt.figure(figsize=(7, 3.5))

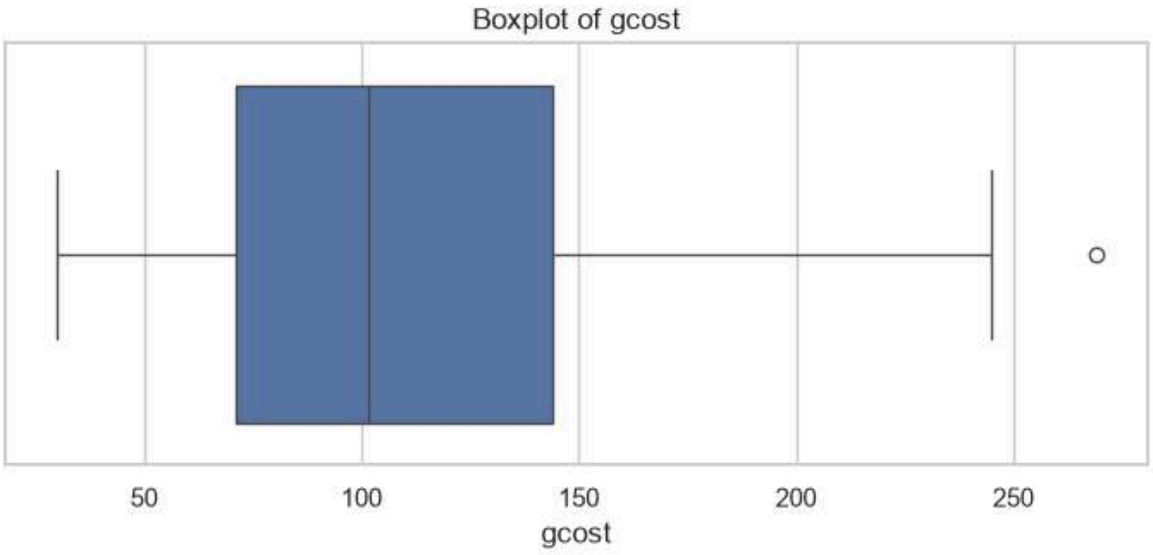
        sns.boxplot(x=values)

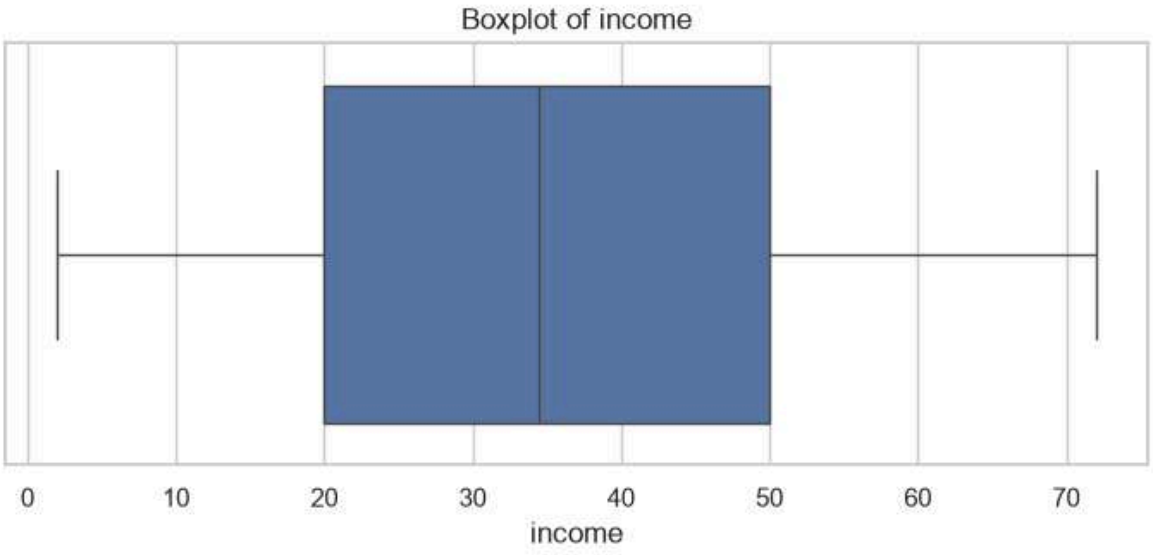
        plt.title(f"Boxplot of {col}")
        plt.xlabel(col)
        plt.tight_layout()
        plt.show()
```

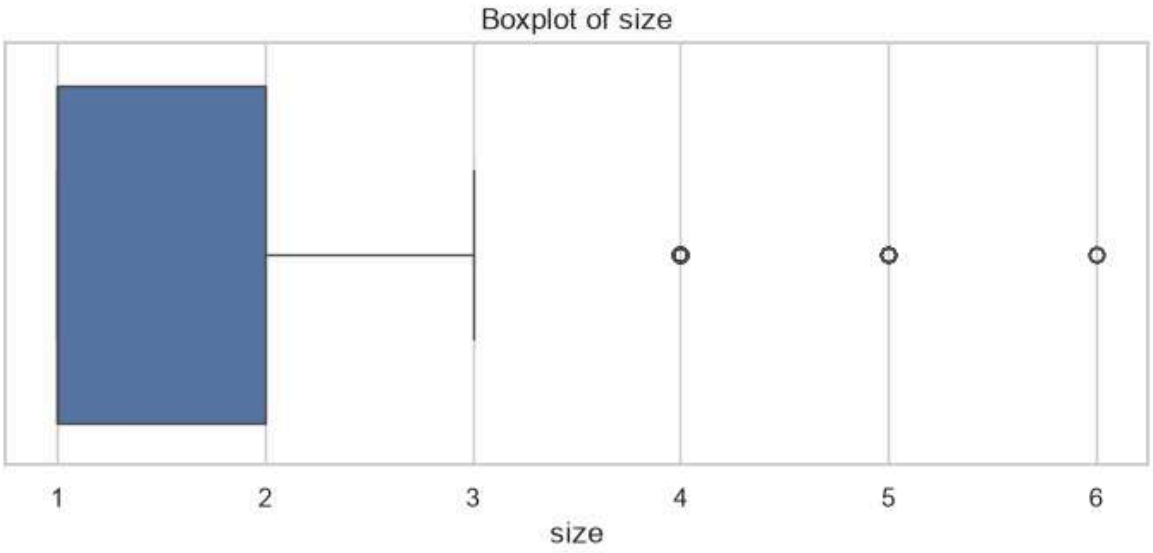





Boxplot of travel
0
200
400
600
800
1000
1200
1400
travel


Boxplot of gcost
50
100
150
200
250
gcost


Boxplot of income
0
10
20
30
40
50
60
70
income



Detailed description:

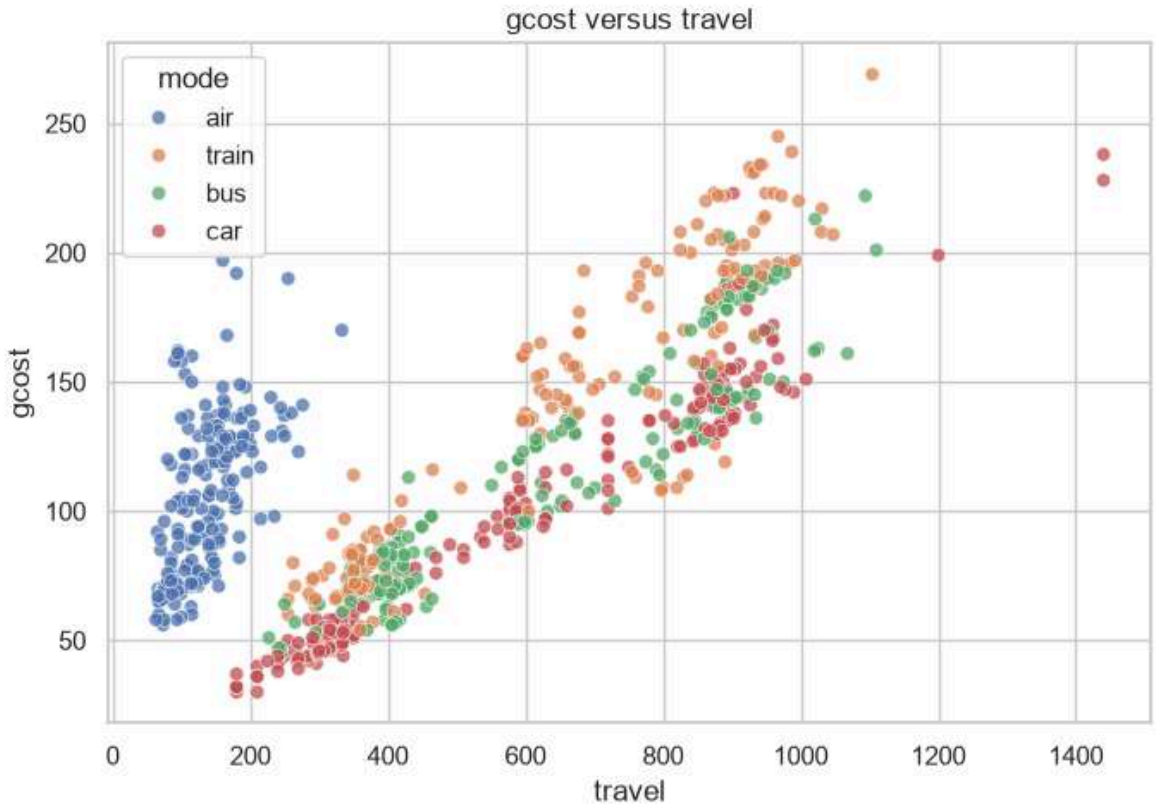
A scatter plot represents two variables on the x- and y-axes. The direction and spread of the points indicate whether the variables have a positive, negative or weak relationship.

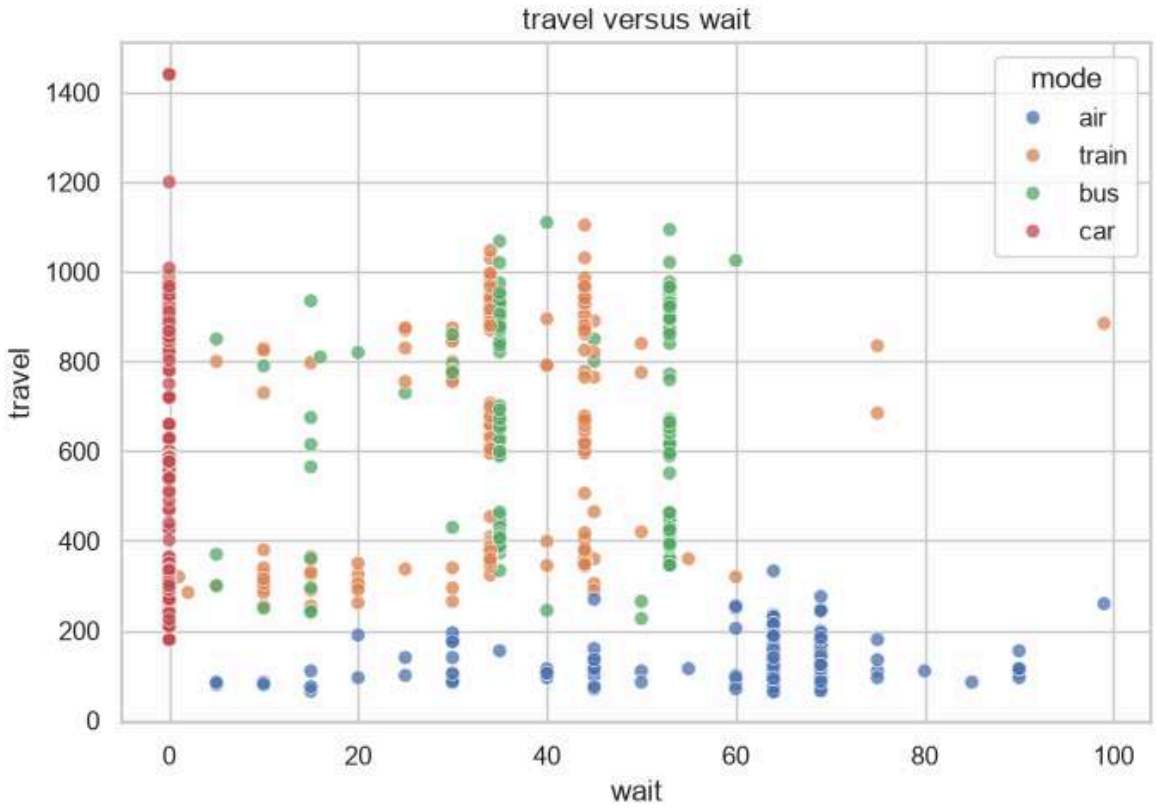
In [71]:
```
# Two-Dimensional (2-D) Statistical Analysis (Compare two numerical variables us
scatter_pairs = [
    ("travel", "gcost"),
    ("wait", "travel"),
    ("vcost", "gcost")
]

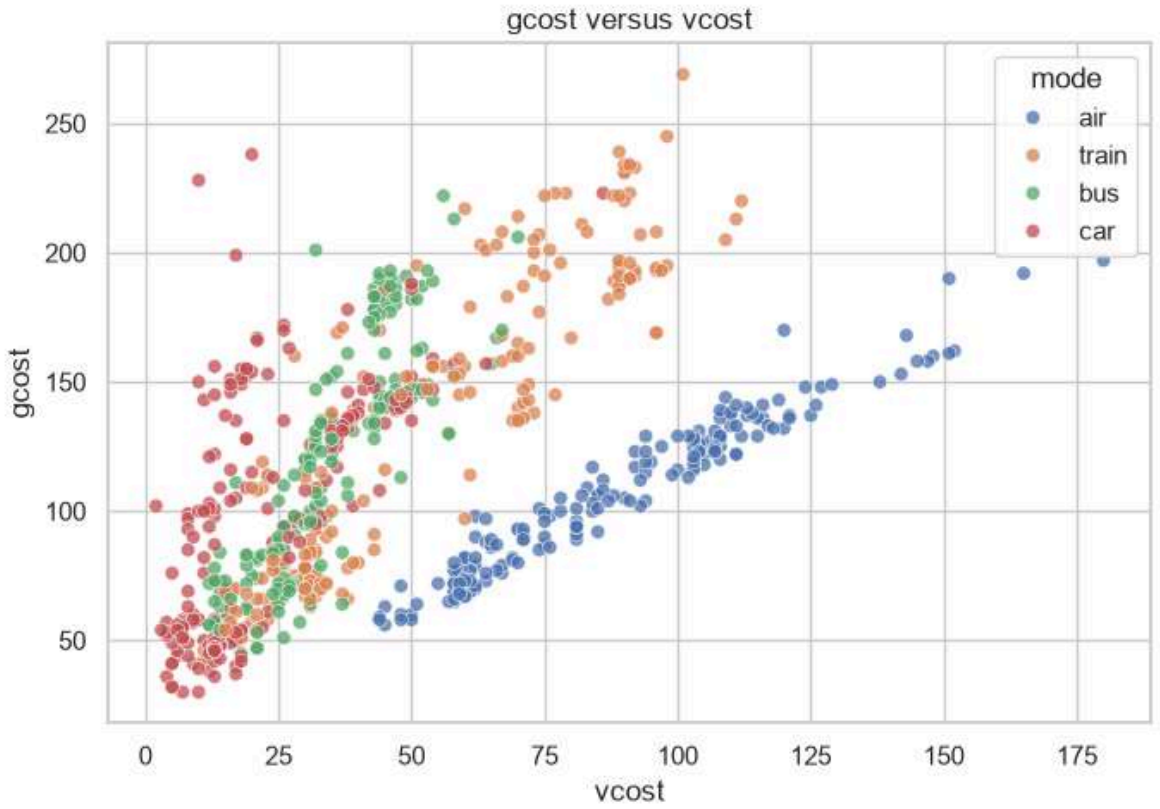
for x_col, y_col in scatter_pairs:
    if x_col in df_m4.columns and y_col in df_m4.columns:
        plt.figure(figsize=(7, 5))

        sns.scatterplot(
            data=df_m4,
            x=x_col,
            y=y_col,
            hue="mode" if "mode" in df_m4.columns else None,
            alpha=0.75
        )

        plt.title(f"{y_col} versus {x_col}")
        plt.xlabel(x_col)
        plt.ylabel(y_col)
        plt.tight_layout()
        plt.show()
```


gcost versus travel
mode
air
train
bus
car
gcost
travel
0
200
400
600
800
1000
1200
1400
50
100
150
200
250


travel versus wait
mode
air
train
bus
car
travel
wait
0
20
40
60
80
100
200
400
600
800
1000
1200
1400



Detailed description: Pearson correlation coefficients range from -1 to +1. They summarize linear association and complement the scatter plots.

In [72]:
```
# Calculate pairwise correlations
correlation_matrix = df_m4[numeric_cols].corr(numeric_only=True)

print("Correlation matrix:")
display(correlation_matrix.round(3))
```

```
Correlation matrix:
```

	wait	vcost	travel	gcost	income	size
wait	1.000	0.580	-0.332	0.158	0.013	0.101
vcost	0.580	1.000	-0.160	0.525	0.075	-0.125
travel	-0.332	-0.160	1.000	0.717	0.018	0.025
gcost	0.158	0.525	0.717	1.000	0.099	0.101
income	0.013	0.075	0.018	0.099	1.000	0.181
size	0.101	-0.125	0.025	0.101	0.181	1.000

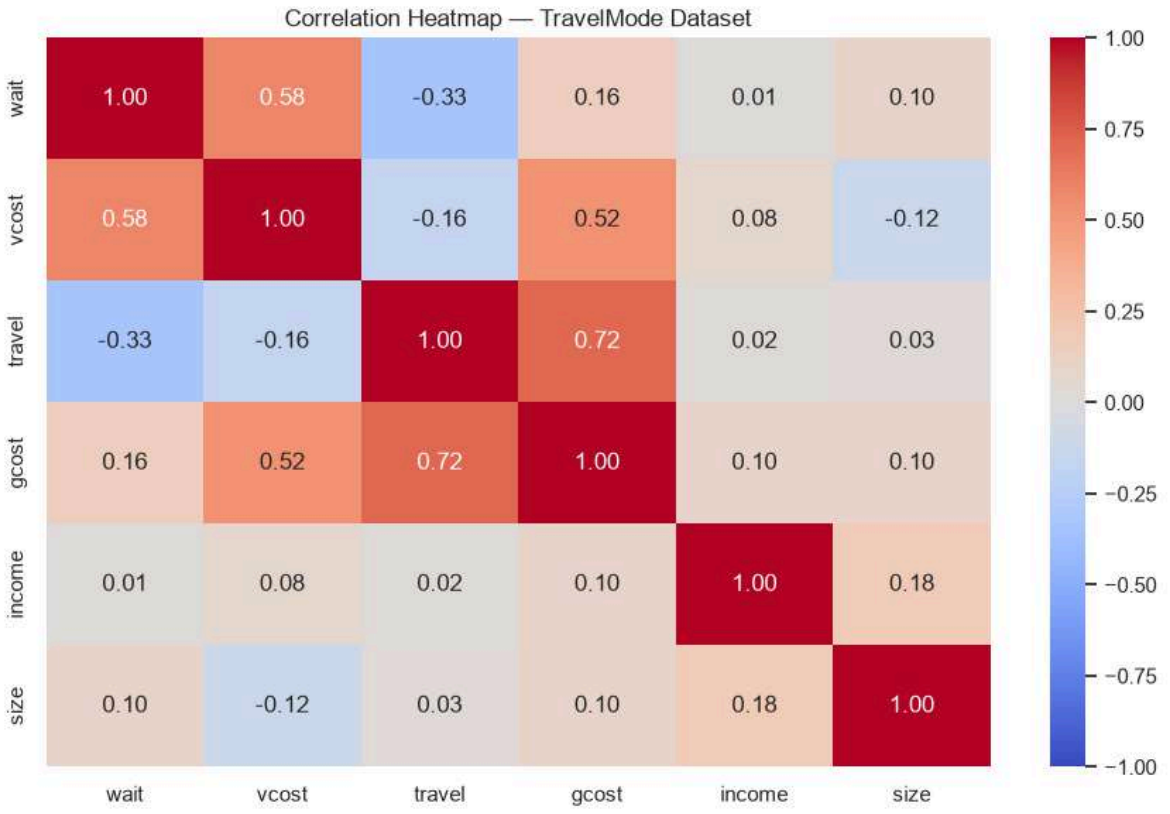
Detailed description:

A heatmap uses colour and numerical labels to make relationships between several variables easier to compare.

In [73]:
```
# Visualize the correlations
plt.figure(figsize=(9, 6))
```

```
sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0,
    vmin=-1,
    vmax=1
)

plt.title("Correlation Heatmap — TravelMode Dataset")
plt.tight_layout()
plt.show()
```



Detailed description:

A contingency table summarizes the frequency of combinations of two categorical variables. Here, we compare mode with choice to understand how the recorded choices are distributed across transport alternatives.

In [74]:
```
# Contingency Tables and n-Dimensional Analysis
if {"mode", "choice"}.issubset(df_m4.columns):
    contingency_table = pd.crosstab(
        df_m4["mode"],
        df_m4["choice"],
        margins=True
    )

    print("Contingency table: Mode vs Choice")
    display(contingency_table)
```

```
else:
    print("The mode or choice column is missing.")
```

```
Contingency table: Mode vs Choice
```

choice	no	yes	All
mode			
air	152	58	210
bus	180	30	210
car	151	59	210
train	147	63	210
All	630	210	840

Detailed description:

Row-wise percentages normalize each mode's counts to 100%, making it easier to compare categories even when their total frequencies differ.

In [75]:
```
#Calculate row-wise percentages
if {"mode", "choice"}.issubset(df_m4.columns):
    row_percentages = pd.crosstab(
        df_m4["mode"],
        df_m4["choice"],
        normalize="index"
    ) * 100

    print("Choice percentages within each mode:")
    display(row_percentages.round(2))
```

```
Choice percentages within each mode:
```

choice	no	yes
mode		
air	72.38	27.62
bus	85.71	14.29
car	71.90	28.10
train	70.00	30.00

Detailed description:

n-dimensional analysis considers multiple variables together. Grouping by both mode and choice lets us compare the count, mean and median of travel metrics across combinations of categories.

In [76]:
```
# Perform n-dimensional statistical analysis
group_cols = [
    col for col in ["mode", "choice"]
    if col in df_m4.columns
]

metric_cols = [
    col for col in ["wait", "vcost", "travel", "gcost"]
    if col in df_m4.columns
]

if group_cols and metric_cols:
    multidimensional_summary = (
        df_m4.groupby(group_cols, observed=True)[metric_cols]
        .agg(["count", "mean", "median"])
        .round(2)
    )

    display(multidimensional_summary)
else:
    print("Required grouping or metric columns are not available.")
```

		wait			vcost			travel		
		count	mean	median	count	mean	median	count	mean	median	cou
mode	choice										
air	no	152	66.53	69.0	152	80.55	72.5	152	137.10	138.0	1
	yes	58	46.53	45.0	58	97.57	93.5	58	124.83	110.0	
bus	no	180	44.40	53.0	180	33.41	31.0	180	631.23	595.0	1
	yes	30	25.20	22.5	30	33.73	35.0	30	618.83	750.0	
car	no	151	0.00	0.0	151	23.09	17.0	151	591.11	577.0	1
	yes	59	0.00	0.0	59	15.64	13.0	59	527.37	510.0	
train	no	147	38.76	34.0	147	57.29	59.0	147	640.69	622.0	1
	yes	63	28.52	25.0	63	37.46	30.0	63	532.67	365.0	

Detailed description:

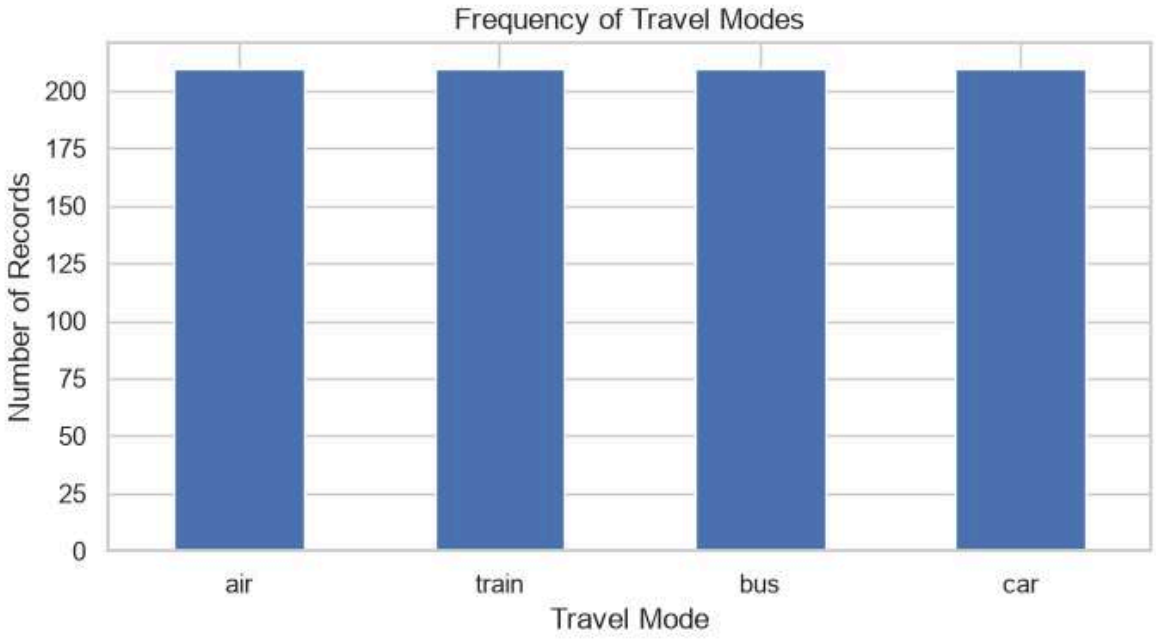
A bar plot compares categorical counts. The height of each bar corresponds to the number of records associated with a mode.

In [77]:
```
#Create a bar plot of travel-mode frequencies
if "mode" in df_m4.columns:
    mode_counts = df_m4["mode"].value_counts()

    plt.figure(figsize=(7, 4))
    mode_counts.plot(kind="bar")
```

```
    plt.title("Frequency of Travel Modes")
    plt.xlabel("Travel Mode")
    plt.ylabel("Number of Records")
    plt.xticks(rotation=0)
    plt.tight_layout()
    plt.show()

    print("Travel-mode frequencies:")
    display(mode_counts.to_frame("Frequency"))
```




```
Travel-mode frequencies:
```

mode	Frequency
air	210
train	210
bus	210
car	210

Detailed description:

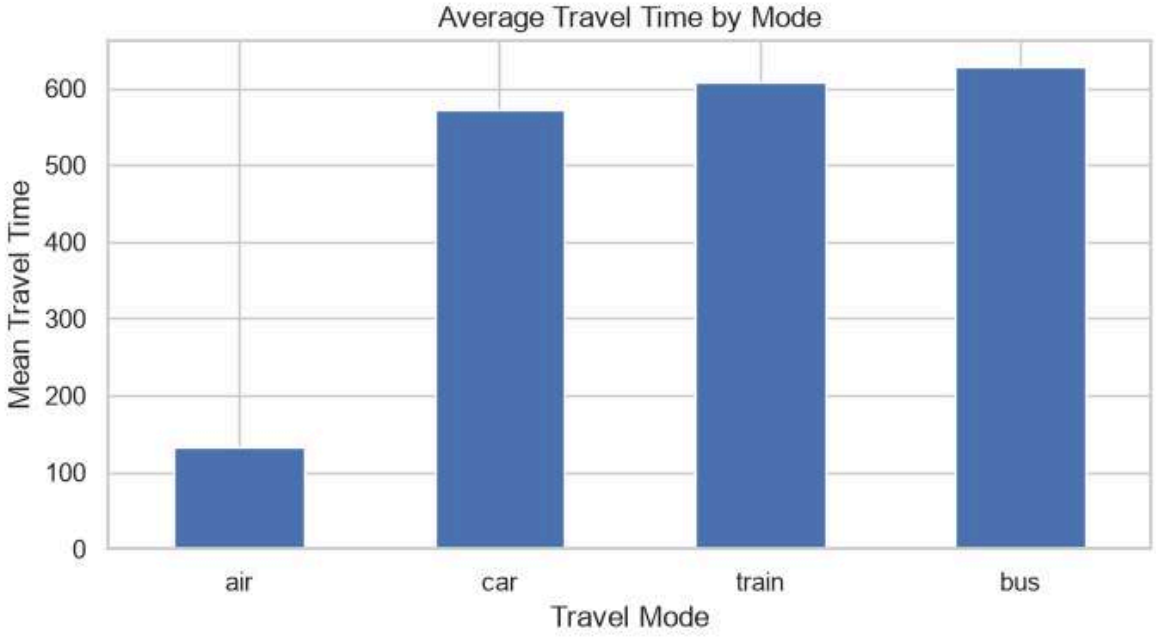
Instead of plotting counts, this bar plot displays the mean travel value for each mode. It helps compare the average recorded travel time across categories.

In [78]:
```
# Create a bar plot of average travel time by mode
if {"mode", "travel"}.issubset(df_m4.columns):
    mean_travel = (
        df_m4.groupby("mode", observed=True)["travel"]
        .mean()
        .sort_values()
    )

    plt.figure(figsize=(7, 4))
    mean_travel.plot(kind="bar")
```

```
    plt.title("Average Travel Time by Mode")
    plt.xlabel("Travel Mode")
    plt.ylabel("Mean Travel Time")
    plt.xticks(rotation=0)
    plt.tight_layout()
    plt.show()

    display(mean_travel.to_frame("Mean Travel Time").round(2))
```



mode	Mean Travel Time
air	133.71
car	573.20
train	608.29
bus	629.46

Detailed description:

A dot chart represents each mode's average generalized cost as a point. It provides a compact alternative to a bar plot and makes differences between group averages easier to inspect.

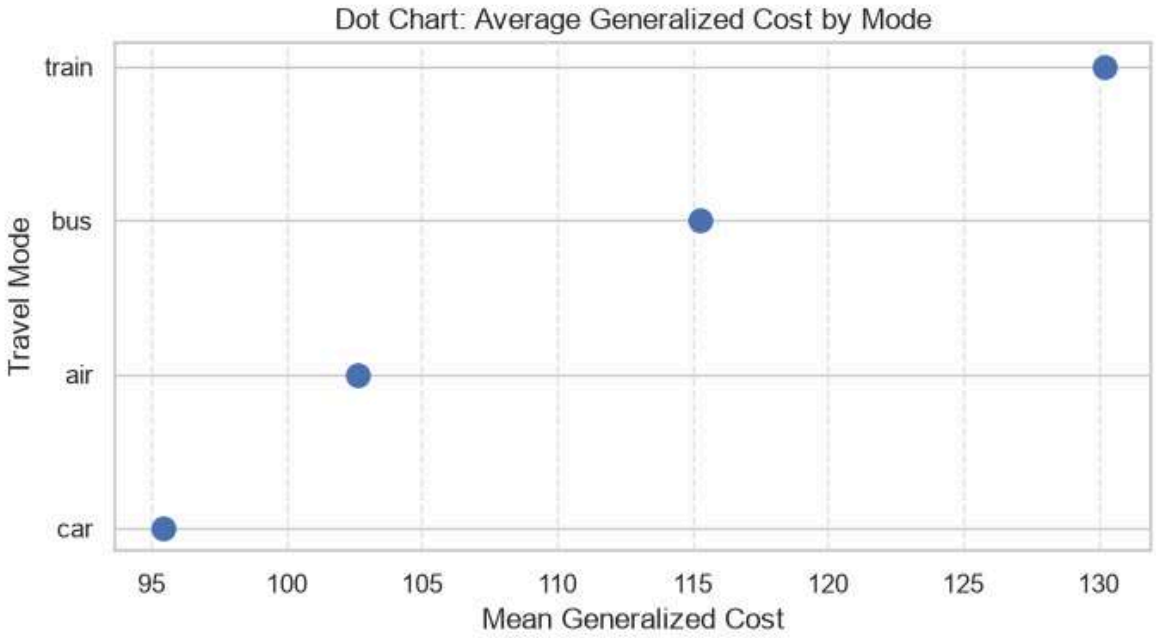
In [79]:
```
#Create a dot chart
if {"mode", "gcost"}.issubset(df_m4.columns):
    mean_gcost = (
        df_m4.groupby("mode", observed=True)["gcost"]
        .mean()
        .sort_values()
    )

    plt.figure(figsize=(7, 4))
```

```
    plt.scatter(
        mean_gcost.values,
        mean_gcost.index,
        s=90
    )

    plt.title("Dot Chart: Average Generalized Cost by Mode")
    plt.xlabel("Mean Generalized Cost")
    plt.ylabel("Travel Mode")
    plt.grid(axis="x", linestyle="--", alpha=0.5)
    plt.tight_layout()
    plt.show()

    print("Average generalized cost by mode:")
    display(mean_gcost.to_frame("Mean Generalized Cost").round(2))
```




```
Average generalized cost by mode:
```

mode	Mean Generalized Cost
car	95.41
air	102.65
bus	115.26
train	130.20

Conclusion

Data Summarization and Visualization were performed on the TravelMode dataset using Python in Jupyter Notebook. Statistical measures such as mean, median, variance, standard deviation and interquartile range were calculated to summarize numerical variables. Histograms and boxplots were used for one-dimensional analysis, while scatter plots and correlation heatmaps were used to investigate relationships between numerical

variables. Contingency tables were generated to compare travel modes and choice categories, and grouped summaries were used for n-dimensional analysis. Bar plots and dot charts were created to visualize travel-mode frequencies and average travel metrics. The results were interpreted using the actual numerical values and patterns observed in the graphs.

In [82]:
```
# Prepare and scale data for clustering

import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler

# Use the cleaned dataset from Module 2
df_m5 = df_imputed.copy()

# Select numerical features available in the dataset
cluster_features = [
    col for col in ["wait", "vcost", "travel", "gcost", "income", "size"]
    if col in df_m5.columns
]

print("Selected features:", cluster_features)

# Convert selected columns to numeric values
X_cluster = df_m5[cluster_features].apply(
    pd.to_numeric, errors="coerce"
)

# Handle infinite and missing values
X_cluster = X_cluster.replace([np.inf, -np.inf], np.nan)
X_cluster = X_cluster.fillna(X_cluster.median()).fillna(0)

# Standardize the features
scaler_m5 = StandardScaler()
X_scaled_m5 = scaler_m5.fit_transform(X_cluster)

print("Scaled data shape:", X_scaled_m5.shape)
print("Missing values after cleaning:", np.isnan(X_scaled_m5).sum())
```

```
Selected features: ['wait', 'vcost', 'travel', 'gcost', 'income', 'size']
Scaled data shape: (840, 6)
Missing values after cleaning: 0
```

Detailed description:

K-Means assigns each record to one of three groups based on similarity across the standardized numerical features.

In [86]:
```
# K-MEANS CLUSTERING
if len(X_scaled_m5) < 3:
    raise ValueError("K-Means with three clusters requires at least three record

kmeans_m5 = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
```

```
)

kmeans_labels_m5 = kmeans_m5.fit_predict(X_scaled_m5)
df_m5["kmeans_cluster"] = kmeans_labels_m5

print("K-Means cluster sizes:")
display(
    df_m5["kmeans_cluster"]
    .value_counts()
    .sort_index()
    .to_frame("Number of records")
)

print("K-Means inertia:", round(kmeans_m5.inertia_, 3))
```

```
K-Means cluster sizes:
```

kmeans_cluster	Number of records
0	288
1	203
2	349

```
K-Means inertia: 2871.562
```

Detailed description:

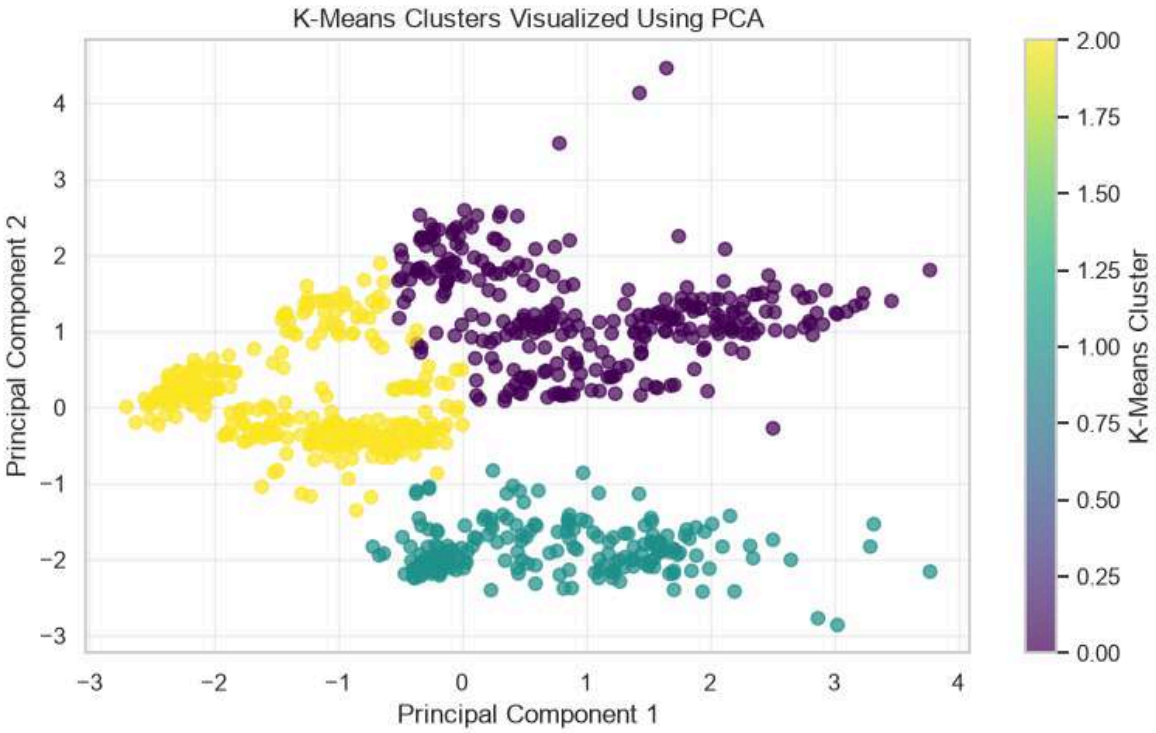
PCA reduces the standardized features to two components for visualization. Each point represents one observation, and its colour indicates its K-means cluster. The axes represent principal components, not individual travel variables.

In [89]:
```
# PCA visualization of K-Means clusters
pca_m5 = PCA(n_components=2)
X_pca_m5 = pca_m5.fit_transform(X_scaled_m5)

plt.figure(figsize=(8, 5))
scatter = plt.scatter(
    X_pca_m5[:, 0],
    X_pca_m5[:, 1],
    c=kmeans_labels_m5,
    cmap="viridis",
    alpha=0.7
)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-Means Clusters Visualized Using PCA")
plt.colorbar(scatter, label="K-Means Cluster")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(
    "Variance explained by first two components:",
```

```
    round(pca_m5.explained_variance_ratio_.sum(), 4)
)
```




```
Variance explained by first two components: 0.6003
```

Detailed description:

Ward linkage merges groups based on changes in within-cluster variation. The dendrogram is truncated for readability, and the model assigns three hierarchical clusters for comparison.

In [92]:
```
# HIERARCHICAL CLUSTERING AND DENDROGRAM
from scipy.cluster.hierarchy import linkage, dendrogram
import matplotlib.pyplot as plt


# MODULE 5.4: HIERARCHICAL CLUSTERING AND DENDROGRAM

Z_m5 = linkage(X_scaled_m5, method="ward")

plt.figure(figsize=(13, 6))
dendrogram(
    Z_m5,
    truncate_mode="lastp",
    p=30,
    leaf_rotation=90,
    leaf_font_size=9
)

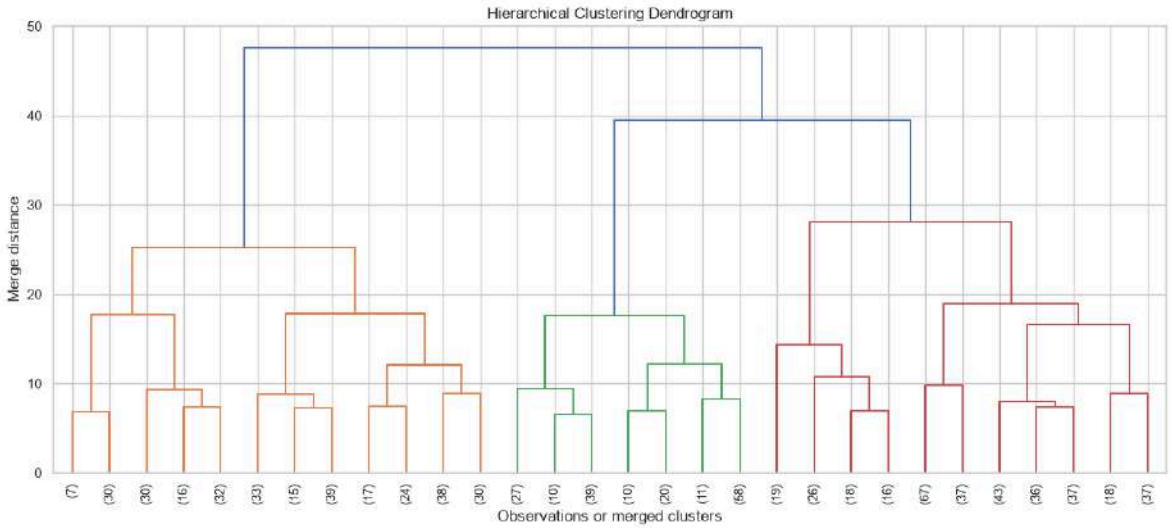
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Observations or merged clusters")
plt.ylabel("Merge distance")
plt.tight_layout()
plt.show()
```

```
from sklearn.cluster import AgglomerativeClustering

hierarchical_m5 = AgglomerativeClustering(
    n_clusters=3,
    linkage="ward"
)

hierarchical_labels_m5 = hierarchical_m5.fit_predict(X_scaled_m5)
df_m5["hierarchical_cluster"] = hierarchical_labels_m5

print("Hierarchical cluster sizes:")
display(
    df_m5["hierarchical_cluster"]
    .value_counts()
    .sort_index()
    .to_frame("Number of records")
)
```




```
Hierarchical cluster sizes:
```

hierarchical_cluster	Number of records
0	354
1	311
2	175

Detailed description:

DBSCAN groups points that are close together and sufficiently dense. Unlike K-Means, it does not require a predefined number of clusters. The eps and min_samples parameters control how clusters are formed.

In [95]:
```
# DBSCAN AND NEAREST-NEIGHBOR DISTANCE GRAPH

n_samples_m5 = len(X_scaled_m5)

if n_samples_m5 < 3:
    raise ValueError("DBSCAN analysis requires at least three records.")
```

```
k_m5 = min(5, n_samples_m5 - 1)

neighbors_m5 = NearestNeighbors(n_neighbors=k_m5 + 1)
neighbors_m5.fit(X_scaled_m5)
distances_m5, _ = neighbors_m5.kneighbors(X_scaled_m5)

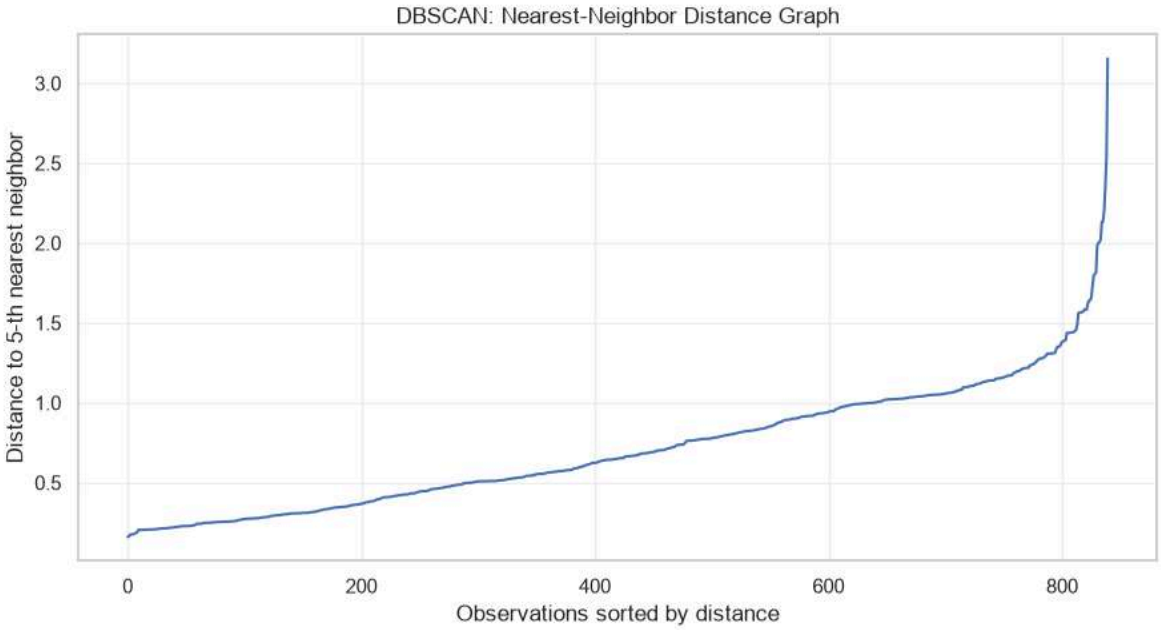
k_distances_m5 = np.sort(distances_m5[:, -1])

plt.figure(figsize=(9, 5))
plt.plot(k_distances_m5)
plt.xlabel("Observations sorted by distance")
plt.ylabel(f"Distance to {k_m5}-th nearest neighbor")
plt.title("DBSCAN: Nearest-Neighbor Distance Graph")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Initial eps value; adjust after examining the graph
dbscan_m5 = DBSCAN(eps=1.5, min_samples=5)
dbscan_labels_m5 = dbscan_m5.fit_predict(X_scaled_m5)
df_m5["dbscan_cluster"] = dbscan_labels_m5

print("DBSCAN cluster sizes (-1 means noise):")
display(
    df_m5["dbscan_cluster"]
    .value_counts()
    .sort_index()
    .to_frame("Number of records")
)

print("Number of clusters:", len(set(dbscan_labels_m5) - {-1}))
print("Number of noise records:", int(np.sum(dbscan_labels_m5 == -1)))
```




```
DBSCAN cluster sizes (-1 means noise):
```

dbscan_cluster	Number of records
-1	9
0	831

```
Number of clusters: 1
Number of noise records: 9
```

Detailed description:

Spectral clustering builds a similarity graph and uses its structure to identify groups. Unlike K-means, it can identify some non-spherical cluster shapes. It can be computationally expensive on large datasets.

In [98]:
```
# SPECTRAL CLUSTERING

spectral_m5 = SpectralClustering(
    n_clusters=3,
    affinity="nearest_neighbors",
    n_neighbors=min(10, len(X_scaled_m5) - 1),
    random_state=42
)

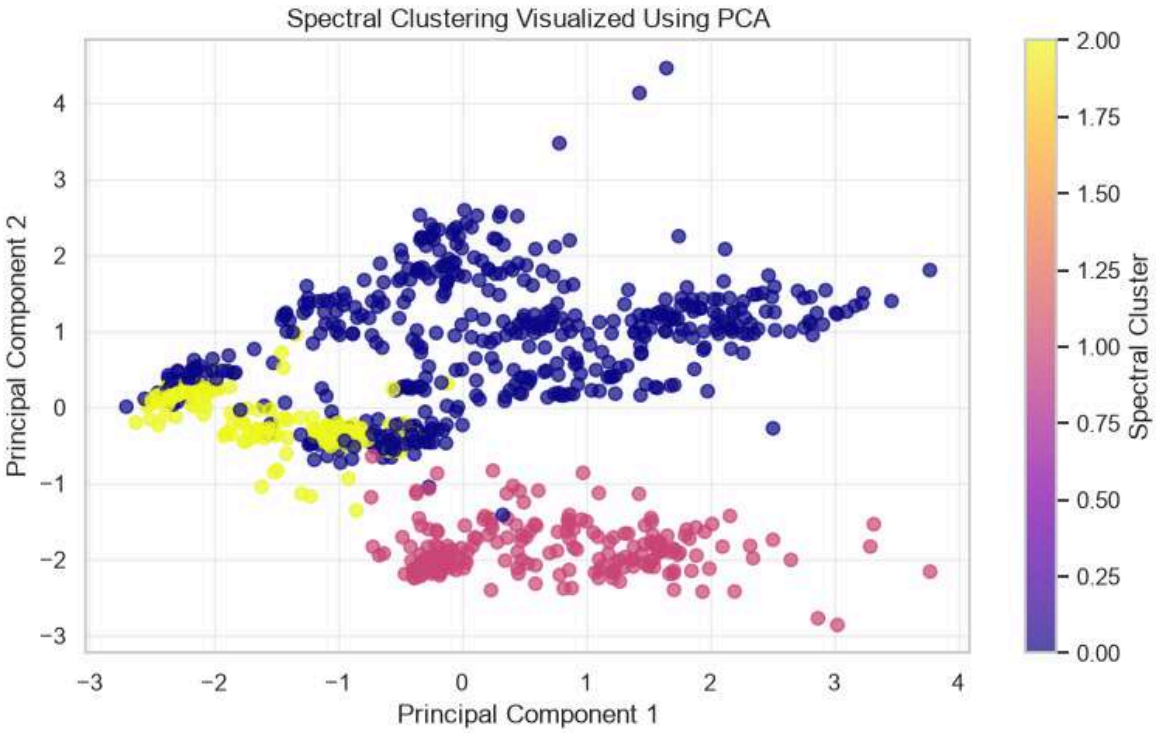
spectral_labels_m5 = spectral_m5.fit_predict(X_scaled_m5)
df_m5["spectral_cluster"] = spectral_labels_m5

print("Spectral cluster sizes:")
display(
    df_m5["spectral_cluster"]
    .value_counts()
    .sort_index()
    .to_frame("Number of records")
)

plt.figure(figsize=(8, 5))
plt.scatter(
    X_pca_m5[:, 0],
    X_pca_m5[:, 1],
    c=spectral_labels_m5,
    cmap="plasma",
    alpha=0.7
)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Spectral Clustering Visualized Using PCA")
plt.colorbar(label="Spectral Cluster")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Spectral cluster sizes:
```

spectral_cluster	Number of records
0	475
1	204
2	161



Detailed description:

The silhouette score ranges from -1 to 1. Higher values generally indicate better-separated clusters. For DBSCAN, noise points are excluded from the score.

In [100…

```
# Compare silhouette scores
# SILHOUETTE SCORE COMPARISON

def safe_silhouette_m5(X, labels, exclude_noise=False):
    labels = np.asarray(labels)

    if exclude_noise:
        mask = labels != -1
        X_eval = X[mask]
        labels_eval = labels[mask]
    else:
        X_eval = X
        labels_eval = labels

    unique_labels = np.unique(labels_eval)

    if 2 <= len(unique_labels) < len(labels_eval):
        return round(silhouette_score(X_eval, labels_eval), 4)
```

```
    return np.nan

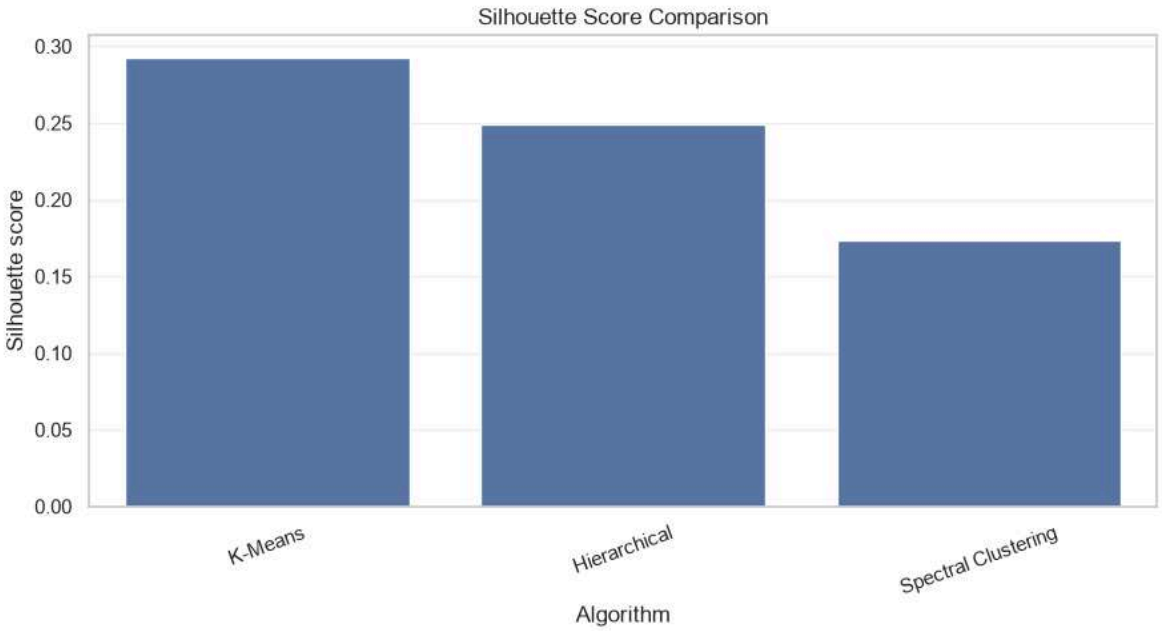
silhouette_results_m5 = pd.DataFrame([
    {
        "Algorithm": "K-Means",
        "Silhouette score": safe_silhouette_m5(
            X_scaled_m5, kmeans_labels_m5
        )
    },
    {
        "Algorithm": "Hierarchical",
        "Silhouette score": safe_silhouette_m5(
            X_scaled_m5, hierarchical_labels_m5
        )
    },
    {
        "Algorithm": "DBSCAN",
        "Silhouette score": safe_silhouette_m5(
            X_scaled_m5, dbscan_labels_m5, exclude_noise=True
        )
    },
    {
        "Algorithm": "Spectral Clustering",
        "Silhouette score": safe_silhouette_m5(
            X_scaled_m5, spectral_labels_m5
        )
    }
])

display(silhouette_results_m5)

plt.figure(figsize=(9, 5))
valid_scores_m5 = silhouette_results_m5.dropna(
    subset=["Silhouette score"]
)

if not valid_scores_m5.empty:
    sns.barplot(
        data=valid_scores_m5,
        x="Algorithm",
        y="Silhouette score"
    )
    plt.xticks(rotation=20)
    plt.title("Silhouette Score Comparison")
    plt.grid(axis="y", alpha=0.3)
    plt.tight_layout()
    plt.show()
else:
    print("No valid silhouette scores could be calculated.")
```

	Algorithm	Silhouette score
0	K-Means	0.2923
1	Hierarchical	0.2487
2	DBSCAN	NaN
3	Spectral Clustering	0.1732



Detailed description:

This demonstration uses a reproducible sample of up to 100 records to limit computation. Each edge represents a distance between two observations, and the MST connects all sampled points with the minimum possible total edge weight.

In [102…

```
# MINIMUM SPANNING TREE (MST)

from sklearn.metrics import pairwise_distances
from scipy.sparse.csgraph import minimum_spanning_tree
import matplotlib.pyplot as plt
import numpy as np


mst_sample_size_m5 = min(100, len(X_scaled_m5))
mst_sample_m5 = X_scaled_m5[:mst_sample_size_m5]

distance_matrix_m5 = pairwise_distances(
    mst_sample_m5,
    metric="euclidean"
)

mst_m5 = minimum_spanning_tree(csr_matrix(distance_matrix_m5))
mst_dense_m5 = mst_m5.toarray()
mst_edges_m5 = np.argwhere(mst_dense_m5 > 0)

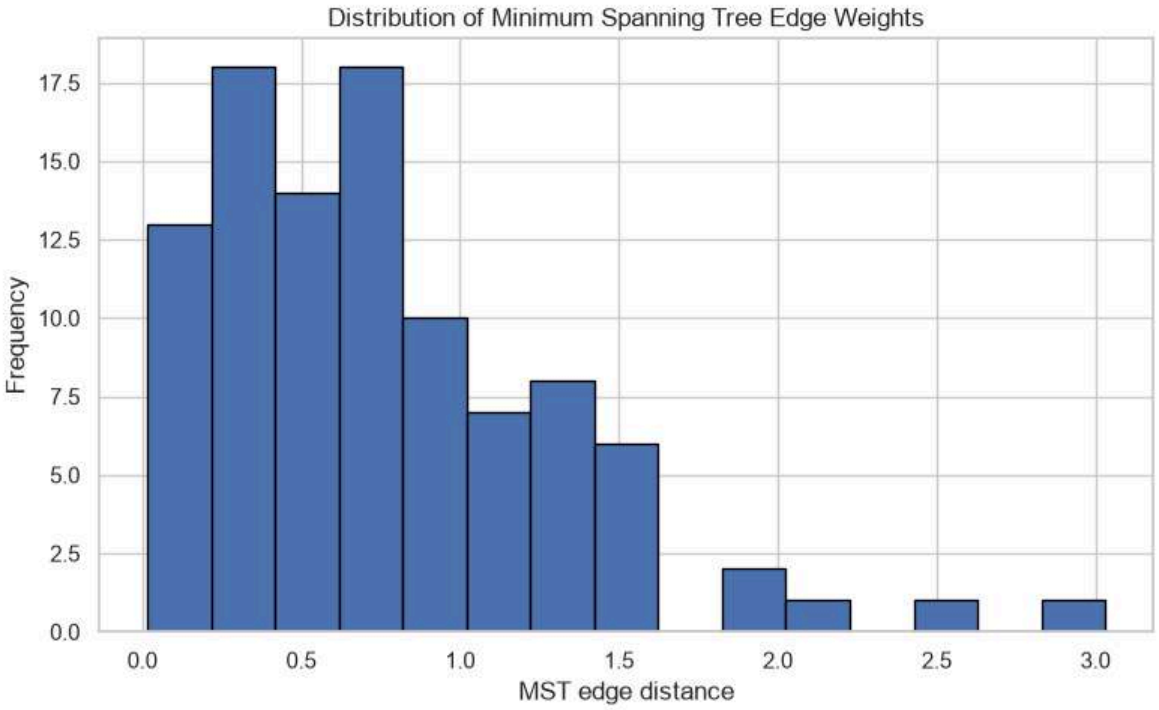
edge_weights_m5 = mst_dense_m5[
    mst_dense_m5 > 0
]

print("Number of sampled records:", mst_sample_size_m5)
print("Number of MST edges:", len(edge_weights_m5))
print("Expected edges in a connected tree:", mst_sample_size_m5 - 1)
print("Longest MST edge:", round(edge_weights_m5.max(), 4))

plt.figure(figsize=(8, 5))
plt.hist(edge_weights_m5, bins=15, edgecolor="black")
```

```
plt.xlabel("MST edge distance")
plt.ylabel("Frequency")
plt.title("Distribution of Minimum Spanning Tree Edge Weights")
plt.tight_layout()
plt.show()
```

```
Number of sampled records: 100
Number of MST edges: 99
Expected edges in a connected tree: 99
Longest MST edge: 3.0306
```



Detailed description:

GMM models the data as a mixture of three probability distributions and assigns each observation to the component with the highest estimated probability.

In [105…

```
# GAUSSIAN MIXTURE MODEL

from sklearn.mixture import GaussianMixture
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd


gmm_m5 = GaussianMixture(
    n_components=3,
    covariance_type="full",
    random_state=42,
    n_init=3
)

gmm_labels_m5 = gmm_m5.fit_predict(X_scaled_m5)
df_m5["gmm_cluster"] = gmm_labels_m5

print("GMM cluster sizes:")
display(
```

```
    df_m5["gmm_cluster"]
    .value_counts()
    .sort_index()
    .to_frame("Number of records")
)

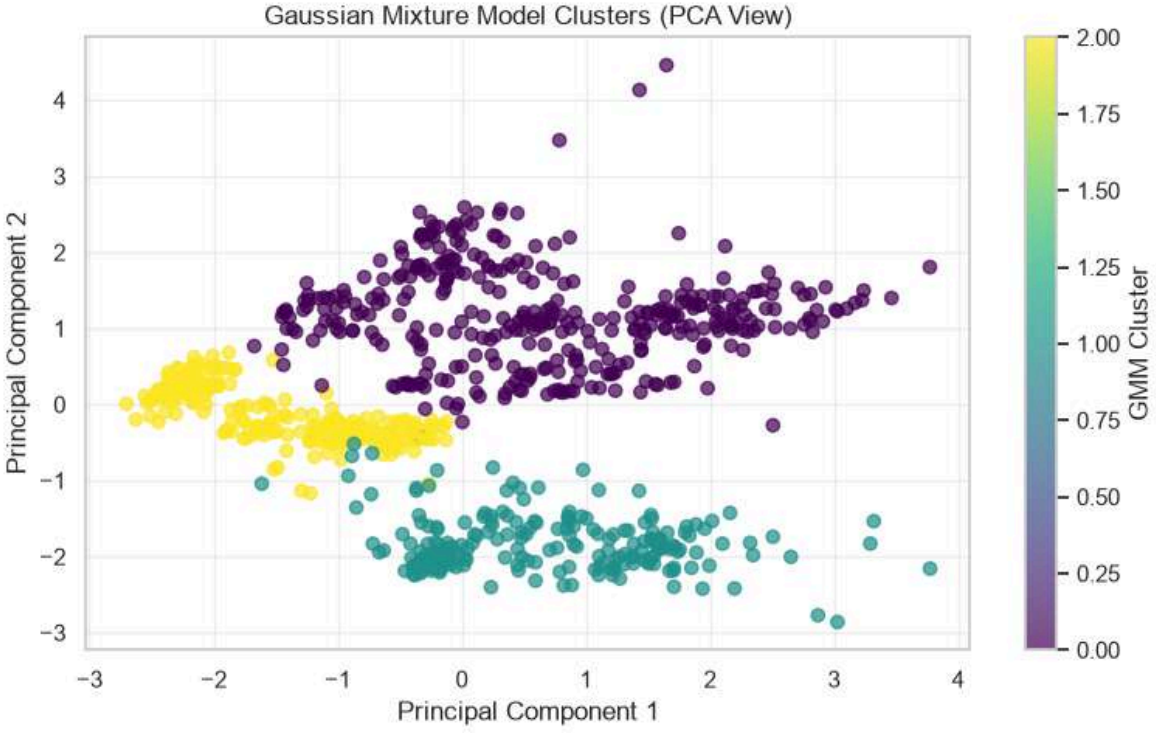
print("GMM BIC:", round(gmm_m5.bic(X_scaled_m5), 3))
print("GMM AIC:", round(gmm_m5.aic(X_scaled_m5), 3))

plt.figure(figsize=(8, 5))
plt.scatter(
    X_pca_m5[:, 0],
    X_pca_m5[:, 1],
    c=gmm_labels_m5,
    cmap="viridis",
    alpha=0.7
)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Gaussian Mixture Model Clusters (PCA View)")
plt.colorbar(label="GMM Cluster")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
GMM cluster sizes:
```

gmm_cluster	Number of records
0	362
1	210
2	268

```
GMM BIC: 8987.462
GMM AIC: 8594.59
```



Detailed description:

Isolation Forest assigns each record an inlier or potential-outlier label. A label of -1 means the model flags a potential outlier; 1 means an inlier.

In [107…

```
# ISOLATION FOREST

from sklearn.ensemble import IsolationForest
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

isolation_m5 = IsolationForest(
    contamination="auto",
    random_state=42
)

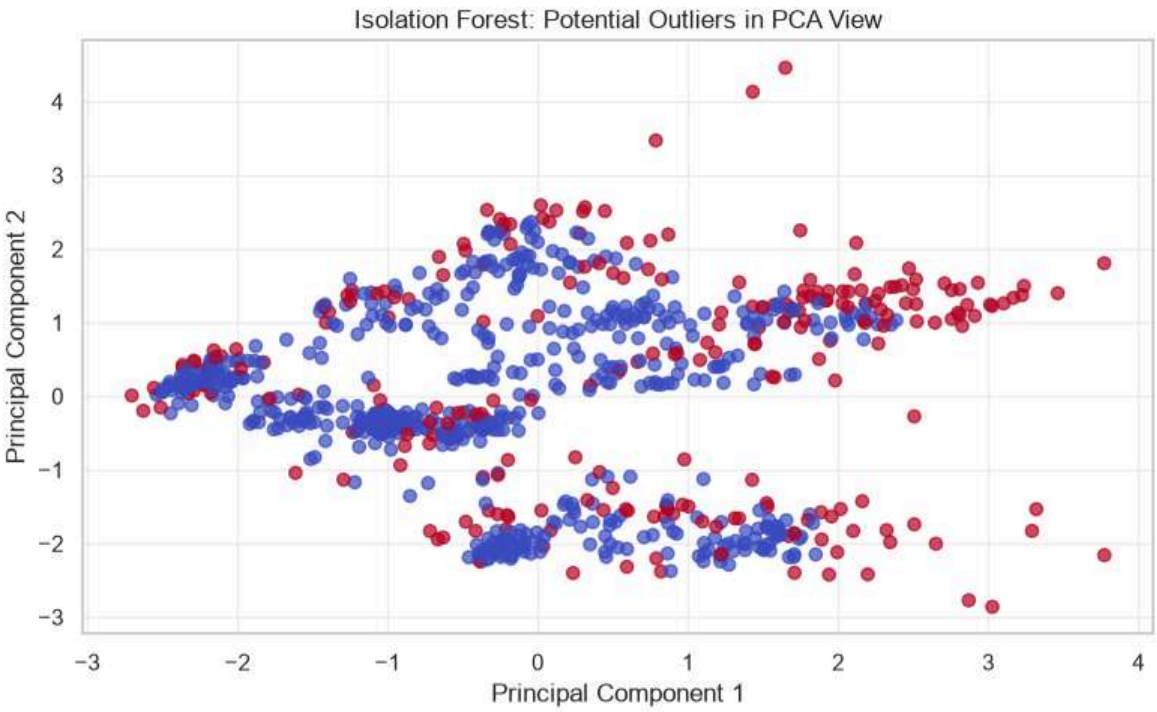
outlier_labels_m5 = isolation_m5.fit_predict(X_scaled_m5)
df_m5["outlier_status"] = np.where(
    outlier_labels_m5 == -1,
    "Potential Outlier",
    "Inlier"
)

print("Outlier detection summary:")
display(
    df_m5["outlier_status"]
    .value_counts()
    .to_frame("Number of records")
)

plt.figure(figsize=(8, 5))
plt.scatter(
    X_pca_m5[:, 0],
    X_pca_m5[:, 1],
    c=(outlier_labels_m5 == -1),
    cmap="coolwarm",
    alpha=0.7
)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Isolation Forest: Potential Outliers in PCA View")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Outlier detection summary:
```

outlier_status	Number of records
Inlier	595
Potential Outlier	245



Conclusion

Focused on applying unsupervised learning techniques to identify patterns in the TravelMode dataset. Numerical features were standardized before applying K-Means, Hierarchical Clustering, DBSCAN, Spectral Clustering, and Gaussian Mixture Models. A dendrogram was used to visualize hierarchical clustering, while PCA helped visualize clusters in two dimensions. DBSCAN identified dense groups and potential noise observations, and Isolation Forest was used to detect potential outliers. Clustering performance was evaluated using silhouette scores, K-Means inertia, and GMM BIC/AIC values. Overall, these methods helped explore similarities among travel records and understand the structure of the dataset. The most suitable algorithm should be selected based on the actual evaluation metrics, cluster separation, and interpretability.

Detailed description:

Creates a separate Module 6 dataset and a scaled feature matrix so that features with larger numerical ranges do not dominate the dimensionality-reduction results.

In [108…

```
# DATA PREPARATION

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler

df_m6 = df_imputed.copy()

features_m6 = [
```

```
    col for col in ["wait", "vcost", "travel", "gcost", "income", "size"]
    if col in df_m6.columns
]

if len(features_m6) < 2:
    raise ValueError("At least two numerical features are required.")

X_m6 = df_m6[features_m6].apply(pd.to_numeric, errors="coerce")
X_m6 = X_m6.replace([np.inf, -np.inf], np.nan)
X_m6 = X_m6.fillna(X_m6.median()).fillna(0)

scaler_m6 = StandardScaler()
X_scaled_m6 = scaler_m6.fit_transform(X_m6)

print("Selected features:", features_m6)
print("Original dataset shape:", df_m6.shape)
print("Scaled feature matrix shape:", X_scaled_m6.shape)
print("Missing values after preprocessing:", np.isnan(X_scaled_m6).sum())
```

```
Selected features: ['wait', 'vcost', 'travel', 'gcost', 'income', 'size']
Original dataset shape: (840, 11)
Scaled feature matrix shape: (840, 6)
Missing values after preprocessing: 0
```

Detailed description:

PCA transforms correlated numerical features into new components that capture as much variation in the data as possible. The explained variance ratio shows how much information each component retains.

In [109…

```
# PRINCIPAL COMPONENT ANALYSIS (PCA)

from sklearn.decomposition import PCA

pca_m6 = PCA()
X_pca_m6 = pca_m6.fit_transform(X_scaled_m6)

pca_variance = pd.DataFrame({
    "Principal Component": [
        f"PC{i+1}" for i in range(len(pca_m6.explained_variance_ratio_))
    ],
    "Explained Variance Ratio": pca_m6.explained_variance_ratio_,
    "Cumulative Variance": np.cumsum(pca_m6.explained_variance_ratio_)
})

display(pca_variance.round(4))

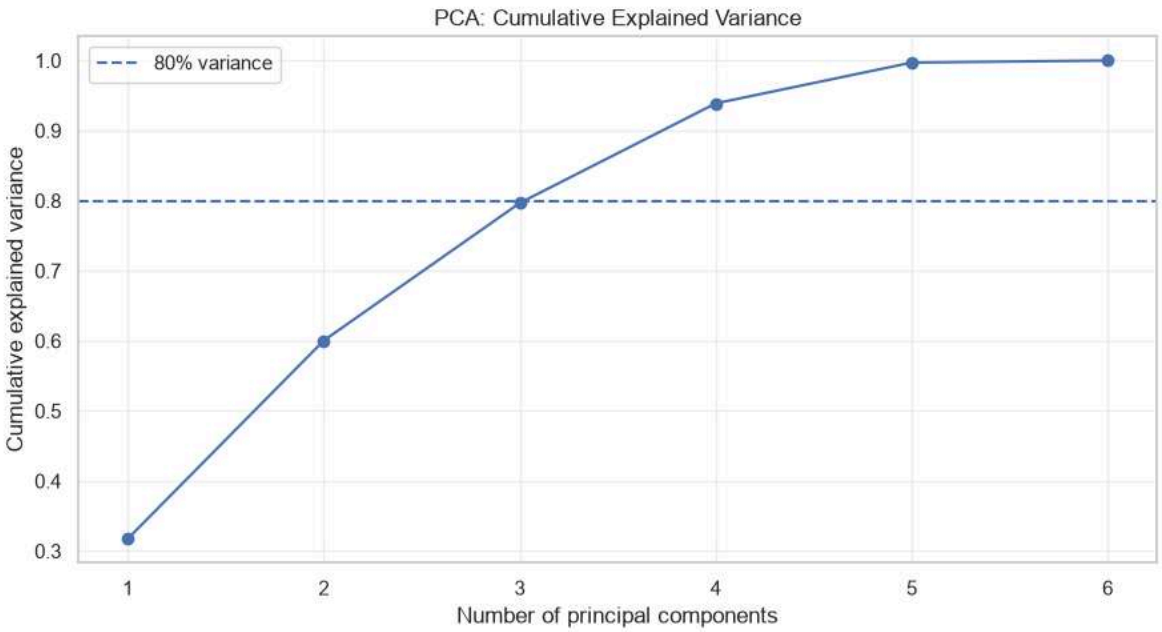
# Plot cumulative explained variance
plt.figure(figsize=(9, 5))
plt.plot(
    range(1, len(pca_m6.explained_variance_ratio_) + 1),
    np.cumsum(pca_m6.explained_variance_ratio_),
    marker="o"
)
plt.axhline(y=0.80, linestyle="--", label="80% variance")
plt.xlabel("Number of principal components")
plt.ylabel("Cumulative explained variance")
```

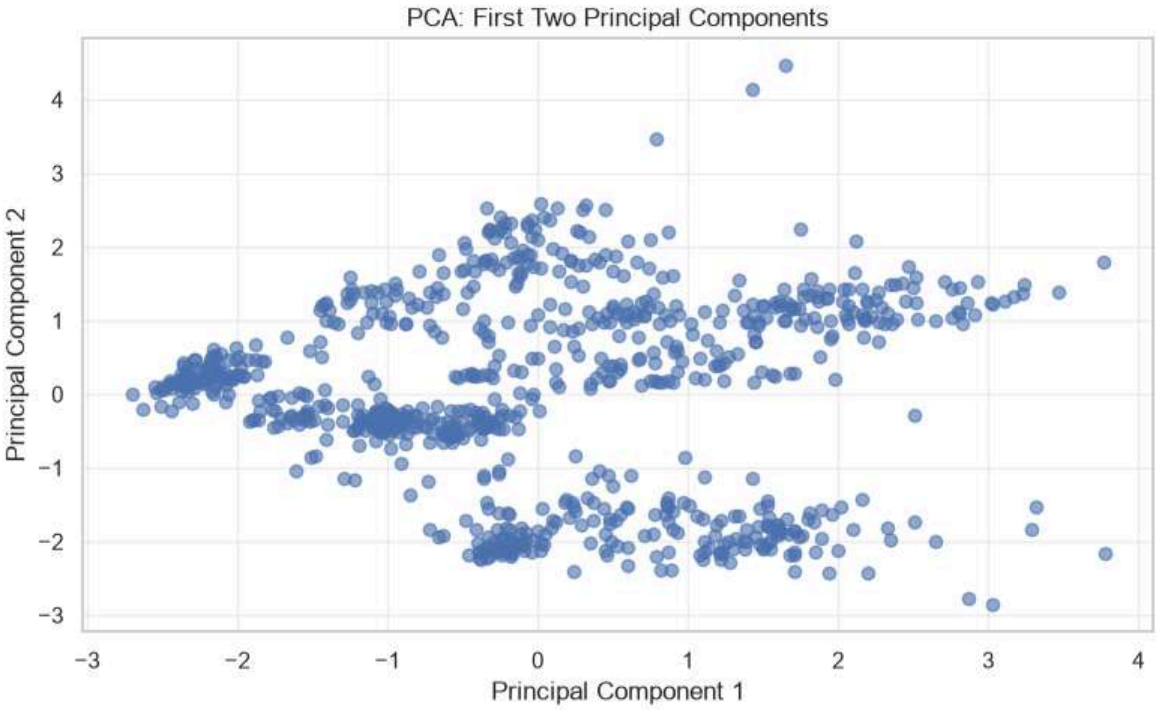
```
plt.title("PCA: Cumulative Explained Variance")
plt.xticks(range(1, len(features_m6) + 1))
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

# Visualize the first two principal components
plt.figure(figsize=(8, 5))
plt.scatter(X_pca_m6[:, 0], X_pca_m6[:, 1], alpha=0.6)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA: First Two Principal Components")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(
    "Variance retained by first two components:",
    round(pca_m6.explained_variance_ratio_[:2].sum(), 4)
)
```

	Principal Component	Explained Variance Ratio	Cumulative Variance
0	PC1	0.3174	0.3174
1	PC2	0.2829	0.6003
2	PC3	0.1963	0.7966
3	PC4	0.1422	0.9388
4	PC5	0.0582	0.9970
5	PC6	0.0030	1.0000






```
Variance retained by first two components: 0.6003
```

Detailed description:

Truncated SVD extracts components from the scaled data and reports the variance captured by them. It is useful for dimensionality reduction, particularly with large or sparse matrices.

In [110…

```python
# SINGULAR VALUE DECOMPOSITION (SVD)

from sklearn.decomposition import TruncatedSVD

n_components_svd = min(2, X_scaled_m6.shape[1] - 1)

svd_m6 = TruncatedSVD(
    n_components=n_components_svd,
    random_state=42
)

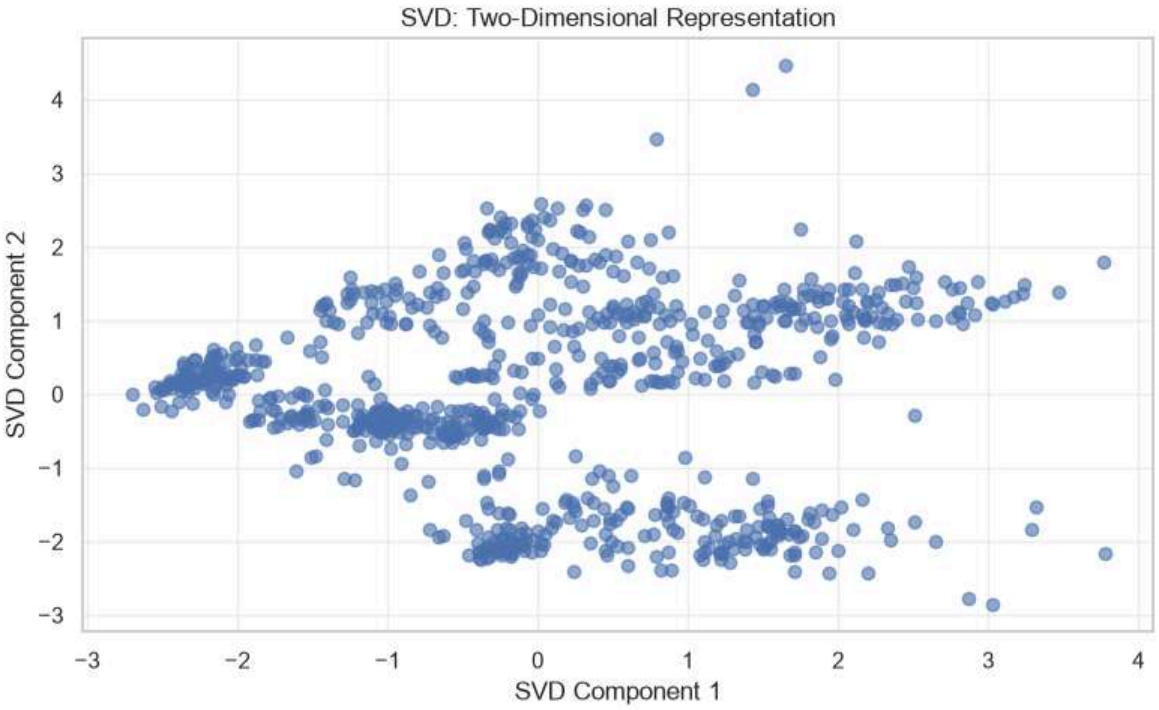
X_svd_m6 = svd_m6.fit_transform(X_scaled_m6)

print("Original feature count:", X_scaled_m6.shape[1])
print("Reduced feature count:", X_svd_m6.shape[1])
print(
    "Explained variance ratio:",
    np.round(svd_m6.explained_variance_ratio_, 4)
)
print(
    "Total variance ratio captured:",
    round(svd_m6.explained_variance_ratio_.sum(), 4)
)

plt.figure(figsize=(8, 5))
plt.scatter(X_svd_m6[:, 0], X_svd_m6[:, 1], alpha=0.6)
plt.xlabel("SVD Component 1")
```

```
plt.ylabel("SVD Component 2")
plt.title("SVD: Two-Dimensional Representation")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Original feature count: 6
Reduced feature count: 2
Explained variance ratio: [0.3174 0.2829]
Total variance ratio captured: 0.6003
```



Detailed description:

Factor Analysis models observed variables as combinations of latent factors and feature-specific noise. Here, two factors are extracted to explore relationships among the standardized travel-related features.

In [111…

```
# FACTOR ANALYSIS

from sklearn.decomposition import FactorAnalysis

n_factors_m6 = min(2, X_scaled_m6.shape[1])

fa_m6 = FactorAnalysis(
    n_components=n_factors_m6,
    random_state=42
)

X_factors_m6 = fa_m6.fit_transform(X_scaled_m6)

factor_loadings_m6 = pd.DataFrame(
    fa_m6.components_.T,
    index=features_m6,
    columns=[f"Factor {i+1}" for i in range(n_factors_m6)]
)
```

```
print("Factor loadings:")
display(factor_loadings_m6.round(3))

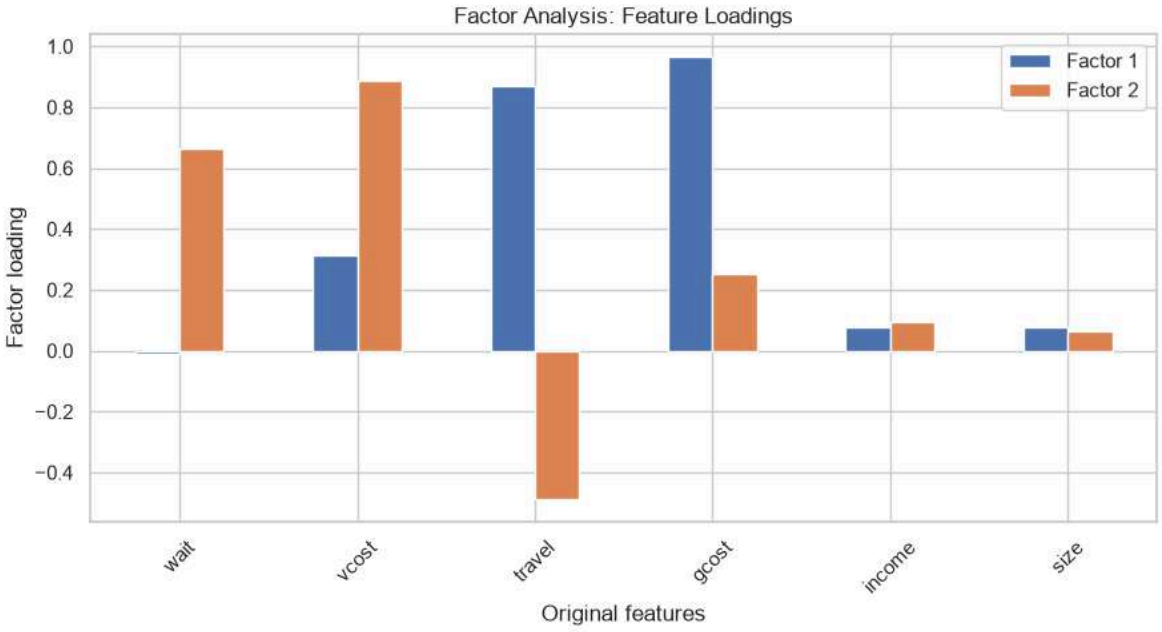
plt.figure(figsize=(9, 5))
factor_loadings_m6.plot(kind="bar", figsize=(9, 5))
plt.title("Factor Analysis: Feature Loadings")
plt.xlabel("Original features")
plt.ylabel("Factor loading")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

print("Factor score matrix shape:", X_factors_m6.shape)
```

```
Factor loadings:
```

	Factor 1	Factor 2
wait	-0.010	0.664
vcost	0.312	0.886
travel	0.869	-0.487
gcost	0.966	0.251
income	0.076	0.094
size	0.079	0.064

```
<Figure size 900x500 with 0 Axes>
```




```
Factor score matrix shape: (840, 2)
```

Detailed description:

Uses PCA's cumulative explained variance to estimate the number of components needed to retain at least 90% of the variation in the standardized features. This is a practical estimate of dimensionality, not a definitive measurement of the dataset's true intrinsic dimension.

In [112…

```
# INTRINSIC DIMENSIONALITY ESTIMATION

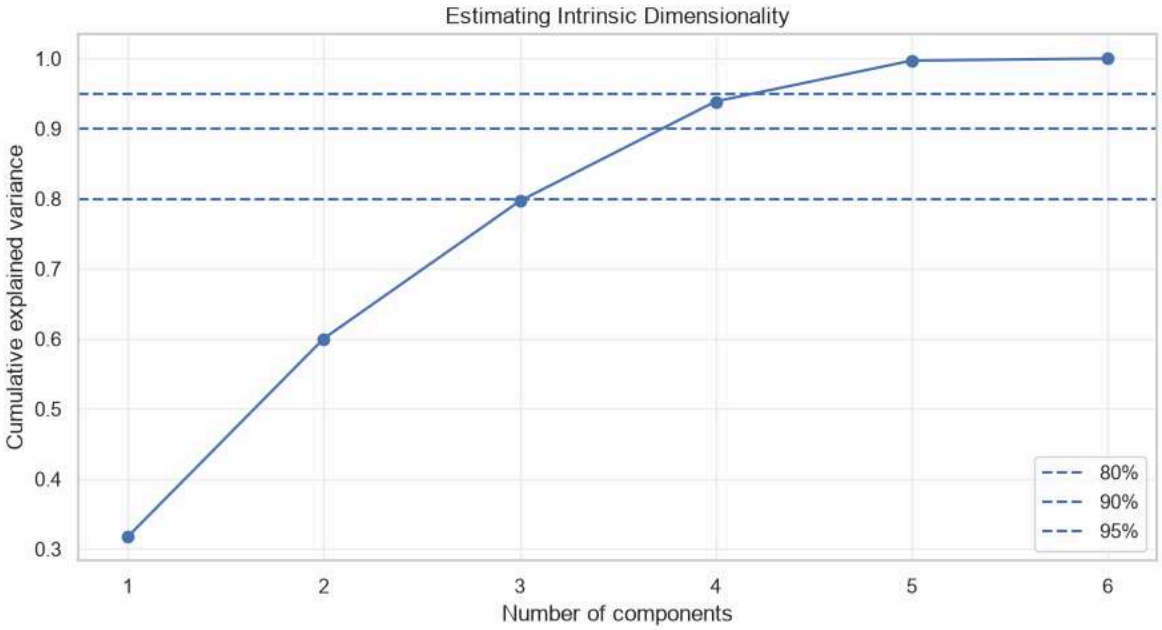
cumulative_variance_m6 = np.cumsum(
    pca_m6.explained_variance_ratio_
)

components_80 = np.searchsorted(cumulative_variance_m6, 0.80) + 1
components_90 = np.searchsorted(cumulative_variance_m6, 0.90) + 1
components_95 = np.searchsorted(cumulative_variance_m6, 0.95) + 1

print("Original number of features:", len(features_m6))
print("Components required for 80% variance:", components_80)
print("Components required for 90% variance:", components_90)
print("Components required for 95% variance:", components_95)

plt.figure(figsize=(9, 5))
plt.plot(
    range(1, len(cumulative_variance_m6) + 1),
    cumulative_variance_m6,
    marker="o"
)
plt.axhline(0.80, linestyle="--", label="80%")
plt.axhline(0.90, linestyle="--", label="90%")
plt.axhline(0.95, linestyle="--", label="95%")
plt.xlabel("Number of components")
plt.ylabel("Cumulative explained variance")
plt.title("Estimating Intrinsic Dimensionality")
plt.xticks(range(1, len(features_m6) + 1))
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

```
Original number of features: 6
Components required for 80% variance: 4
Components required for 90% variance: 4
Components required for 95% variance: 5
```



Detailed description:

MDS is a nonlinear dimensionality-reduction technique that places observations in a low-dimensional space. Since it can be computationally expensive, this cell uses a reproducible sample of up to 300 records.

In [114…

```
# FIX: PREPARE MDS INPUT DATA

import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import pairwise_distances
from sklearn.manifold import MDS

sample_size_m6 = min(300, len(X_scaled_m6))

rng_m6 = np.random.default_rng(42)
sample_indices_m6 = np.sort(
    rng_m6.choice(
        len(X_scaled_m6),
        size=sample_size_m6,
        replace=False
    )
)

X_mds_input = X_scaled_m6[sample_indices_m6]

print("Number of observations selected:", sample_size_m6)
print("MDS input shape:", X_mds_input.shape)
```

```
Number of observations selected: 300
MDS input shape: (300, 6)
```

In [115…

```
# MULTIDIMENSIONAL SCALING (MDS)

import warnings
import matplotlib.pyplot as plt
from sklearn.manifold import MDS

# Hide only FutureWarnings from this MDS cell
warnings.filterwarnings("ignore", category=FutureWarning)

mds_m6 = MDS(
    n_components=2,
    metric_mds=True,
    metric="euclidean",
    random_state=42,
    n_init=1,
    max_iter=100
)

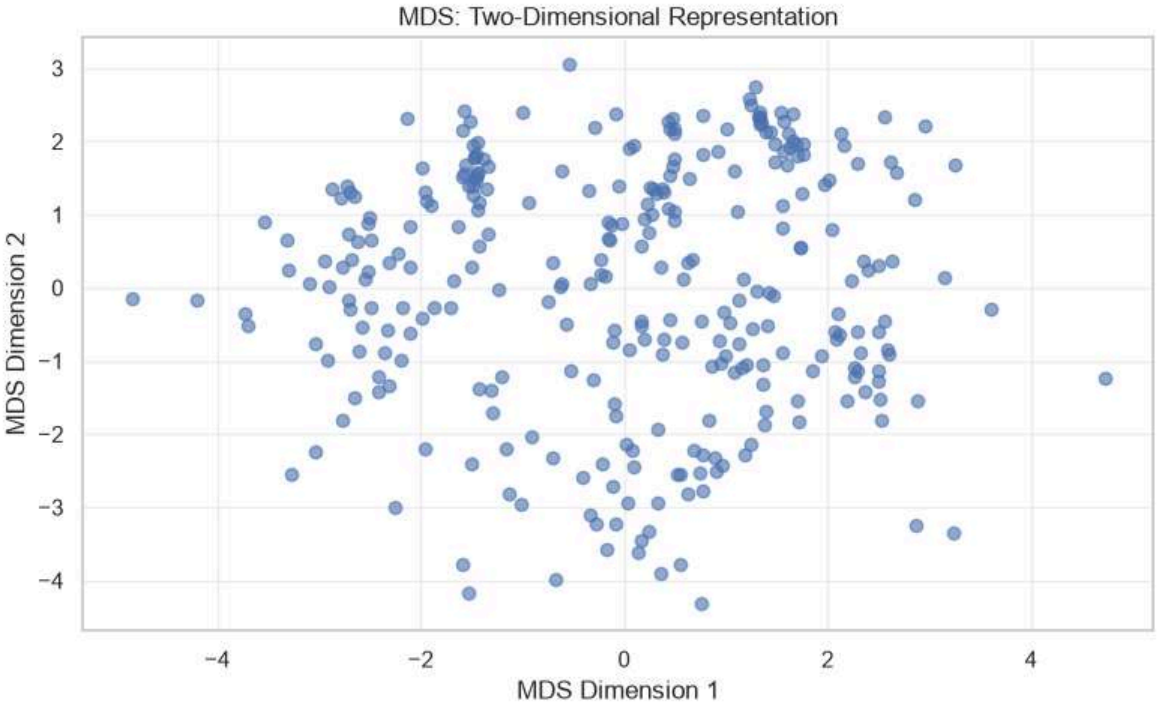
X_mds_m6 = mds_m6.fit_transform(X_mds_input)

print("Number of observations used:", sample_size_m6)
print("MDS stress:", round(mds_m6.stress_, 3))

plt.figure(figsize=(8, 5))
plt.scatter(X_mds_m6[:, 0], X_mds_m6[:, 1], alpha=0.6)
plt.xlabel("MDS Dimension 1")
plt.ylabel("MDS Dimension 2")
plt.title("MDS: Two-Dimensional Representation")
```

```
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Number of observations used: 300
MDS stress: 34164.968
```



Conclusion

Dimensionality reduction was performed on the TravelMode dataset using PCA, SVD, Factor Analysis, MDS, and Isomap. PCA and SVD reduced the number of features while measuring retained variance, and Factor Analysis explored relationships between observed variables and underlying factors. Intrinsic dimensionality was estimated using cumulative explained variance, while MDS and Isomap provided two-dimensional representations of the observations. The evaluation metrics and visualizations helped examine how effectively each technique represented the original data with fewer dimensions.

Detailed description:

Uses numerical variables such as wait, vcost, gcost, income, and size to predict travel. The target is not included among the input features, preventing target leakage.

In [116…

```
# IMPORT LIBRARIES AND PREPARE DATA

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
```

```
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.pipeline import make_pipeline

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    classification_report,
    roc_curve,
    roc_auc_score
)

# Use the cleaned dataset from Module 2
df_m7 = df_imputed.copy()

print("Dataset shape:", df_m7.shape)
print("Available columns:", df_m7.columns.tolist())
display(df_m7.head())
```

```
Dataset shape: (840, 11)
Available columns: ['rownames', 'individual', 'mode', 'choice', 'wait', 'vcost',
'travel', 'gcost', 'income', 'size', 'travel_category']
```

	rownames	individual	mode	choice	wait	vcost	travel	gcost	income	size	travel_
0	1	1	air	no	69	59	100	70	35	1	
1	2	1	train	no	34	31	372	71	35	1	
2	3	1	bus	no	35	25	417	70	35	1	
3	4	1	car	yes	0	10	180	30	35	1	
4	5	2	air	no	64	58	68	68	30	2	

Detailed description:

Uses wait as the single predictor for travel. The test set evaluates how accurately the model predicts travel time for unseen records.

In [117…

```
# SIMPLE LINEAR REGRESSION

if not {"wait", "travel"}.issubset(df_m7.columns):
    raise ValueError("The dataset must contain 'wait' and 'travel'.")

simple_data = df_m7[["wait", "travel"]].apply(
    pd.to_numeric, errors="coerce"
```

```
).replace([np.inf, -np.inf], np.nan).dropna()

X_simple = simple_data[["wait"]]
y_simple = simple_data["travel"]

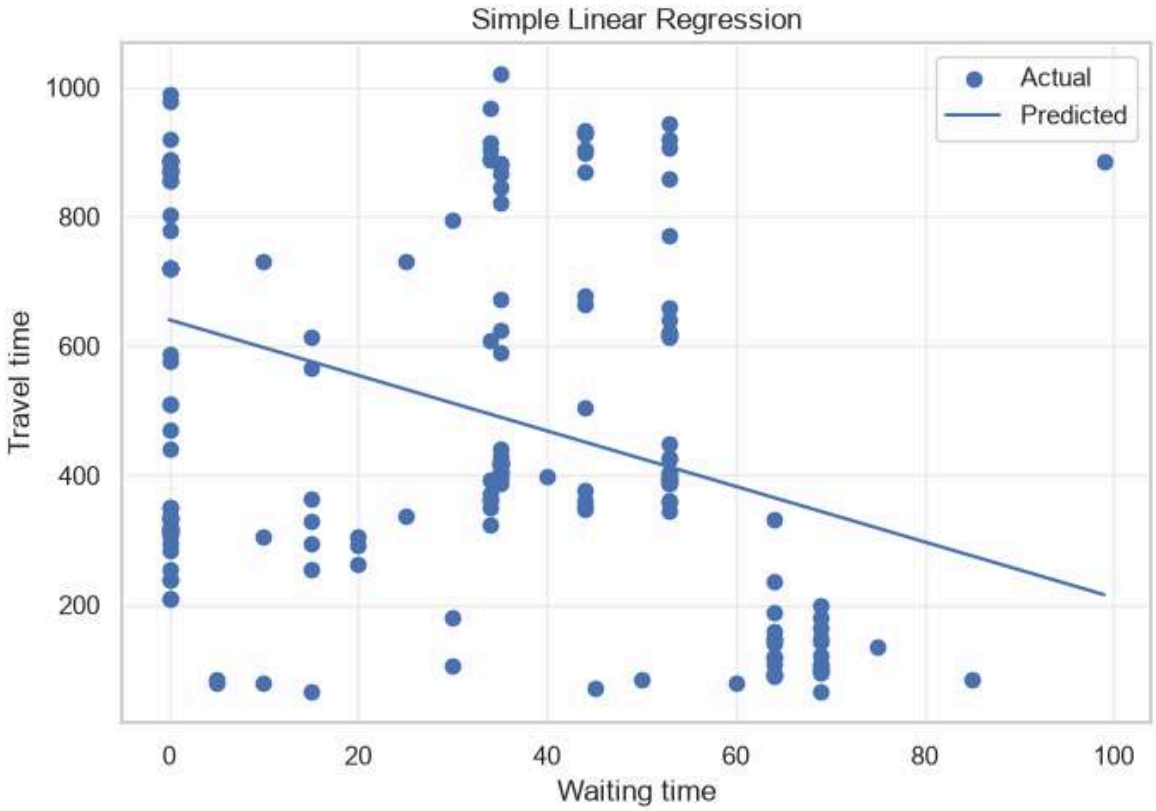
X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X_simple, y_simple, test_size=0.20, random_state=42
)

simple_model = LinearRegression()
simple_model.fit(X_train_s, y_train_s)
y_pred_s = simple_model.predict(X_test_s)

print("Simple Linear Regression")
print("MAE:", round(mean_absolute_error(y_test_s, y_pred_s), 4))
print("RMSE:", round(np.sqrt(mean_squared_error(y_test_s, y_pred_s)), 4))
print("R-squared:", round(r2_score(y_test_s, y_pred_s), 4))

plt.figure(figsize=(7, 5))
plt.scatter(X_test_s["wait"], y_test_s, label="Actual")
order = np.argsort(X_test_s["wait"].to_numpy())
plt.plot(
    X_test_s["wait"].to_numpy()[order],
    y_pred_s[order],
    label="Predicted"
)
plt.xlabel("Waiting time")
plt.ylabel("Travel time")
plt.title("Simple Linear Regression")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Simple Linear Regression
MAE: 233.7845
RMSE: 273.6987
R-squared: 0.0459
```



Detailed description:

Uses available numerical features together to predict travel, then compares the model's predictions against actual test values.

In [118…

```
# MULTIPLE LINEAR REGRESSION

multi_features = [
    c for c in ["wait", "vcost", "gcost", "income", "size"]
    if c in df_m7.columns
]

if not multi_features or "travel" not in df_m7.columns:
    raise ValueError("Required regression columns are missing.")

multi_data = df_m7[multi_features + ["travel"]].apply(
    pd.to_numeric, errors="coerce"
).replace([np.inf, -np.inf], np.nan)

multi_data = multi_data.dropna(subset=["travel"])
X_multi = multi_data[multi_features].fillna(
    multi_data[multi_features].median()
).fillna(0)
y_multi = multi_data["travel"]

X_train_multi, X_test_multi, y_train_multi, y_test_multi = train_test_split(
    X_multi, y_multi, test_size=0.20, random_state=42
)

multiple_model = LinearRegression()
multiple_model.fit(X_train_multi, y_train_multi)
y_pred_multi = multiple_model.predict(X_test_multi)
```

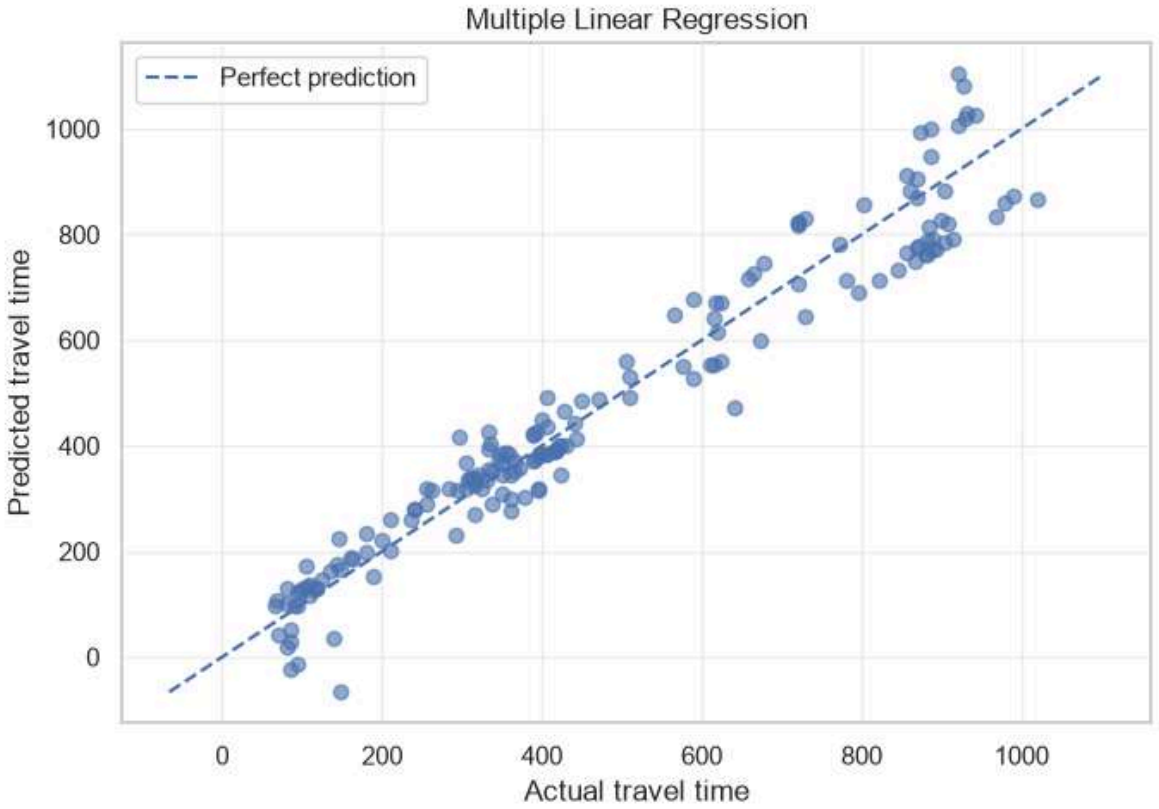
```
print("Multiple Linear Regression")
print("MAE:", round(mean_absolute_error(y_test_multi, y_pred_multi), 4))
print("RMSE:", round(np.sqrt(mean_squared_error(y_test_multi, y_pred_multi)), 4)
print("R-squared:", round(r2_score(y_test_multi, y_pred_multi), 4))

display(pd.DataFrame({
    "Feature": multi_features,
    "Coefficient": multiple_model.coef_
}).round(4))

plt.figure(figsize=(7, 5))
plt.scatter(y_test_multi, y_pred_multi, alpha=0.6)
low = min(y_test_multi.min(), y_pred_multi.min())
high = max(y_test_multi.max(), y_pred_multi.max())
plt.plot([low, high], [low, high], "--", label="Perfect prediction")
plt.xlabel("Actual travel time")
plt.ylabel("Predicted travel time")
plt.title("Multiple Linear Regression")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Multiple Linear Regression
MAE: 54.8577
RMSE: 68.2228
R-squared: 0.9407
```

	Feature	Coefficient
0	wait	-0.8022
1	vcost	-6.9100
2	gcost	7.1315
3	income	-0.0895
4	size	-55.5490



Detailed description:

Converts the choice labels into numeric classes and selects numerical predictors. This cell supports binary or multiclass classification, so it won't fail simply because choice contains more than two classes.

In [119…

```
# PREPARE CLASSIFICATION DATA

if "choice" not in df_m7.columns:
    raise ValueError("The dataset does not contain a 'choice' column.")

class_features = [
    c for c in ["wait", "vcost", "travel", "gcost", "income", "size"]
    if c in df_m7.columns
]

class_data = df_m7[class_features + ["choice"]].copy()
class_data["choice"] = class_data["choice"].astype("string").str.strip()
class_data = class_data.replace([np.inf, -np.inf], np.nan)
class_data = class_data.dropna(subset=["choice"])
class_data = class_data[class_data["choice"] != ""]

X_class = class_data[class_features].apply(pd.to_numeric, errors="coerce")
X_class = X_class.fillna(X_class.median()).fillna(0)

label_encoder = LabelEncoder()
y_class = label_encoder.fit_transform(class_data["choice"].astype(str))

print("Classification features:", class_features)
print("Class labels:", list(label_encoder.classes_))
```

```
print("Class distribution:")
display(class_data["choice"].value_counts().to_frame("Record count"))
```

```
Classification features: ['wait', 'vcost', 'travel', 'gcost', 'income', 'size']
Class labels: ['no', 'yes']
Class distribution:
```

choice	Record count
no	630
yes	210

Detailed description:

Splits the data while preserving class proportions where possible. Each model predicts choice on the same test records for a fair comparison.

In [120…

```
# TRAIN CLASSIFICATION MODELS

if len(label_encoder.classes_) < 2:
    raise ValueError("Classification requires at least two classes.")

X_train_class, X_test_class, y_train_class, y_test_class = train_test_split(
    X_class,
    y_class,
    test_size=0.20,
    random_state=42,
    stratify=y_class
)

models = {
    "Logistic Regression": make_pipeline(
        StandardScaler(),
        LogisticRegression(max_iter=2000, random_state=42)
    ),
    "Decision Tree": DecisionTreeClassifier(
        max_depth=5, random_state=42
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=100, random_state=42
    ),
    "SVM": make_pipeline(
        StandardScaler(),
        SVC(random_state=42)
    )
}

trained_models = {}
predictions = {}
results = []

for name, model in models.items():
    model.fit(X_train_class, y_train_class)
    pred = model.predict(X_test_class)
```

```
    trained_models[name] = model
    predictions[name] = pred

    results.append({
        "Algorithm": name,
        "Accuracy": accuracy_score(y_test_class, pred),
        "Precision (weighted)": precision_score(
            y_test_class, pred, average="weighted", zero_division=0
        ),
        "Recall (weighted)": recall_score(
            y_test_class, pred, average="weighted", zero_division=0
        ),
        "F1-score (weighted)": f1_score(
            y_test_class, pred, average="weighted", zero_division=0
        )
    })

print("All classification models trained successfully.")
```

```
All classification models trained successfully.
```

Detailed description:

The confusion matrices show how many observations from each actual class were assigned to each predicted class. Correct predictions appear along the diagonal; off-diagonal values are classification errors.

In [121…

```
# CONFUSION MATRICES FOR ALL MODELS

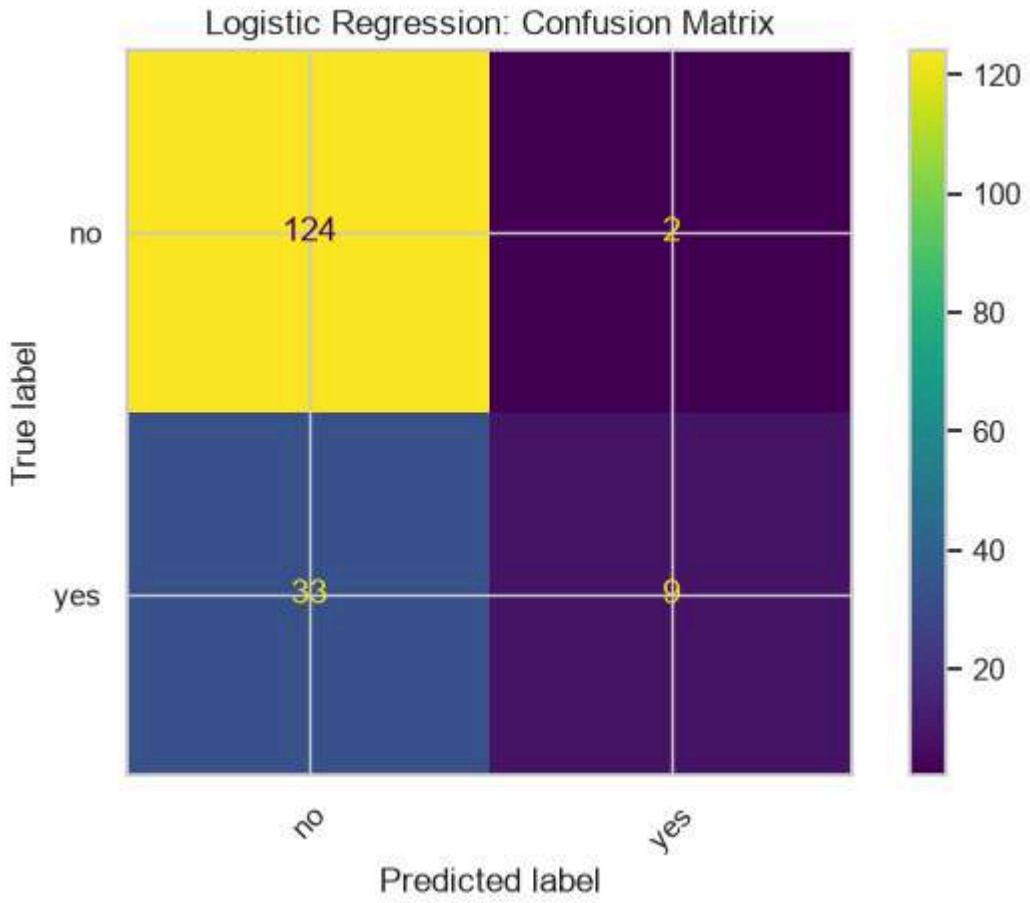
for name, pred in predictions.items():
    cm = confusion_matrix(
        y_test_class,
        pred,
        labels=np.arange(len(label_encoder.classes_))
    )

    print(f"\n{name} — Confusion Matrix")
    display(pd.DataFrame(
        cm,
        index=[f"Actual: {c}" for c in label_encoder.classes_],
        columns=[f"Predicted: {c}" for c in label_encoder.classes_]
    ))

    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=label_encoder.classes_
    )
    disp.plot(values_format="d", xticks_rotation=45)
    plt.title(f"{name}: Confusion Matrix")
    plt.tight_layout()
    plt.show()
```

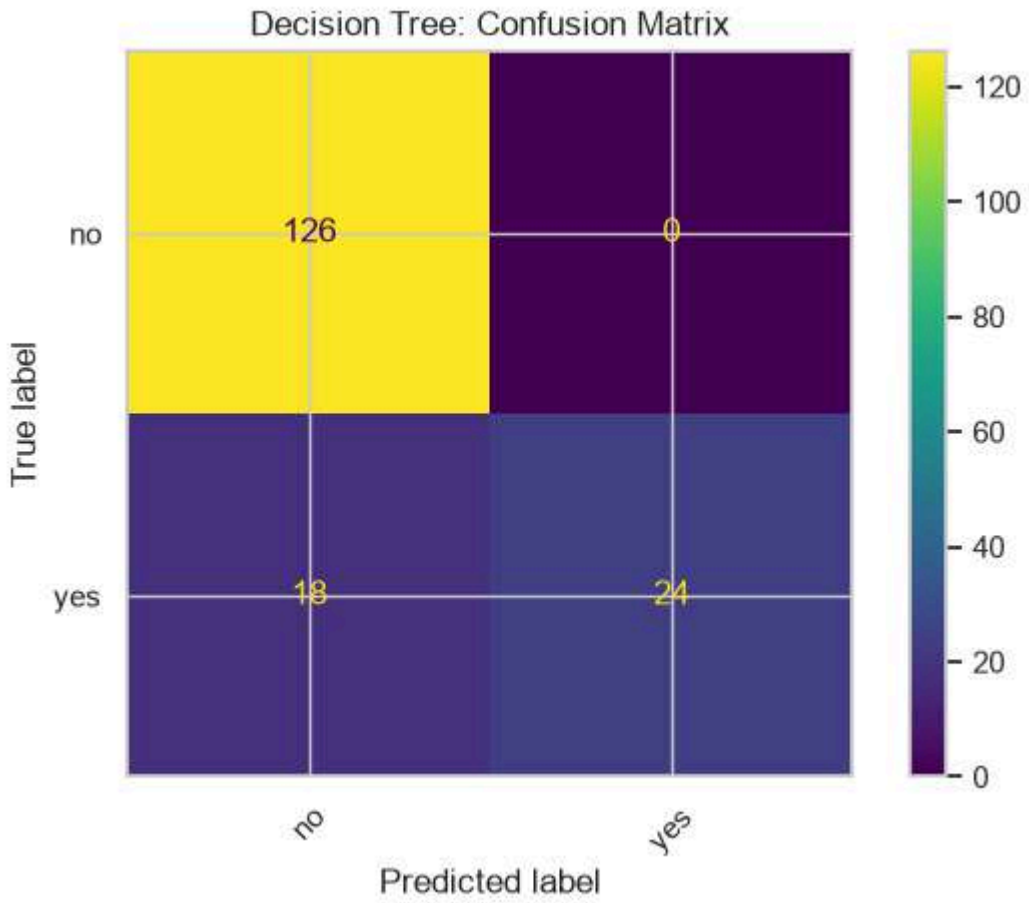
```
Logistic Regression — Confusion Matrix
```

	Predicted: no	Predicted: yes
Actual: no	124	2
Actual: yes	33	9



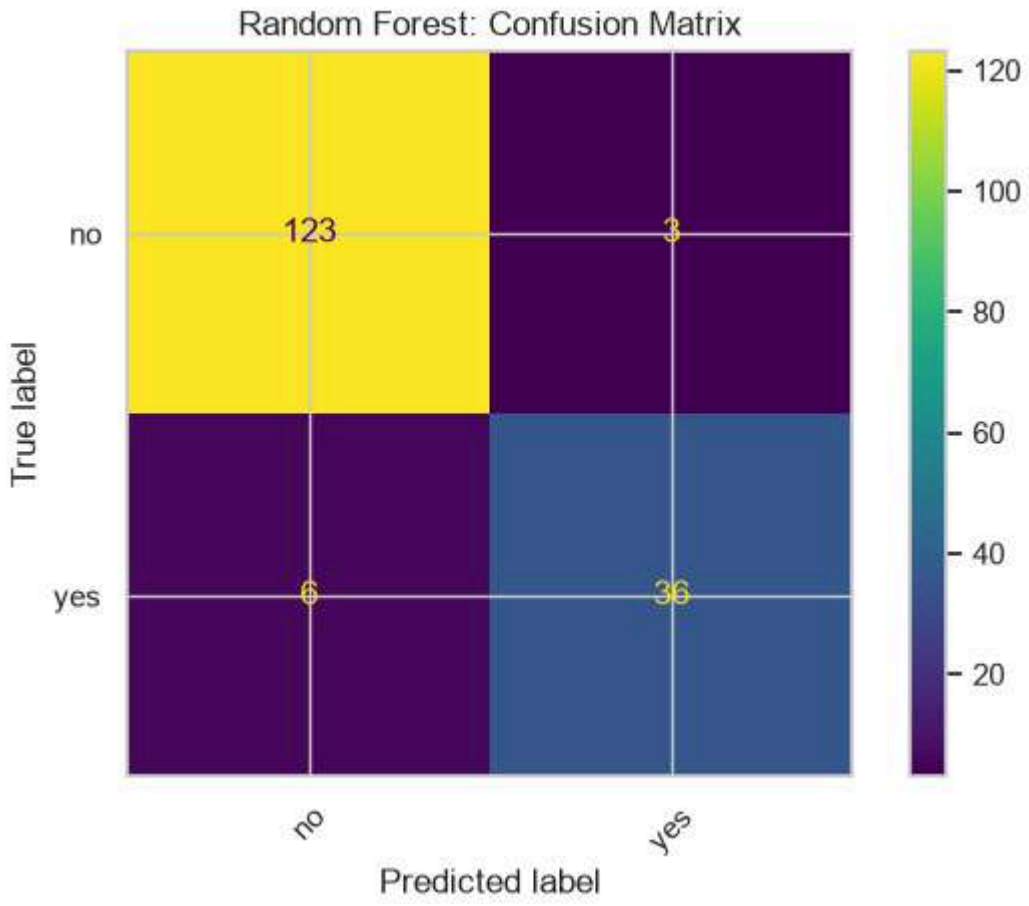
Decision Tree — Confusion Matrix

	Predicted: no	Predicted: yes
Actual: no	126	0
Actual: yes	18	24



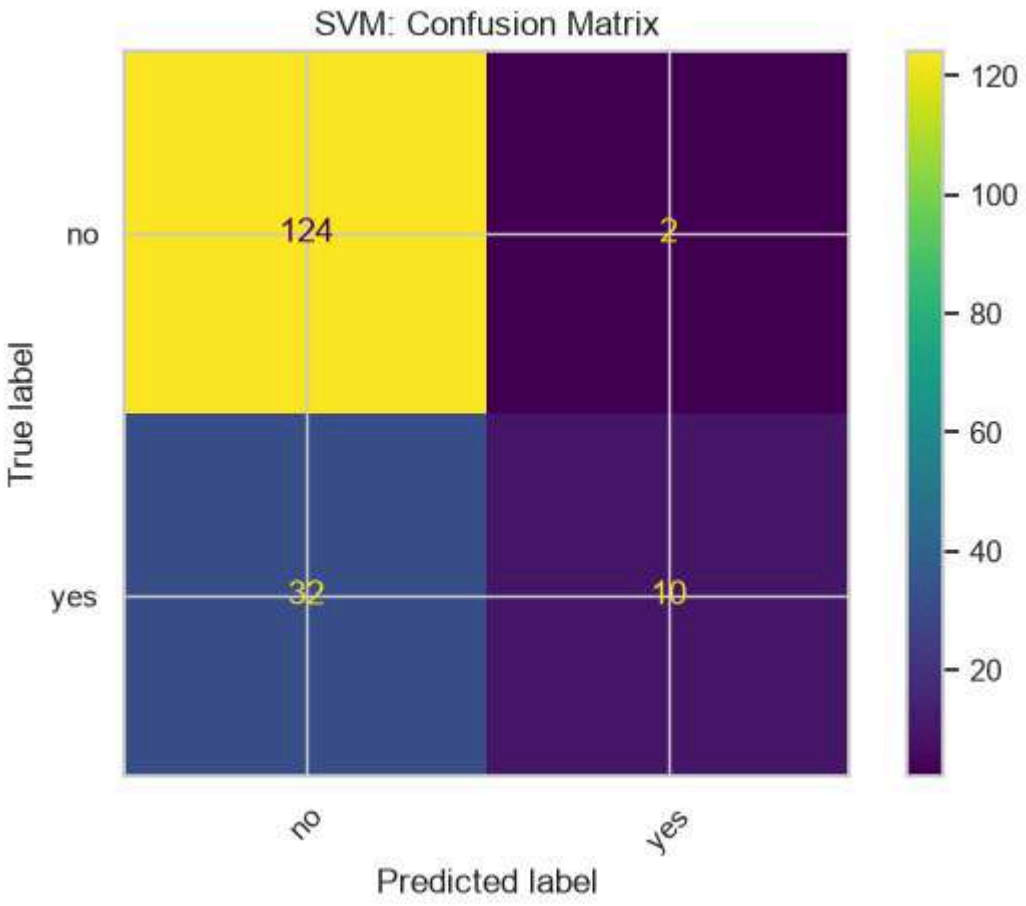
Random Forest — Confusion Matrix

	Predicted: no	Predicted: yes
Actual: no	123	3
Actual: yes	6	36



SVM — Confusion Matrix

	Predicted: no	Predicted: yes
Actual: no	124	2
Actual: yes	32	10



Detailed description:

Creates a single table so you can identify the strongest model according to the metrics rather than relying on the confusion matrices alone.

In [122…

```
# CLASSIFICATION METRICS COMPARISON

comparison_models = pd.DataFrame(results)

display(
    comparison_models
    .sort_values("F1-score (weighted)", ascending=False)
    .round(4)
)

best_model_name = comparison_models.sort_values(
    "F1-score (weighted)", ascending=False
).iloc[0]["Algorithm"]

print("Highest weighted F1-score:", best_model_name)
```

	Algorithm	Accuracy	Precision (weighted)	Recall (weighted)	F1-score (weighted)
2	Random Forest	0.9464	0.9459	0.9464	0.9458
1	Decision Tree	0.8929	0.9062	0.8929	0.8818
3	SVM	0.7976	0.8045	0.7976	0.7522
0	Logistic Regression	0.7917	0.7969	0.7917	0.7421

```
Highest weighted F1-score: Random Forest
```

Justification:

Random Forest was selected as the best-performing classification model for the TravelMode dataset because it achieved the highest accuracy of 94.64%, weighted precision of 94.59%, weighted recall of 94.64%, and weighted F1-score of 94.58% among the four evaluated algorithms. Its ensemble approach combines multiple decision trees, helping it capture complex patterns in the dataset. Therefore, based on the observed test-set metrics, Random Forest is the preferred model for predicting travel-mode choice.

Detailed description:

This cell generates an ROC curve only when choice contains exactly two classes. For multiclass targets, ROC analysis requires a different approach.

In [123…

```
# ROC CURVE AND AUC

if len(label_encoder.classes_) == 2:
    logistic_for_roc = trained_models["Logistic Regression"]

    probabilities = logistic_for_roc.predict_proba(X_test_class)[:, 1]

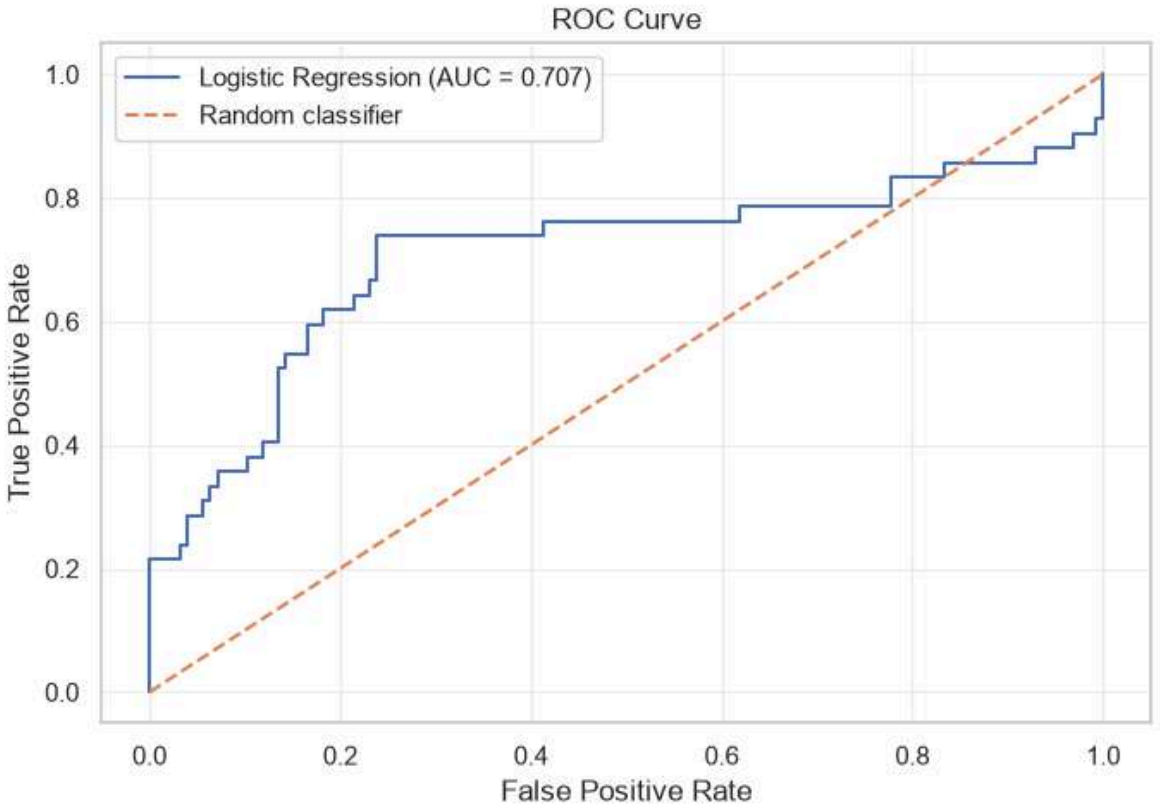
    fpr, tpr, thresholds = roc_curve(y_test_class, probabilities)
    auc_value = roc_auc_score(y_test_class, probabilities)

    print("ROC-AUC:", round(auc_value, 4))

    plt.figure(figsize=(7, 5))
    plt.plot(fpr, tpr, label=f"Logistic Regression (AUC = {auc_value:.3f})")
    plt.plot([0, 1], [0, 1], "--", label="Random classifier")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("ROC Curve")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
else:
    print(
        "ROC curve skipped: choice has more than two classes. "
```

```
        "Use multiclass ROC with one-vs-rest if required."
    )
```

```
ROC-AUC: 0.7065
```



Conclusion

Implemented Simple Linear Regression and Multiple Linear Regression to predict travel time, followed by Logistic Regression, Decision Tree, Random Forest, and SVM to classify travel-mode choice. Regression performance was evaluated using MAE, RMSE, and R^2, while classification performance was evaluated using confusion matrices, accuracy, precision, recall, weighted F1-score, and ROC-AUC where applicable. Actual-versus-predicted plots and confusion-matrix heatmaps helped visualize model performance. The most suitable model should be selected based on the actual evaluation results and the project's objective.

In [ ]: